Machine Learning

From Experience to Classifiers

A textbook treatment of Lectures 1 to 5

Based on the lecture notes and slides of Mounir Ghogho

October 25, 2026

Contents

How to read this book **3**

1 What Does It Mean to Learn? **4**
1.1 Learning from Experience . . . 4
1.2 Two Routes to Machine Ability . . . 5
1.3 What Counts as Experience? . . . 9
1.4 Supervised Learning: Learning from Examples . . . 11
1.5 Representations . . . 13
1.6 Four Ingredients of a Learning System . . . 16
1.7 The Challenge of Generalization . . . 18
1.8 A Connection to Information Theory . . . 21
Notes on the original lecture notes . . . 23
Chapter takeaways . . . 24

2 From Representations to Decisions **25**
How to read the formulas in this chapter . . . 25
2.1 From Objects to Numerical Representations . . . 26
2.2 Decision Making When the Distribution Is Known . . . 28
2.3 From Known Distributions to Finite Data . . . 35
2.4 Parameterized Models . . . 40
2.5 Learning the Parameters from Data . . . 42
2.6 Evaluating a Learned Model . . . 48
2.7 The Learning Problem in One Picture . . . 51
Notes on the original slides and notes . . . 52
Chapter takeaways . . . 53

3 From Fitting to Generalization **54**
How to read the formulas in this chapter . . . 54
3.1 Generalization and Inductive Bias . . . 54
3.2 Model Capacity . . . 57
3.3 Bias and Variance . . . 59
3.4 Controlling Complexity . . . 62
3.5 Feature and Model Selection . . . 65
3.6 Reliable Evaluation and Data Splitting . . . 67
3.7 Diagnosis and Lab Preparation . . . 70
Notes on the original slides and notes . . . 71
Chapter takeaways . . . 72

4 Discriminative and Generative Classification **73**
How to read the formulas in this chapter . . . 73
4.1 Two Routes to the Posterior . . . 73
4.2 The Discriminative Route . . . 75

4.3 The Generative Route . . . 79
4.4 Gaussian Class-Conditional Models . . . 82
4.5 Naive Bayes . . . 86
4.6 Comparing the Two Routes . . . 90
4.7 Class Imbalance and Prior Shift . . . 91
4.8 Unfamiliar Inputs and Combined Systems . . . 93
4.9 Model Choice and Lab Preparation . . . 94
Notes on the original slides and notes . . . 95
Chapter takeaways . . . 95

5 Learning by Combining Models **97**
How to read the formulas in this chapter . . . 97
5.1 Why Combine Models? . . . 98
5.2 Averaging and Voting . . . 99
5.3 Decision Trees as Base Learners . . . 102
5.4 Bagging and Out-of-Bag Evaluation . . . 104
5.5 Random Forests . . . 106
5.6 Feature Importance . . . 108
5.7 Boosting for Regression . . . 110
5.8 Boosting for Binary Classification . . . 113
5.9 XGBoost: One Objective for Structure and Leaf Values . . . 116
5.10 Multi-Class Boosting and Other Libraries . . . 119
5.11 Bagging versus Boosting, and When Ensembles Do Not Help . . . 120
5.12 Stacking . . . 121
5.13 Mixture of Experts . . . 123
5.14 Where Ensembles Are Used, and Lab 3 . . . 124
Notes on the original slides . . . 124
Chapter takeaways . . . 125

How to read this book

Each concept in this book is presented in the same rhythm.

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Basic probability (random variables, conditional distributions, expectations), vectors in $\mathbb{R}^d$, and a first course in calculus. The information-theoretic quantities used (entropy, cross-entropy, Kullback–Leibler divergence) are defined and proved from scratch where they appear.

Chapter 1

What Does It Mean to Learn?

1.1 Learning from Experience

What are we trying to get?

We want machines that can *do things we cannot fully describe*: recognise a handwritten digit, flag a fraudulent payment, translate a sentence. This section pins down what "learning" means so that we can later ask, precisely, what a machine needs in order to do it.

Motivation

Learning is what happens between "could not do it" and "can do it", and experience is the fuel.

A child learns to understand language and to recognise faces. A musician becomes fluent through practice. A student becomes able to solve problems that were unfamiliar a month earlier. In each case something changed in the learner, and the change was driven by *experience*: observation, practice, interaction, instruction, feedback. But experience alone is not enough. A stone exposed to a thousand piano lessons learns nothing. The learner also brings a *capacity to change* in response to experience. For humans and animals this capacity comes from a biological system shaped by evolution and development: the brain and nervous system provide structures that support perception, memory and adaptation.

A first picture, which we will refine throughout the course, is

$$\text{Capacity to Learn} + \text{Experience} \xrightarrow{\text{Learning}} \text{Acquired Abilities}.$$

Definition 1.1.1 (Learning, informal)**.** *Learning* is the process through which experience changes a learner and develops what it can do. The capacity to learn is itself shaped by development and previous experience, so the picture above is a starting point rather than a complete description of biological learning.

What kinds of abilities do we learn?

Many acquired abilities involve four broad activities. They overlap, and they are not an exhaustive classification of intelligence, but they give us a vocabulary that also applies to artificial systems.

Recognise	**Predict**	**Reason**	**Act**
What is it?	What may happen?	What follows?	What should I do?

- *Recognition* identifies objects, people, sounds, words and patterns despite variations in how they appear. You recognise your friend from behind, in poor light, wearing a hat.
- *Prediction* uses previous experience and current information to anticipate what may happen: dark clouds suggest rain, a tilting glass suggests a spill.
- *Reasoning* works through a new problem rather than recalling a stored answer: solving an equation you have never seen, planning a route around a road closure.
- *Acting* includes physical and practical skills acquired through repeated interaction: riding a bicycle, pouring water without spilling.

Language draws on several of these at once: recognising words, predicting what comes next, reasoning about meaning, and acting by producing speech.

Example 1.1.2 (One task, four abilities)**.** A radiologist reading a chest X-ray *recognises* anatomical structures and abnormal shadows, *predicts* how a lesion may evolve, *reasons* from symptoms and image to a differential diagnosis, and *acts* by ordering a test or a treatment. A machine-learning system for radiology may take on any subset of these: a classifier that flags suspicious regions addresses recognition; a model of tumour growth addresses prediction; a treatment-recommendation system addresses decision and action. Keeping the four activities separate helps state precisely what such a system is supposed to do.

This raises the question that has driven much of artificial intelligence:

How can we give machines similar abilities?

Summary: Learning from experience

- Learning = capacity to learn + experience → acquired ability.
- Abilities we care about: recognise, predict, reason, act. Language combines them.
- The question of the course: what does a *machine* need in order to acquire abilities from experience?

1.2 Two Routes to Machine Ability

What are we trying to get?

There are two ways to get a machine to do something: tell it *how* (write the rules), or give it *experience* and let it work the rules out. We want to understand when each route is the right one, and why the second route exists at all.

The first idea: encode human knowledge

Route 1: humans learn first, then transfer the result to the machine as explicit instructions.

One approach is to let humans acquire the knowledge and then transfer it to a machine. The idea predates modern computers: Pascal's seventeenth-century *Pascaline* embodied the procedures of arithmetic in a physical mechanism. Pro-

grammable computers later allowed far richer procedures to be expressed as algorithms. The principle is

$$\text{Human Knowledge} \longrightarrow \text{Explicit Rules or Procedures} \longrightarrow \text{Computer}.$$

This remains the right approach for many problems. When we know a reliable procedure for sorting numbers, calculating a tax, checking a password, or solving a well-defined numerical problem, we implement it directly. Learning it again from examples would only add uncertainty.

Example 1.2.1 (Programs that should stay programs)**.**

- *Income tax.* The rule is written in law: brackets, rates, deductions. A program applies it exactly. A learned model would approximate the law from past tax returns and be wrong on rare cases. Nobody wants an approximately correct tax bill.
- *Sorting.* Merge sort is proved correct for every input. There is nothing to learn.
- *Password check.* The procedure is "hash the input and compare." A learned model would have to be right *every* time, which learning never guarantees.

Early artificial intelligence extended the first idea to domains requiring expertise. In an *expert system*, knowledge obtained from specialists was represented as explicit facts and rules. A medical system could contain rules relating symptoms and test results to diagnoses. Systems such as MYCIN showed that carefully constructed rule bases could support sophisticated reasoning in restricted domains. As such systems became more ambitious, extracting, expressing and maintaining the required knowledge became increasingly difficult. This became known as the *knowledge acquisition bottleneck*.

Example 1.2.2 (The knowledge engineer's year)**.** Imagine building, in the 1980s, an expert system for diagnosing faults in industrial machines, with two hours per week of access to an experienced engineer. After months the system contains hundreds of rules such as

```
IF motor temperature is high AND vibration is strong THEN
                    inspect the bearings.
```

Three kinds of difficulty appear, and it is worth separating them.

- *Obtaining the knowledge.* The engineer "just knows" that a certain rattle means a loose coupling, and cannot say what "that rattle" is in terms a program can test. Two hours a week is far too little to elicit thousands of such judgements.
- *Representing it as rules.* Real judgements are graded ("fairly high temperature"), interacting ("unless the machine has just started"), and full of exceptions. Encoding them produces long chains of rules that contradict each other in corners nobody anticipated.
- *Unfamiliar situations.* When the factory installs a new generation of machines, faults appear that the engineer has never seen. The rules say nothing, and there is no mechanism for the system to improve from the new failures it observes.

A learning-based approach attacks all three: it extracts patterns from logged sensor readings and repair records rather than from interviews, it represents graded and interacting effects numerically, and it can be retrained on the new machines' data. Expert knowledge remains valuable nonetheless: it tells us *which* measurements to record, it can check whether a learned model's behaviour is physically plausible, and it covers rare but catastrophic faults for which no data exist.

Can we always write the rules?

The gap: we can do *things we cannot* describe.

The difficulty is sharpest for abilities that humans perform easily without being able to describe the procedure they use. Consider handwritten-digit recognition. People recognise the same digit across large changes in handwriting, position, orientation, size and stroke thickness. Writing a program that covers these variations through explicit rules is much harder.

Example 1.2.3 (Trying to write the rules for a "3")**.** You are given a 20×20 binary image and must decide whether it shows a 3. Some natural rules:

R1. There are two "bumps" opening to the left: the rightmost black pixel of the top half and the rightmost black pixel of the bottom half both lie in the right third of the image.

R2. The middle row contains black pixels only in the right half (the "waist" of the 3).

R3. The number of black pixels lies between 40 and 90.

R4. No black pixels touch the left border (a 3 has no left vertical stroke, unlike an 8 or a 0).

Now shift the digit four pixels to the right: R1 still holds, R2 may fail if the waist crosses the centre column, R4 holds. Write it narrowly: R1 may fail because the bumps no longer reach the right third. Rotate by 15°: the waist is no longer on the middle row, so R2 fails. Draw it with a thick stroke: R3 fails because the pixel count doubles.

Every fix creates new problems. "Right third" becomes "right third of the bounding box," which requires computing the bounding box, which is corrupted by a stray dot. "Middle row" becomes "the row of minimum horizontal extent between the two bumps," which is ambiguous for a flattened 3 and also matches some 5s and 8s. The pixel-count range widens until it excludes nothing. Rules pile up, each with parameters nobody knows how to set, and the interactions between rules become impossible to reason about. At this point, showing the computer a few thousand labelled 3s and non-3s, and letting it find its own regularities, is the sensible move. The thought experiment reveals the essential difference: explicit programming requires us to *know and state* the procedure; learning requires only that we can *recognise* correct behaviour on examples.

Human vision is the broader version of the same point. We know a great deal about the visual system, yet we do not possess a complete explicit description of the computations that allow people to recognise the enormous variety of objects in natural scenes. This illustrates a distinction that the whole field rests on:

Having an ability $\neq$ having an explicit procedure that describes the ability.

The second idea: make the machine learn

When useful experience is available while a complete procedure is hard to specify, another approach becomes attractive. Instead of transferring the *result* of human learning to a computer, we ask whether the machine can acquire the ability from experience itself. The question changes from "what rules should humans write?" to

How can we give a machine the capacity to learn from experience?

The picture of Section 1.1 now applies to machines:

$$\text{Capacity to Learn} + \text{Experience} \xrightarrow{\text{Learning}} \text{Acquired Ability}.$$

Designing computational methods that make this possible is the subject of *machine learning*. The capacity to learn is provided by design choices: what the system can represent, what can change through experience, what information guides those changes, and what counts as good performance. Machine learning still involves programming. Human designers construct the learning system and its learning mechanisms; part of the task-specific behaviour is then acquired from experience rather than written as rules.

Technique: Deciding between the two routes

Ask three questions.

1. *Can a competent person write down the complete procedure?* If yes (tax, sorting, physics simulation), program it.
2. *Can we recognise correct behaviour on examples, and do we have (or can we collect) many examples?* If yes and question 1 was no (digits, speech, fraud), learn it.
3. *What is the cost of a mistake?* Learning gives approximately correct behaviour. When "approximately" is unacceptable (safety interlocks, cryptography), keep the explicit program or wrap the learned component in explicit safeguards.

Most real systems mix both: a learned model produces a prediction and an explicit program decides what to do with it under fixed business rules.

Gotcha

- "Machine learning replaces programming" is false. Learning replaces the *task-specific rules*; the learner itself, the data pipeline and the decision logic are still programmed.
- "Expert knowledge is obsolete" is also false. It decides what to measure, sanity-checks the model, and covers cases with no data.

Summary: Two routes

- Route 1: humans learn, then write rules. Right when the procedure is known and exactness matters.
- Route 2: give the machine experience and a capacity to learn. Right when we can recognise correct behaviour but cannot describe the procedure.

- Expert systems hit the knowledge acquisition bottleneck; digit and object recognition show that ability does not imply a describable procedure.

1.3 What Counts as Experience?

What are we trying to get?

Before choosing any algorithm we must know *what information the learner will receive*. Different forms of experience carry different information, and that difference decides which kind of learning is even possible.

Three forms of experience

Experience can reach a machine in three broad forms.

Examples	**Observations**	**Interactions**
input + desired output	observed data only	actions + consequences

- *Examples* pair an input with a desired output: an image with the digit it shows, a house description with its sale price. Examples give *direct guidance* about what the output should be.
- *Observations* are data without a supplied answer: images, text, audio, sensor logs. They contain *regularities* that can be exploited without anyone labelling them.
- *Interactions* arise when an agent acts in an environment and observes what follows. The learner partly *shapes its own future experience*, because its actions change what happens next.

Definition 1.3.1 (The three learning settings)**.** The three forms of experience lead to three broad learning settings.

- *Supervised learning*: experience consists of examples with desired outputs.
- *Unsupervised learning*: the learner searches for useful structure in observations without desired outputs. *Self-supervised learning* also learns from observations, but constructs a learning signal from the observations themselves (Section 1.5).
- *Reinforcement learning*: experience comes through interaction; consequences and rewards provide information about the quality of behaviour.

These are broad settings, not mutually exclusive kinds of system. A single system can use more than one.

Example 1.3.2 (Same hospital, four projects)**.** A hospital holds a large collection of chest X-rays; for some patients a diagnosis was recorded.

1. *Predict whether a new X-ray shows pneumonia.* Experience: images paired with diagnoses. Feedback: whether the prediction matches the recorded diagnosis. **Supervised.**
2. *Organise thousands of unlabelled X-rays into groups with similar visual*

characteristics. Experience: images only. Feedback: none from outside; the learner judges structure by internal criteria such as within-group similarity. **Unsupervised.**

3. *Remove part of each X-ray and predict the missing region.* Experience: images only, but each image supplies its own target (the hidden region). Feedback: how well the reconstruction matches the hidden pixels. **Self-supervised.**

4. *Teach a robotic device to position an imaging sensor, rewarding good positioning and penalising unsafe movements.* Experience: the device's own actions and their consequences. Feedback: delayed and evaluative (a reward), not a correct answer. **Reinforcement.**

The paradigms differ not by the data type (all four use X-rays) but by *what information about "doing well" reaches the learner*: a correct answer per example, nothing external, an answer manufactured from the data itself, or a reward earned through action.

Example 1.3.3 (Same records, three questions)**.** A university has ten years of student records: previous grades, attendance, course choices, and eventual outcome.

- *Will each student graduate?* Input: the record at entry; output: graduated or not. Supervised, with a categorical output: *classification.*
- *What will the final graduation grade be?* Same input; output: a number. Supervised: *regression.*
- *Which students have similar academic trajectories?* Input: the records; no desired output at all. Unsupervised.

The same collection of observations supports several learning problems because the *problem* is defined by the question asked, not by the data alone.

Gotcha

The word "unsupervised" does not mean "no objective." Grouping X-rays still needs a criterion for what a good grouping is; the difference is that no human supplies the desired answer for each example.

Summary: What counts as experience

- Examples (input + output), observations (data only), interactions (actions + consequences).
- They lead to supervised, unsupervised or self-supervised, and reinforcement learning.
- The paradigm is decided by what feedback about "doing well" reaches the learner, not by the data type.

1.4 Supervised Learning: Learning from Examples

What are we trying to get?

Supervised learning is the setting this course develops mathematically in Chapter 2. Here we want a clear picture of its ingredients: what an example is, where the desired outputs come from, how the machine uses them, and what kinds of outputs it may be asked to produce.

Definition 1.4.1 (Supervised example, dataset)**.** A *supervised example* is a pair (x, y) of an *input* x and a *desired output* y. A *training dataset* is a finite collection of such pairs, $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$. The desired output is called the *target*, the *label*, or the *ground truth*, depending on the application. The learner's goal is to acquire a model that produces useful outputs for *new* inputs.

Where do labels come from?

"Ground truth" is a strong phrase for something that was measured with error, judged by a person, or observed only later.

The desired output must itself come from somewhere, and this has consequences for what can be learned.

- *Measurement.* Temperature, electricity consumption, the recorded sale price of a house. Measurements carry instrument error and recording mistakes.
- *Annotation.* Identifying an object in an image, interpreting a medical scan. Experts disagree, and labels depend on the conventions chosen when the task was defined (is a "dog" label correct for a wolf-dog hybrid?).
- *Delayed outcome.* Whether a customer repays a loan or a machine fails within a month is only known after time has passed. Such labels are available only for old inputs, and the world may have changed since.

The quality of the supervision limits what can be learned. A model trained on labels that are wrong 10% of the time cannot be expected to agree with the (noisy) labels more than about 90% of the time, and its errors will partly mirror the annotators' errors.

Example 1.4.2 (Phishing detection: designing the experience)**.** Asked to build a phishing-website detector, the first job is to decide what experience to collect, before any algorithm is chosen.

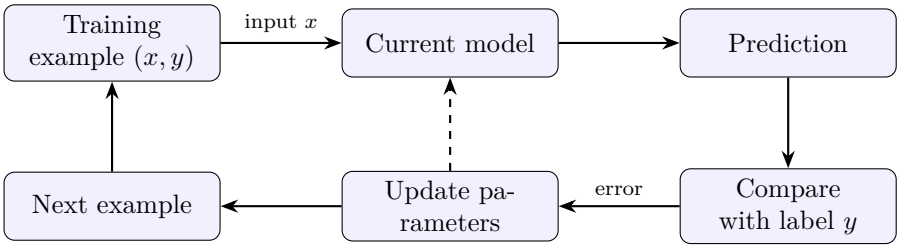
- *Where do examples come from?* Reported phishing pages (from security feeds and user reports) and legitimate pages (from popular sites and from the organisation's own traffic).
- *How varied should they be?* Many languages, many impersonated brands, pages built with different tools, both crude and sophisticated attacks; otherwise the model learns "phishing = badly spelled English page imitating one bank."
- *How are correct outcomes established?* A page is phishing if a security analyst confirmed it, or if the impersonated organisation confirmed it. Both are imperfect; some legitimate pages are mislabelled by over-eager reporters.
- *How fast does the world change?* Phishing strategies change monthly. Labels from three years ago describe attacks nobody uses any more, and

today's attacks are absent from the data.

A very large dataset can still give a poor system if the experience is badly chosen: a million pages all from one feed, one language and one year teaches a narrow, outdated notion of phishing. Volume does not substitute for coverage.

How does the machine learn from examples?

During training, the model receives an input and produces a prediction. The prediction is compared with the desired output, and the comparison produces feedback that the learning algorithm uses to change the model. The process continues across the available training experience.



The computational process through which experience changes the model is called *training*. Different learning methods perform the changes differently; Chapter 2 turns this informal cycle into a mathematical problem.

Regression and classification

Definition 1.4.3 (Regression, classification)**.** A supervised problem is a *regression* problem when the desired output is numerical (a real number or a vector of real numbers), and a *classification* problem when the desired output is a category from a finite set of classes. The type of problem is decided by the *output*, not by the input.

Example 1.4.4 (Ten tasks, sorted by their output)**.**

Task	Input	Output	Type
House price	description	price in MAD	regression
Digit recognition	image	one of $\{0, \dots, 9\}$	classification
Spam filter	email	spam / legitimate	classification
Tomorrow's rainfall	weather data	millimetres	regression
Will it rain?	weather data	yes / no	classification
Tumour size	medical image	millimetres	regression
Tumour type	medical image	benign / malignant	classification
Battery lifetime	usage logs	hours	regression
Species	photograph	one of 10 species	classification
Energy demand at 18:00	calendar, weather	megawatts	regression

The same medical image appears twice: predicting its size is regression, predicting its category is classification. The same weather data appears twice as well. The *desired output* decides.

Some supervised problems have richer outputs. *Object detection* returns several objects with their locations (a list of boxes with class labels); *semantic segmentation* assigns a class to every pixel; *machine translation* outputs a sentence.

These are still supervised (examples come with desired outputs), but the output is structured rather than a single number or class.

Prediction, decision, action

Definition 1.4.5 (Prediction, decision, action)**.** A *prediction* expresses what the model says about an unknown output. A *decision* uses the available information together with a criterion to select a response. An *action* is what is carried out when the decision calls for something to be done.

Example 1.4.6 (A screening system)**.** A model predicts that a patient has a 12% probability of a disease (prediction). A protocol says: recommend further testing when the probability exceeds 10% (decision). The test is scheduled and performed (action). Change the protocol threshold to 20% and the same prediction leads to a different decision. In simple applications prediction and decision coincide; they separate as soon as different errors have different consequences, a theme developed precisely in Section 2.2.

Gotcha

- A label is not truth. Ask how it was produced before trusting an accuracy figure.
- "Regression" does not mean "linear regression" and "classification" does not mean "two classes." The words describe output types.
- A model that predicts probabilities has not yet made a decision. Do not confuse "the model says 0.6" with "the model says yes."

Summary: Supervised learning

- Examples are (input, desired output) pairs; labels come from measurement, annotation or delayed outcomes, each imperfect.
- Training cycle: predict, compare with the label, update, repeat.
- Regression: numerical output. Classification: categorical output. The output decides the type.
- Prediction, decision and action are distinct steps; they separate when errors have unequal costs.

1.5 Representations: What Is Presented to the Learner?

What are we trying to get?

A learner never sees a house or a photograph; it sees numbers. We want to understand what those numbers should be, why the choice matters enormously, and how modern systems learn the numbers themselves, sometimes without any labels at all.

Feature engineering

The form of experience says what information exists*; the representation says what information the model* gets.

For some problems, useful input features can be chosen with ordinary knowledge of the application. A house-price model might receive floor area, location, age, and numbers of bedrooms and bathrooms. Human designers decide these are worth presenting; the model discovers how they relate to price. Some supplied features may turn out to have little influence.

Perceptual problems make the choice much harder. An image contains millions of pixel values; an audio recording is a long waveform; useful information in text depends on relationships among many words. For many years, successful systems relied on representations designed by specialists: edges, corners, textures and shapes for vision; spectral features such as mel-frequency cepstral coefficients for speech. This *feature engineering* let domain knowledge determine what information was presented, and the model learned how to use it. The choice of representation can decide whether learning succeeds at all.

Example 1.5.1 (Two representations of the same email)**.** A spam filter could receive (a) the raw sequence of characters, or (b) a vector of counts: how many times each of 10,000 common words appears, whether the sender is in the address book, the number of links, the number of exclamation marks. Representation (b) makes "many links, no known sender, the word *prize*" directly visible as a few coordinates; representation (a) hides it behind thousands of characters whose positions vary from message to message. With a few thousand examples, (b) is learnable and (a) is hopeless. With billions of examples and a model able to build its own representation, (a) becomes viable and can surpass (b), because the designer of (b) forgot features that matter.

Representation learning

Feature engineering raises a further possibility: instead of asking humans to decide all useful features in advance, can the machine discover a useful representation from experience? This is the aim of *representation learning*. The idea is broader than deep learning and predates its success; representations can be learned by many methods.

Definition 1.5.2 (Representation)**.** A *representation* is a transformation of the original input into a form that makes useful structure easier for a learning model to exploit. The two pipelines are

$$\text{Raw Input} \longrightarrow \text{Human-Designed Features} \longrightarrow \text{Learned Predictor},$$
$$\text{Raw Input} \longrightarrow \text{Learned Representation} \longrightarrow \text{Learned Predictor}.$$

Deep learning suits the second pipeline because several layers can construct successive representations, with higher-level features (object parts, shapes) built from lower-level ones (edges, local patterns). The second pipeline does not remove humans from the design: they still choose the architecture, the learning method, the training experience and the objective. What changes is that more of the representation is acquired from data.

Learning representations from observations: self-supervision

Representation learning does not require a label for every observation. In *self-supervised learning* a learning problem is constructed from the observations them-

selves: part of an observation is used to predict another part.

Example 1.5.3 (Three self-supervised tasks)**.**

- *Text.* Hide a word in "The train to _____ leaves at nine" and ask the model to fill it in. The sentence supplies both the context and the answer.
- *Images.* Mask a square region of a photograph and ask the model to reconstruct it from the rest. Success requires knowing what objects and textures look like.
- *Audio.* Remove half a second of speech and predict it from the surrounding sound.

Nobody labelled anything; the data are their own supervision. The task gives the model a reason to discover regularities, and the representation it builds can later be adapted, with a small labelled dataset, to classification, detection or other tasks. Learning from unlabelled observations and transferring the result predates the current generation of large models and remains one of the main uses of representation learning.

From self-supervision to generation

Self-supervised learning can also produce a model whose predictive ability is useful *directly*. Every piece of text already tells us which token followed the previous ones, so a long document provides many training examples for free. An *autoregressive language model* learns to predict the next token from the tokens that came before. Once it can predict, it can *generate*: predict a next token, append it, predict again.

Example 1.5.4 (A next-token model you can compute by hand)**.** Take the tiny corpus

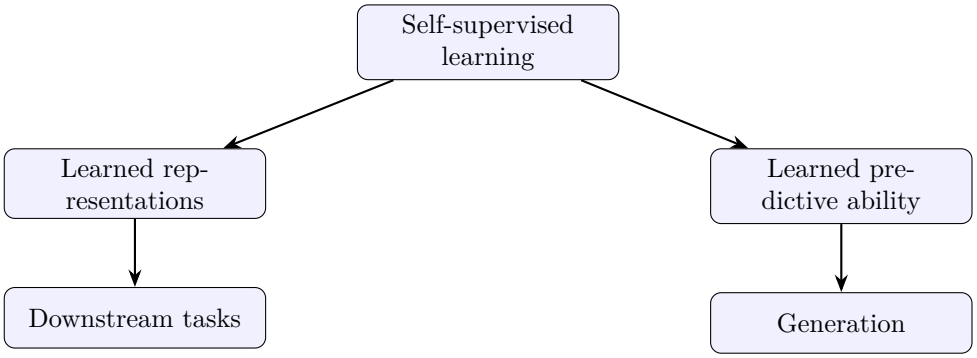
```
the cat sat on the mat .  the cat ate the rat .
```

The simplest next-token model looks only at the previous word and estimates probabilities by counting. After `the`, the next word is `cat` twice, `mat` once, `rat` once:

$$\mathbb{P}(\texttt{cat} \mid \texttt{the}) = \tfrac{2}{4}, \qquad \mathbb{P}(\texttt{mat} \mid \texttt{the}) = \tfrac{1}{4}, \qquad \mathbb{P}(\texttt{rat} \mid \texttt{the}) = \tfrac{1}{4}.$$

After `cat`: `sat` and `ate`, each with probability $\frac{1}{2}$. After `sat`: `on` with probability 1. After `on`: `the`. After `mat` or `rat`: the full stop. Nobody wrote a label: the corpus itself says what followed what.

Generation. Start from `the`. Sample the next word from $(\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, say `cat`; then from $(\frac{1}{2}, \frac{1}{2})$, say `ate`; after `ate` the only continuation is `the`; and so on. One possible output is `the cat ate the mat .`, a sentence that never appears in the corpus. The model has recombined regularities it found. Large language models do the same thing with a context of thousands of tokens and a model with billions of parameters instead of a count table, but the principle, self-supervised next-token prediction followed by repeated sampling, is exactly this.



The two branches are two *uses* of what was learned, not two kinds of model. A generative language model also learns internal representations, and a representation learned by self-supervision can become part of a generative system.

Summary: Representations

- The representation decides which information the model actually receives; a good one can make an impossible problem easy.
- Feature engineering: humans design the features. Representation learning: the machine learns them.
- Self-supervision manufactures targets from the data (fill in the missing word, pixel, sound). Next-token prediction is self-supervised, and repeated sampling from a next-token model is generation.

1.6 Four Ingredients of a Learning System

What are we trying to get?

We want a checklist that applies to *any* learning system, from a count table to a language model, so that we can describe a new application without naming an algorithm and can see what each design choice is responsible for.

Definition 1.6.1 (The four ingredients)**.** Almost any learning system can be examined through four questions, corresponding to four ingredients.

Experience	What information is available?
Model	In what form can acquired knowledge be represented?
Learning algorithm	How does experience change the model?
Evaluation criterion	How is success judged?

This framework begins to answer the question of Section 4.1: what gives a machine the capacity to learn? The model determines what can be represented; the learning algorithm provides the mechanism through which experience changes it; the evaluation criterion indicates which changes are useful; the experience determines what evidence exists in the first place. Representation learning adds one possibility: the way the input is internally represented can itself be shaped by experience.

Example 1.6.2 (Four systems, four ingredients)**.**

	Experience	Model	Learning algorithm	Evaluation
Bigram text model (Ex. 1.5.4)	a corpus	a table of next-word probabilities	count and normalise	probability assigned to held-out text
Digit classifier	labelled images	a function from pixels to 10 probabilities	adjust parameters to reduce error on examples	accuracy on new images
Fraud detector	past transactions with confirmed outcomes	a function from transaction features to fraud probability	same, but with rare positives weighted up	cost of missed fraud plus cost of blocked customers
Warehouse robot	its own actions in simulation	a policy from sensor readings to motor commands	reinforcement from rewards	task completion and safety in the real warehouse

Each row can be discussed, criticised and improved without naming a single algorithm. That is the point of the checklist.

Example 1.6.3 (Designing an application from scratch)**.** *Task: predict, from a farm's sensor data, whether an irrigation pump will fail in the next week.*

1. *Experience.* Two years of hourly readings (current draw, vibration, temperature, flow) from 300 pumps, with the maintenance log recording each failure and its date. Each week of readings becomes an input; the label is whether a failure occurred in the following week.
2. *Model output.* A probability of failure within seven days, so that the maintenance team can rank pumps.
3. *Information for improvement.* Disagreement between predicted probability and what actually happened, aggregated over many pump-weeks. Failures are rare, so the learner must be told that missing one is much worse than a false alarm.
4. *Judging success.* On pumps and months not used during development: how many failures were flagged in advance, and how many unnecessary inspections were triggered, converted into money.

Ingredient by ingredient: (1) is experience, (2) is the model (its output form), (3) is the learning algorithm's signal, (4) is the evaluation criterion.

Technique: Writing the four ingredients for a new problem

- Start with the *evaluation criterion*: what would make the client say "it works"? Everything else must serve it.
- Then *experience*: what is logged already, what can be labelled, what changes over time.
- Then the *model output*: a number, a class, a probability, a ranking, a

sentence?

- Only then the *learning algorithm*. It is the ingredient with the most textbooks and the least influence on whether the project makes sense.

Summary: Four ingredients

- Experience, model, learning algorithm, evaluation criterion.
- The capacity to learn lives in the last three; the evidence lives in the first.
- Describe any application through the four questions before choosing methods.

1.7 The Challenge of Generalization

What are we trying to get?

Experience is always finite, and the situations a deployed system meets are new. We want a precise name for "being useful on new situations," a clear reason why performance on the training examples does not settle it, and a way to test for it.

Finite experience, infinite situations

Doing well on what you have seen is not the goal. It is a side effect, and a dangerous one to optimise for.

A digit recogniser cannot see every handwritten digit that will ever be produced; a medical system cannot observe every future patient; a robot cannot encounter every situation before deployment. A useful learner must acquire something from finite experience that remains useful when a new input appears.

Definition 1.7.1 (Generalization)**.** *Generalization* is the ability of a learned system to perform well in new situations that were not encountered during learning. The central challenge of machine learning is

$$\text{Finite Experience} \xrightarrow{\text{Learning}} \text{Ability Useful in New Situations.}$$

The following small proposition is the mathematical root of the difficulty. It says that the training data never pin down the rule.

Proposition 1.7.2 (Finite data are consistent with infinitely many rules)**.** *Let $\mathcal{X}$ be an infinite input space, let $\mathcal{Y}$ contain at least two elements, and let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ be any finite dataset. Then there are infinitely many functions $f : \mathcal{X} \to \mathcal{Y}$ that agree with the data, i.e. $f(x^{(i)}) = y^{(i)}$ for all i, and any two of them may disagree on every input not in the data.*

Proof. Let $S = \{x^{(1)}, \dots, x^{(N)}\}$, a finite set, so $\mathcal{X} \setminus S$ is infinite. Fix two distinct outputs $y_0 \neq y_1$ in $\mathcal{Y}$. For each subset $T \subseteq \mathcal{X} \setminus S$ define $f_T(x) = y^{(i)}$ if $x = x^{(i)} \in S$, $f_T(x) = y_1$ if $x \in T$, and $f_T(x) = y_0$ otherwise. Every f_T agrees with the data. Distinct subsets $T \neq T'$ give distinct functions (they differ on any $x \in T \triangle T'$), and an infinite set has infinitely many subsets. Taking $T = \mathcal{X} \setminus S$ and $T' = \emptyset$ gives two data-consistent functions that disagree on every input outside the data. □

Key insight

Off the training inputs, the data say nothing. Whatever a learned system does there comes from its *model* and its *learning algorithm*, that is, from assumptions built in by the designer. Generalization is therefore never a property of the data alone; it is a property of the match between the assumptions and the world. Chapter 2 introduces the formal objects (hypothesis spaces, risks) needed to study this match.

Example 1.7.3 (The perfect memoriser)**.** The function "look up x in the training set and return its label; otherwise return class 0" has zero error on the training examples. It has learned nothing. A learner evaluated only on its training data cannot distinguish this lookup table from a genuinely useful model. This is why performance on previously encountered examples gives only part of the evidence about what has been learned.

Shortcuts: right answers for the wrong reasons

Example 1.7.4 (Wolves and snow)**.** A system learns to distinguish photographs of wolves from dogs and performs extremely well on the developers' examples. After deployment its performance drops. Investigation shows that most wolf photographs used in learning contained snow, whereas most dog photographs did not.

1. *Initial success.* "White background ⇒ wolf" is a rule consistent with almost all of the training examples (Proposition 1.7.2 guarantees such alternative rules exist). It is far simpler to detect than wolf anatomy, so the learner adopted it.
2. *Failure on new photographs.* Deployed users photograph dogs in snow and wolves in summer. The shortcut now gives systematically wrong answers, because the correlation it exploited was a property of the collection, not of wolves.
3. *Diagnosis.* Two test photographs: a husky standing in snow, and a wolf on green grass. A system that learned about wolves classifies them correctly; a system that learned about snow gets both wrong.

Nothing here assumes anything about the model's internals. The reasoning uses only the definition of generalization: the system performed well on the situations it had seen and poorly on new ones, and the explanation is a regularity that held in the experience but not in the world.

Example 1.7.5 (Quantity versus diversity)**.** A company must choose one image collection for recognising ten bird species: Dataset A, one million photographs from one camera in one park; or Dataset B, fifty thousand photographs by many people in many countries, seasons, weathers, backgrounds and angles. For a system to be used by the public, B is the better choice: A's million images share lighting, background, altitude and season, so many of the regularities a learner will find in A (this park's foliage, this camera's colour balance) will not hold elsewhere, exactly as in Example 1.7.4. B covers more of the situations the deployed system will meet. A would be preferable if the system is to be deployed *in that park with that camera*, or if the species

are so similar that the sheer number of examples is needed to tell them apart and the deployment conditions match. The lesson: the amount of experience and its *representativeness* of future situations are different quantities, and the second is the one generalization depends on.

Example 1.7.6 (The robot that learned in the wrong world)**.** A warehouse robot trained entirely in a simulator with perfectly flat floors, identical box friction and constant lighting performs well in simulation and poorly in the real warehouse. If only the simulator can be changed, vary the things that vary in reality: floor slope and bumps, box weights and surface friction, lighting, sensor noise, worker behaviour, wheel slippage. A robot exposed to a range of conditions must acquire behaviour that works across them, rather than behaviour tuned to one idealised world. Making the simulator *visually* more realistic would not by itself help: if the floor is still perfectly flat, the robot still learns that wheels never slip. What matters is covering the variations that affect the decisions, not photorealism.

Example 1.7.7 (A claim worth questioning)**.** An advertisement says: "Our AI has learned every possible situation from millions of examples." The claim confuses situations that were *observed* with situations that *may occur*. Millions of examples are a finite sample; the situations a deployed system will face are not a fixed list and include ones nobody has recorded, and the world changes after the data were collected. What a system can honestly claim is to have learned regularities from the examples it saw that *are expected* to carry over to similar new situations. How similar, and with what guarantee, is exactly the study of generalization, and no amount of data makes the question disappear.

Technique: Testing for generalization without opening the model

- *Hold out.* Evaluate on examples the learner never saw. (Chapter 2 makes this precise with validation and test sets.)
- *Break the shortcuts.* Construct test inputs where suspected spurious cues are decoupled from the label (dog in snow, wolf on grass).
- *Shift the conditions.* Evaluate on data from a different camera, hospital, country or year than the training data. A large drop reveals reliance on collection-specific regularities.
- *Ask what the data cannot know.* List the situations absent from the experience; the model's behaviour there is an assumption, not a learned fact.

Gotcha

- High accuracy on the training examples is compatible with having learned nothing (the memoriser) or the wrong thing (the snow detector).
- More data helps only if it covers more of the situations that will occur. A billion photographs of one park are one situation.

Summary: Generalization

- Generalization: performing well in situations not encountered during learning.
- Finite data are consistent with infinitely many rules (Proposition 1.7.2); what a learner does off the data comes from its built-in assumptions.
- Shortcuts (snow for wolves) give excellent training performance and fail on new data. Test on held-out, shifted and shortcut-breaking inputs.

1.8 A Connection to Information Theory

What are we trying to get?

We want a numerical way to say how good a set of predicted probabilities is, and a way to talk about how much information a representation keeps. Both come from information theory, and the single inequality proved here is the reason cross-entropy is the workhorse loss of classification in Chapter 2.

Entropy, cross-entropy, divergence

Entropy: how uncertain the truth is. Cross-entropy: how surprised you are, on average, when you believed q and the truth was drawn from p.

Recall that *entropy* describes uncertainty, and *information* tells us how much uncertainty is removed by observing something. From a learning perspective, an input is useful for prediction when it contains information about the output.

Definition 1.8.1 (Entropy, cross-entropy, Kullback–Leibler divergence)**.** Let $p = (p_1, \dots, p_C)$ and $q = (q_1, \dots, q_C)$ be probability distributions on the same finite set of C outcomes, with $q_c > 0$ whenever $p_c > 0$. Define

$$H(p) = -\sum_c p_c \log p_c, \qquad H(p, q) = -\sum_c p_c \log q_c,$$

$$\mathrm{D}_{\mathrm{KL}}(p \,\|\, q) = \sum_c p_c \log \frac{p_c}{q_c},$$

with the convention $0 \log 0 = 0$. Base-2 logarithms measure in bits; natural logarithms, customary in machine learning, measure in nats.

Expanding the divergence, $\mathrm{D}_{\mathrm{KL}}(p\|q) = \sum_c p_c \log p_c - \sum_c p_c \log q_c = -H(p) + H(p, q)$, so

$$H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p \,\|\, q). \tag{1.1}$$

Theorem 1.8.2 (Gibbs' inequality)**.** $\mathrm{D}_{\mathrm{KL}}(p \,\|\, q) \geq 0$*, with equality if and only if* $p = q$*. Equivalently,* $H(p, q) \geq H(p)$ *with equality iff* $q = p$*: among all distributions* q*, the expected surprise* $H(p, q)$ *under truth* p *is smallest when* q *is the truth.*

Proof. **Step 1 (a one-line inequality about logarithms).** For every $t > 0$, $\ln t \leq t - 1$, with equality iff $t = 1$. Indeed $g(t) = t - 1 - \ln t$ has $g'(t) = 1 - 1/t$, negative on $(0, 1)$ and positive on $(1, \infty)$, so g has its unique minimum at $t = 1$ where $g(1) = 0$.

Step 2 (apply it term by term). Let $S = \{c : p_c > 0\}$. Using natural

logarithms,

$$-\,\mathrm{D_{KL}}(p\|q) = \sum_{c\in S} p_c \ln \frac{q_c}{p_c} \;\le\; \sum_{c\in S} p_c\Big(\frac{q_c}{p_c} - 1\Big) = \sum_{c\in S} q_c - \sum_{c\in S} p_c = \sum_{c\in S} q_c - 1 \le 0,$$

since $\sum_{c\in S} q_c \le 1$. Hence $\mathrm{D_{KL}}(p\|q) \ge 0$. Changing the base of the logarithm multiplies everything by a positive constant, so the sign is unaffected.

Step 3 (equality). Equality requires equality in Step 1 for every $c \in S$, i.e. $q_c = p_c$ for all c with $p_c > 0$, and also $\sum_{c\in S} q_c = 1$, which forces $q_c = 0$ whenever $p_c = 0$. So $q = p$. Conversely $p = q$ gives every term $\log 1 = 0$. □

Key insight

The whole theorem is the tangent-line inequality $\ln t \le t - 1$, applied to the ratios q_c/p_c and averaged under p. The averaging turns a family of pointwise inequalities into one statement about the whole distribution. This trick, "bound the log by its tangent, then average," recurs throughout machine learning.

Example 1.8.3 (Numbers)**.** Let the truth be $p = (0.7, 0.3)$ over two classes. In bits, $H(p) = -0.7\log_2 0.7 - 0.3\log_2 0.3 \approx 0.881$.

Prediction q	$H(p,q)$	$\mathrm{D_{KL}}(p\|q)$	Comment
$(0.7, 0.3)$	0.881	0	the truth: minimum possible
$(0.5, 0.5)$	1.000	0.119	hedging costs 0.119 bits
$(0.9, 0.1)$	1.103	0.222	overconfident in the right direction: worse than hedging
$(0.3, 0.7)$	1.370	0.489	wrong direction
$(0.99, 0.01)$	2.003	1.122	very overconfident: the 0.3 mass pays $\log_2 100$ each

Two lessons. The floor 0.881 is a property of the *problem*: no prediction beats it, because the outcome is genuinely uncertain. And overconfidence is punished sharply: $(0.9, 0.1)$ is further from the truth in cross-entropy than the uninformative $(0.5, 0.5)$, even though it favours the right class.

Why this matters for learning

Two connections deserve to be kept in mind for the rest of the course.

- *Cross-entropy as a criterion.* A classifier may assign probabilities to several classes. Cross-entropy compares those probabilities with what actually occurs: giving high probability to the correct class incurs a small penalty, giving it little probability a large one. By Theorem 1.8.2, the expected penalty is minimised exactly when the predicted probabilities equal the true conditional probabilities. A familiar information-theoretic quantity thus becomes one of the most widely used learning criteria (Section 2.5).

- *Representations and information.* Raw observations contain many kinds of information, only some of it useful for a given purpose. Representation learning asks how data can be transformed so that useful structure becomes easier to exploit. Information theory supplies the vocabulary: an informative representation retains what is relevant about the output and discards the rest.

Technique: Computing with entropies

- $H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p\|q)$: compute the entropy once, then only the divergence changes with the prediction.
- Cross-entropy of a *one-hot* truth ($p_y = 1$) is simply $-\log q_y$: the penalty for the probability given to the observed class. This is the form used for training in Chapter 2.
- D_{KL} is not symmetric: $\mathrm{D}_{\mathrm{KL}}(p\|q) \neq \mathrm{D}_{\mathrm{KL}}(q\|p)$ in general. Check the order before quoting it.
- $q_c = 0$ for an outcome with $p_c > 0$ gives $H(p, q) = +\infty$: never predict probability exactly zero for something that can happen.

Classic problems

- *Which q minimises $H(p, q)$?* $q = p$, by Gibbs' inequality; the minimum value is $H(p)$.
- *p uniform on C outcomes: $H(p)$?* $\log C$. It is the maximum entropy on C outcomes (apply Gibbs with q uniform: $\log C - H(p) = \mathrm{D}_{\mathrm{KL}}(p\|\mathrm{unif}) \geq 0$).
- *Truth $(0.5, 0.5)$, prediction $(0.99, 0.01)$: cross-entropy in bits?* $-0.5 \log_2 0.99 - 0.5 \log_2 0.01 \approx 3.33$, versus 1 bit for the honest prediction.

Summary: Information theory

- $H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p\|q)$ and $\mathrm{D}_{\mathrm{KL}} \geq 0$ with equality iff $q = p$ (Gibbs, Theorem 1.8.2).
- Cross-entropy therefore rewards *calibrated* probabilities, punishes overconfidence, and has an irreducible floor $H(p)$ set by the problem.
- Representation learning can be phrased as keeping the information about the output and discarding the rest.

Notes on the original lecture notes

The Lecture 1 notes are conceptual, and every claim in them was checked against the standard history and vocabulary of the field. The following points are worth a reader's attention.

- **A stray citation marker (Section 14 of the notes).** The sentence ending "remains an important use of representation learning" is followed by the text `:contentReference[oaicite:4]index=4`. This is an artefact of the AI-assisted editing acknowledged at the end of the notes, not part of the content; ignore it.
- **"Ground truth" (Section 8).** The notes rightly warn that the term should be used with care. This chapter goes one step further and treats label quality as an explicit limit on achievable accuracy.
- **Four abilities (Section 2).** The notes state that recognise, predict, reason, act are "not an exhaustive classification." Do not use the list as a taxonomy

in an exam answer; use it as vocabulary.

- **Generalization (Section 17).** The notes define generalization and defer the reasons "why several relationships can explain the same finite observations" to Lecture 3. Proposition 1.7.2 supplies the elementary version of that fact now, since it is what makes the wolves-and-snow story possible.
- **Information theory appendix.** The notes recall entropy and cross-entropy from the information theory course without proof. Theorem 1.8.2 proves the one fact everything rests on.

Chapter takeaways

Summary: What it means to learn

- **Learning** turns experience into ability. Machines can be given abilities by writing rules or by giving them experience plus a capacity to learn; the second route exists because having an ability does not imply having a describable procedure.
- **Experience** comes as examples, observations or interactions, leading to supervised, unsupervised/self-supervised and reinforcement learning. The paradigm is decided by the feedback the learner receives.
- **Supervised learning** pairs inputs with desired outputs, whose quality limits what can be learned; regression and classification are distinguished by the output; prediction, decision and action are distinct.
- **Representations** decide what the model sees. They can be designed or learned, and self-supervision lets them be learned from unlabelled data; next-token prediction turns this into generation.
- **Four ingredients** describe any learning system: experience, model, learning algorithm, evaluation criterion.
- **Generalization** is the real goal. Finite data never determine the rule; training performance is not evidence of it; shortcuts are the typical failure.
- **Cross-entropy** is minimised by the true probabilities (Gibbs' inequality), which is why it will be our training criterion for classification.

Chapter 2

From Representations to Decisions

How to read the formulas in this chapter

This chapter is where the mathematics starts, and the slides can look intimidating. Two things make it manageable. First, there are only a handful of symbols, listed below; every formula in the chapter is built from them. Second, every formula is followed by a *Reading the formula* box that says the same thing in words and then plugs in small numbers. If a formula looks frightening, skip to its box, then come back.

Symbol	Meaning in words
$x \in \mathcal{X} \subseteq \mathbb{R}^d$	one input, written as a list of d numbers (the *features*)
$y \in \mathcal{Y}$	the output we want to predict (a number, or a class label)
X, Y	the input and output regarded as *random*: "a house drawn at random and its price"
$\mathbb{P}(X, Y)$	the joint distribution: how likely each (input, output) pair is in the world
$\mathbb{P}(Y \mid X = x)$	the distribution of the output *once we know* the input equals x
$\mathbb{E}[\cdot \mid X = x]$	an average over that conditional distribution
$a \in \mathcal{A}$	an *action* we might take (a predicted number, a chosen class, "raise the alarm")
$\ell(y, a)$	a *loss*: how much it costs to take action a when the truth is y
$f^*(x)$	the best possible decision rule when the distribution is known (the *Bayes rule*)
$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$	the training data: N examples; superscript (i) = example number
$x_j^{(i)}$	feature number j of example number i
f_θ, $\theta \in \Theta$	a model with adjustable numbers θ (the *parameters*)
$\mathcal{H} = \{f_\theta : \theta \in \Theta\}$	the *hypothesis space*: all the functions the model can represent
$R(\theta)$	the *population risk*: average loss over the whole world (unknowable)
$\widehat{R}(\theta)$	the *empirical risk*: average loss over the examples we have
λ	a *hyperparameter*: a setting chosen by us, not learned (e.g. k in kNN)
$\mathbf{1}$(condition)	the *indicator*: equals 1 if the condition holds, 0 otherwise
$\arg\min_a g(a)$	the value of a that makes $g(a)$ smallest ("the best a")
Δ^{C-1}	the set of probability vectors over C classes (nonnegative, sum to 1)

2.1 From Objects to Numerical Representations

What are we trying to get?

A learning algorithm cannot receive a house, only numbers. We want to know how an object becomes a vector x, which choices are hidden in that step, and why one careless choice (coding categories as integers) can silently break everything that follows.

Describe, then encode

Two steps, both made by humans: choose what to describe, then choose how to turn the description into numbers.

Chapter 1 described learning as acquiring, through experience, the ability to make appropriate decisions in new situations. A mathematical learning problem begins by specifying the information available for those decisions. Properties of the object are first *selected* to describe it and then *encoded* mathematically:

$$\text{object} \longrightarrow \text{features describing it} \longrightarrow \text{representation } x \in \mathbb{R}^d.$$

Definition 2.1.1 (Input space, feature vector)**.** Once an object is described by d numerical components, we write

$$x = [x_1, x_2, \ldots, x_d]^\top \in \mathcal{X} \subseteq \mathbb{R}^d,$$

where $\mathcal{X}$ is the *input space*. Components may be of different kinds: a nor-

malised measurement in $[0, 1]$, a binary feature in $\{0, 1\}$, or a categorical feature spread over several binary components (*one-hot encoding*: a category among K possibilities becomes a vector of length K with a single 1 in the position of the category).

Reading the formula

$x = [x_1, \ldots, x_d]^\top$ is just "a list of d numbers, written as a column." For a house described by area 145 m^2, 4 rooms, age 12 years and neighbourhood A out of $\{A, B, C\}$:

$$x = [\,145,\ 4,\ 12,\ \underbrace{1,\ 0,\ 0}_{\text{neighbourhood } A}\,]^\top \in \mathbb{R}^6.$$

Here $d = 6$: three numerical features and three binary components for one categorical feature. A house in neighbourhood C would end with $0, 0, 1$.

Example 2.1.2 (Why not code neighbourhoods as $1, 2, 3$?)**.** Suppose neighbourhoods A, B, C are coded as a single number $1, 2, 3$. Then, in any method that measures similarity by distance (Section 2.3), a house in A is at distance 1 from a house in B but at distance 2 from one in C: the coding has invented an ordering and a notion of "B is halfway between A and C" that has no meaning. With one-hot coding the three neighbourhoods are pairwise at the same distance $\sqrt{2}$: $\|(1,0,0) - (0,1,0)\| = \sqrt{1+1}$. The encoding step is not a formality; it decides what "similar" means.

The representation controls the available information

A house described only by its area offers less information for predicting price than one described by area, location, age, condition and number of rooms. Additional components help when they expose information relevant to the decision; irrelevant, redundant or unreliable features increase d without making the representation more informative. Images, audio and text make this visible: pixel values provide numbers, but shape, object identity and meaning are not explicitly encoded in them. Traditional machine learning relied on human-designed features; modern methods can learn representations from data (Section 1.5).

Example 2.1.3 (Three descriptions of the same email)**.** Description 1: the length in characters ($d = 1$). Description 2: counts of 10,000 words ($d = 10{,}000$). Description 3: the same counts plus 500 random numbers ($d = 10{,}500$). Description 2 contains everything needed to detect most spam; description 1 does not; description 3 contains the same information as 2 but forces the learner to discover that 500 coordinates are noise, which costs data. More features are not automatically better.

Gotcha

- Integer codes for categories create fake distances (Example 2.1.2). Use one-hot coding unless the categories are genuinely ordered ("small, medium, large").
- Units matter. A feature in metres and the same feature in millimetres are

the same information but behave very differently in any distance-based method (Section 2.3).

Summary: Numerical representations

- Object → chosen features → vector $x \in \mathbb{R}^d$. Both arrows are human choices.
- Categorical features become one-hot blocks; integer codes invent spurious order.
- The representation bounds what any later stage can learn.

2.2 Decision Making When the Distribution Is Known

What are we trying to get?

Before worrying about data, we ask an easier question: *if we knew everything about the world*, what would the best decision be? The answer, the Bayes decision rule, is the target that every learning method tries to approximate, and it already tells us that "the best prediction" depends on what mistakes cost.

Uncertainty that remains after seeing the input

Even with a perfect description, two houses can sell at different prices. The conditional distribution is the honest way to say "we know a lot but not everything."

Let the random vector X represent the input and the random variable Y the output of interest. Their joint distribution $\mathbb{P}(X, Y)$ describes which pairs may occur and how likely they are. Once the input takes the value x, the conditional distribution $\mathbb{P}(Y \mid X = x)$ summarises the remaining uncertainty about the output. Houses with the same recorded description may still sell at different prices because relevant information is missing, measurements are imperfect, or market conditions vary. The conditional distribution accounts for this instead of forcing one output onto every input.

Reading the formula

$\mathbb{P}(Y \mid X = x)$ reads "the probabilities of the possible outputs, *given that* the input is x." Concretely, for a 145 m^2 house in neighbourhood A, it might say: price 1.0 million with probability 0.5, 1.2 million with probability 0.3, 1.6 million with probability 0.2. All three prices are possible for that same description. Nothing is being predicted yet; this is a description of the world.

Actions and their consequences

Knowing the possible outputs and their probabilities does not by itself say what to do. That also depends on the consequences of each action.

Definition 2.2.1 (Decision loss)**.** Let $\mathcal{A}$ be the *action space*, the set of choices available to the decision maker. If the true output is y and the action $a \in \mathcal{A}$ is chosen, the *decision loss*

$$\ell_{\text{dec}}(y, a)$$

is the cost assigned to that outcome. In regression, $\mathcal{A}$ may be the set of

numerical predictions; in classification, the set of class labels. Keeping a distinct from y is useful because decisions and outcomes need not be the same kind of object.

Reading the formula

$\ell_{\text{dec}}(y, a)$ is a table (or formula) with two inputs: what actually happened (y) and what you did (a). It returns a number: the damage. Examples:

- Squared error, $\ell_{\text{dec}}(y, a) = (y - a)^2$: predicting $a = 200$ when $y = 230$ costs $30^2 = 900$.
- Absolute error, $|y - a|$: the same mistake costs 30.
- Zero–one loss, $\mathbf{1}(y \neq a)$: any wrong class costs 1, a right one costs 0.
- A cost table for a medical decision: rows are the true condition, columns the action, entries the cost. Deciding "discharge" when the truth is "severe" might cost 20; "treat" when "healthy" might cost 3.

The Bayes decision rule

For a given input, different outputs may still occur. We therefore choose the action with the smallest *expected* loss under the conditional distribution.

Definition 2.2.2 (Bayes decision rule)**.** For each input x, the *Bayes decision rule* chooses

$$f^*(x) \in \arg\min_{a \in \mathcal{A}} \; \mathbb{E}\big[\ell_{\text{dec}}(Y, a) \,\big|\, X = x\big].$$

The quantity minimised, $\mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$, is the *conditional risk* of action a at input x. The word "Bayes" refers to optimal decision making under a probability distribution; it does not require the parameters of any model to have prior distributions.

Reading the formula

Read it in three steps, for one fixed x.

1. List the candidate actions a.
2. For each a, compute the *average damage*: go through every possible output y, multiply its loss $\ell_{\text{dec}}(y, a)$ by its probability $\mathbb{P}(Y = y \mid X = x)$, and add up. That is $\mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$.
3. Pick the a with the smallest average damage. That is $\arg\min_a$.

With the house above (1.0, 1.2, 1.6 million with probabilities $0.5, 0.3, 0.2$) and squared loss, the average damage of predicting $a = 1.2$ is $0.5(0.2)^2 + 0.3(0)^2 + 0.2(0.4)^2 = 0.02 + 0 + 0.032 = 0.052$. Try $a = 1.16$: $0.5(0.16)^2 + 0.3(0.04)^2 + 0.2(0.44)^2 = 0.0128 + 0.00048 + 0.03872 = 0.052$; the two are equal to three decimals, and Theorem 2.2.4 will tell us the exact best value is $a = 1.18$.

The rule minimises *pointwise*, one x at a time. The next theorem confirms that doing the best thing at every x is the best thing overall.

Theorem 2.2.3 (Pointwise optimality is global optimality)**.** *Let $f : \mathcal{X} \to \mathcal{A}$ be any decision rule and f^* the Bayes rule of Definition 2.2.2 (assume the minimum exists at every x and all expectations are finite). Then*

$$\mathbb{E}[\ell_{\text{dec}}(Y, f^*(X))] \;\le\; \mathbb{E}[\ell_{\text{dec}}(Y, f(X))].$$

The left side is called the Bayes risk*; no rule has smaller expected loss.*

Proof. By the tower property of conditional expectation, $\mathbb{E}[\ell_{\text{dec}}(Y, f(X))] = \mathbb{E}\big[\,\mathbb{E}[\ell_{\text{dec}}(Y, f(X)) \mid X]\,\big]$. For every value x, $f(x)$ is one particular action, so by the definition of $f^*(x)$ as a minimiser,

$$\begin{aligned}\mathbb{E}[\ell_{\text{dec}}(Y, f(x)) \mid X = x] \;&\ge\; \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x] \\ &= \mathbb{E}[\ell_{\text{dec}}(Y, f^*(x)) \mid X = x].\end{aligned}$$

Averaging this inequality over x (taking $\mathbb{E}$ over X) preserves it, giving the claim. □

Key insight

An expectation over pairs (X, Y) can be computed as "first average over Y for each fixed x, then average over x." Because the inner averages are minimised separately at each x, the outer average is minimised too. This is why we may think about one input at a time.

Regression: which number to report

Theorem 2.2.4 (Squared loss selects the conditional mean)**.** *Let $\ell_{\text{dec}}(y, a) = (y - a)^2$ and suppose $\mathbb{E}[Y^2 \mid X = x] < \infty$. Write $m(x) = \mathbb{E}[Y \mid X = x]$. Then for every action a,*

$$\mathbb{E}\big[(Y - a)^2 \,\big|\, X = x\big] = \operatorname{Var}(Y \mid X = x) + \big(m(x) - a\big)^2,$$

so the conditional risk is minimised uniquely at $a = m(x)$: $f^*(x) = \mathbb{E}[Y \mid X = x]$.

Proof. Write $Y - a = (Y - m(x)) + (m(x) - a)$ and square:

$$(Y - a)^2 = (Y - m(x))^2 + 2\,(Y - m(x))(m(x) - a) + (m(x) - a)^2.$$

Take conditional expectations given $X = x$. The first term gives $\operatorname{Var}(Y \mid X = x)$. In the middle term, $(m(x) - a)$ is a constant (it does not involve Y), and $\mathbb{E}[Y - m(x) \mid X = x] = m(x) - m(x) = 0$, so the middle term vanishes. The last term is constant. Hence the formula. The variance does not depend on a, and $(m(x) - a)^2 \ge 0$ with equality only at $a = m(x)$. □

Reading the formula

The formula says: *average squared error = unavoidable spread + your own offset squared.* The spread $\operatorname{Var}(Y \mid X = x)$ is there whatever you do: it is the variability of prices among houses with this description. The only part you control is how far your number a is from the conditional mean $m(x)$.
House example: outcomes $1.0, 1.2, 1.6$ with probabilities $0.5, 0.3, 0.2$. Condi-

tional mean $m = 0.5+0.36+0.32 = 1.18$. Variance $= 0.5(0.18)^2+0.3(0.02)^2+0.2(0.42)^2 = 0.0162 + 0.00012 + 0.03528 = 0.0516$. Predicting $a = 1.2$: risk $= 0.0516 + (0.02)^2 = 0.0520$, which is what we computed by hand above. Predicting $a = 1.18$: risk $= 0.0516$, the floor.

Key insight

Add and subtract the mean, then square. The cross term dies because the mean is exactly the number that makes $\mathbb{E}[Y - m] = 0$. This "complete the square around the mean" move is the single most reused computation in statistics.

The loss determines *which summary* of the conditional distribution should be reported.

Proposition 2.2.5 (Absolute loss selects a conditional median)**.** *Let $\ell_{\text{dec}}(y, a) = |y - a|$ and suppose $\mathbb{E}[|Y| \mid X = x] < \infty$. Any conditional median m, i.e. any number with $\mathbb{P}(Y \le m \mid X = x) \ge \frac{1}{2}$ and $\mathbb{P}(Y \ge m \mid X = x) \ge \frac{1}{2}$, minimises $\mathbb{E}[|Y - a| \mid X = x]$ over a.*

Proof. All probabilities and expectations below are conditional on $X = x$; we drop the conditioning to lighten notation. Let m be a median and $a > m$. For each possible value y, compare the two losses:

- if $y \le m$: $|y - a| - |y - m| = (a - y) - (m - y) = a - m$;
- if $y \ge a$: $|y - a| - |y - m| = (y - a) - (y - m) = -(a - m)$;
- if $m < y < a$: $|y - a| - |y - m| = (a - y) - (y - m) = a + m - 2y > a + m - 2a = -(a - m)$.

In all cases $|y-a|-|y-m| \ge (a-m)\big[\mathbf{1}(y \le m)-\mathbf{1}(y > m)\big]$. Taking expectations,

$$\begin{aligned}\mathbb{E}|Y - a| - \mathbb{E}|Y - m| \;&\ge\; (a - m)\big[\mathbb{P}(Y \le m) - \mathbb{P}(Y > m)\big] \\ &= (a - m)\big[2\mathbb{P}(Y \le m) - 1\big] \;\ge\; 0,\end{aligned}$$

because $\mathbb{P}(Y \le m) \ge \frac{1}{2}$. The case $a < m$ is symmetric, using $\mathbb{P}(Y \ge m) \ge \frac{1}{2}$. So no a beats m. □

Proposition 2.2.6 (Asymmetric absolute loss selects a conditional quantile)**.** *Fix $\tau \in (0, 1)$ and let $\ell_\tau(y, a) = \tau(y - a)$ if $y \ge a$ and $(1 - \tau)(a - y)$ if $y < a$ (under-prediction costs τ per unit, over-prediction $1 - \tau$ per unit). Any conditional τ-quantile m, i.e. any m with $\mathbb{P}(Y \le m \mid x) \ge \tau$ and $\mathbb{P}(Y \ge m \mid x) \ge 1 - \tau$, minimises $\mathbb{E}[\ell_\tau(Y, a) \mid x]$.*

Proof. Same three cases as in Proposition 2.2.5, for $a > m$: if $y \le m$ the difference $\ell_\tau(y, a) - \ell_\tau(y, m)$ equals $(1-\tau)(a-m)$; if $y \ge a$ it equals $-\tau(a-m)$; if $m < y < a$ it equals $(1-\tau)(a-y)-\tau(y-m) \ge -\tau(y-m) \ge -\tau(a-m)$. Hence the difference is at least $(a - m)\big[(1 - \tau)\mathbf{1}(y \le m) - \tau\mathbf{1}(y > m)\big]$, and taking expectations gives $(a - m)\big[(1 - \tau)\mathbb{P}(Y \le m) - \tau(1 - \mathbb{P}(Y \le m))\big] = (a - m)\big[\mathbb{P}(Y \le m) - \tau\big] \ge 0$. The case $a < m$ is symmetric with $\mathbb{P}(Y \ge m) \ge 1 - \tau$. Taking $\tau = \frac{1}{2}$ recovers Proposition 2.2.5 up to a factor $\frac{1}{2}$. □

Example 2.2.7 (Three losses, three answers, one distribution)**.** A delivery company must promise a delivery time a (minutes) for a route whose actual

time Y is 100 with probability 0.5, 200 with probability 0.3, and 600 with probability 0.2 (occasional road closures).

- *Squared loss.* Report the mean: $0.5(100) + 0.3(200) + 0.2(600) = 230$. Check against $a = 200$: risk at 230 is $0.5(130)^2 + 0.3(30)^2 + 0.2(370)^2 = 8450 + 270 + 27380 = 36100$; risk at 200 is $0.5(100)^2 + 0 + 0.2(400)^2 = 37000$. The mean wins.
- *Absolute loss.* $\mathbb{P}(Y \le 100) = 0.5$ and $\mathbb{P}(Y \ge 100) = 1$; also $\mathbb{P}(Y \le 200) = 0.8$ and $\mathbb{P}(Y \ge 200) = 0.5$. Every $m \in [100, 200]$ is a median, and indeed the expected absolute loss is 130 for every a in that interval ($a = 100$: $0.3(100) + 0.2(500) = 130$; $a = 200$: $0.5(100) + 0.2(400) = 130$), and larger outside (at $a = 230$: $0.5(130) + 0.3(30) + 0.2(370) = 148$). Note that the mean 230 is *not* optimal here: the rare 600 pulls the mean but barely moves the median.
- *Asymmetric loss.* If being late costs 9 times more than being early, take $\tau = 0.9$: we need $\mathbb{P}(Y \le m) \ge 0.9$ and $\mathbb{P}(Y \ge m) \ge 0.1$, which holds exactly for $m = 600$. Promise 600 minutes: the company prefers to disappoint nobody.

"The best prediction" has three different values. Nothing about the world changed; only the consequences did.

Decision loss	Optimal prediction	Typical use
$(y-a)^2$	conditional mean	symmetric errors
$\lvert y-a\rvert$	conditional median	robust point prediction
asymmetric absolute	conditional quantile	unequal under/over cost

Classification: which class to declare

In classification the output belongs to one of C categories, and the conditional distribution gives the probability of each class, $\mathbb{P}(Y = c \mid X = x)$ for $c = 1, \dots, C$. These probabilities describe what can be predicted about the unknown class. A class *decision* requires an additional step, so in classification the distinction between prediction and decision is particularly clear:

$$\underbrace{\{\mathbb{P}(Y = c \mid X = x)\}_{c=1}^{C}}_{\text{predicted class probabilities}} \xrightarrow{\text{decision criterion}} \underbrace{f^*(x)}_{\text{class decision}} .$$

Theorem 2.2.8 (Zero–one loss selects the most probable class)**.** *Let $\ell_{\text{dec}}(y, a) = \mathbf{1}(y \neq a)$ (every mistake costs 1, correct decisions cost 0). Then*

$$f^*(x) = \arg\max_c \mathbb{P}(Y = c \mid X = x),$$

the Bayes classifier*, and its conditional risk is $1 - \max_c \mathbb{P}(Y = c \mid X = x)$.*

Proof. For action a, the conditional risk is $\mathbb{E}[\mathbf{1}(Y \neq a) \mid X = x] = \mathbb{P}(Y \neq a \mid X = x) = 1 - \mathbb{P}(Y = a \mid X = x)$. Minimising $1 - \mathbb{P}(Y = a \mid X = x)$ over a is the same as maximising $\mathbb{P}(Y = a \mid X = x)$. □

Reading the formula

$\mathbb{E}[\mathbf{1}(Y \neq a) \mid X = x]$ is "the probability of being wrong if I say a." With class probabilities $(0.5, 0.3, 0.2)$, saying class 1 is wrong with probability 0.5, class 2 with probability 0.7, class 3 with probability 0.8. Say class 1. The expected error rate is 0.5, and no decision does better; that residual is the problem's own uncertainty.

Choosing the most probable class is optimal *only when all mistakes cost the same.* When they do not, Bayes decision theory still applies: choose the action with the smallest conditional expected cost, which need not be the most probable class.

Theorem 2.2.9 (Cost-sensitive threshold for a binary decision)**.** *Consider a binary problem in which class* 1 *is an event to detect,* $p(x) = \mathbb{P}(Y = 1 \mid X = x)$*, correct decisions cost* 0*, a false positive (declaring* 1 *when the truth is* 0*) costs* $C_{\mathrm{FP}} > 0$ *and a false negative (declaring* 0 *when the truth is* 1*) costs* $C_{\mathrm{FN}} > 0$*. The Bayes rule declares positive if and only if*

$$p(x) > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}$$

(with either decision optimal at equality). Equal costs give the threshold $\frac{1}{2}$*.*

Proof. Deciding "positive" is wrong exactly when $Y = 0$, which has probability $1 - p(x)$, so its conditional expected cost is $C_{\mathrm{FP}}\,[1 - p(x)]$. Deciding "negative" is wrong exactly when $Y = 1$, so its cost is $C_{\mathrm{FN}}\,p(x)$. Positive is better when

$$\begin{aligned} C_{\mathrm{FP}}(1 - p(x)) < C_{\mathrm{FN}}\,p(x) &\iff C_{\mathrm{FP}} < (C_{\mathrm{FP}} + C_{\mathrm{FN}})\,p(x) \\ &\iff p(x) > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}. \end{aligned}$$

With $C_{\mathrm{FP}} = C_{\mathrm{FN}}$ the right side is $\frac{1}{2}$. □

Reading the formula

Two numbers to compare, nothing more.

- Cost of raising the alarm $= C_{\mathrm{FP}} \times \mathbb{P}(\text{no event}) = C_{\mathrm{FP}}\,(1 - p)$.
- Cost of staying quiet $= C_{\mathrm{FN}} \times \mathbb{P}(\text{event}) = C_{\mathrm{FN}}\,p$.

Cyberattack detection with $C_{\mathrm{FN}} = 10$ (missed intrusion) and $C_{\mathrm{FP}} = 1$ (an hour of investigation): threshold $1/(1 + 10) \approx 0.09$. At $p(x) = 0.2$: alarm costs $1 \times 0.8 = 0.8$, silence costs $10 \times 0.2 = 2$. Raise the alarm, even though an attack is *unlikely.* Expensive false negatives push the threshold down; expensive false positives push it up.

Example 2.2.10 (Where the same probability gives different decisions)**.**

- *Cyberattack detection.* Missed attack: prolonged intrusion. False alarm: investigation time. Threshold well below 0.5; accept many false alarms.
- *Card fraud with amount-dependent cost.* Blocking a legitimate payment costs a fixed $C_{\mathrm{FP}} = 5$ (customer annoyance). Missing fraud costs the amount A of the transaction: $C_{\mathrm{FN}} = A$. The threshold is $5/(5 + A)$. For a 50 MAD purchase, block if $p > 0.09$; for 1000 MAD, block if $p > 0.005$.

The threshold depends on x through the amount: the decision criterion is more elaborate than one fixed number.

- *Court trial.* Wrongful conviction (false positive) is usually treated as far worse than wrongful acquittal. Threshold far above 0.5: "beyond reasonable doubt."
- *Medical screening.* A missed disease is costly, an unnecessary follow-up test is cheap. Threshold below 0.5.

In each case the probability $p(x)$ is the same kind of object; only the loss differs. Probability estimation and decision policy are separate design choices.

Example 2.2.11 (Three classes and a cost table: the most probable class loses)**.** A triage system estimates that a patient is healthy, mildly ill or severely ill with probabilities $(0.5, 0.3, 0.2)$. Three actions: discharge, monitor, treat. Costs (rows: truth; columns: action):

	discharge	monitor	treat
healthy	0	1	3
mild	4	0	2
severe	20	6	0

Conditional expected costs: discharge $= 0.5(0)+0.3(4)+0.2(20) = 5.2$; monitor $= 0.5(1) + 0.3(0) + 0.2(6) = 1.7$; treat $= 0.5(3) + 0.3(2) + 0.2(0) = 2.1$. The Bayes decision is *monitor*, although "healthy" is the most probable state and would be the zero–one answer. The 20 in the corner (discharging a severe case) dominates the calculation.

Technique: Solving a Bayes decision problem by hand

1. Write the conditional probabilities of the outcomes at the given x as a row.
2. Write the loss table: one row per outcome, one column per action.
3. For each column, take the dot product with the probability row. That is the conditional risk of the action.
4. Pick the smallest column. For regression with a formula loss, use the shortcuts: squared $\to$ mean, absolute $\to$ median, τ-asymmetric $\to$ τ-quantile.
5. For binary detection, skip the table: compare $C_{\mathrm{FP}}(1-p)$ with $C_{\mathrm{FN}}\,p$, or use the threshold $C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$.

Classic problems

- $Y \mid x$ *is* 0 *with probability* 0.6 *and* 10 *with probability* 0.4. *Best prediction under squared loss? Under absolute loss?* Mean 4; median 0 (since $\mathbb{P}(Y \le 0) = 0.6 \ge \frac{1}{2}$ and $\mathbb{P}(Y \ge 0) = 1$).
- $C_{\mathrm{FN}} = 4\,C_{\mathrm{FP}}$. *Threshold?* $1/(1+4) = 0.2$.
- *Bayes error rate with class probabilities* $(0.45, 0.35, 0.20)$*?* $1 - 0.45 = 0.55$: no classifier can do better at this x.

- *When is the zero–one Bayes classifier also optimal under unequal costs?* Whenever the probabilities are so lopsided that the ranking of conditional risks does not change, e.g. $p(x) = 0.999$ for any moderate cost ratio. In general it is not.

Gotcha

- "Predict the most likely value" is the Bayes rule for zero–one loss only. Under squared loss the mean may be a value that never occurs (230 minutes in Example 2.2.7).
- The threshold formula has C_{FP} on top. Students often invert it: remember that expensive false negatives (C_{FN} large) must give a *small* threshold.
- The Bayes rule needs the true $\mathbb{P}(Y \mid X = x)$. In practice we only have estimates; Section 2.3 onward is about getting them.

Summary: Decisions with a known distribution

- Probabilities describe uncertainty; the loss describes consequences; together they determine the optimal action via $f^*(x) = \arg\min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$.
- Squared loss → conditional mean (Theorem 2.2.4); absolute loss → median; asymmetric → quantile.
- Zero–one loss → most probable class (Theorem 2.2.8); unequal costs → threshold $C_{\text{FP}}/(C_{\text{FP}}+C_{\text{FN}})$ (Theorem 2.2.9), or a cost-table comparison in general.
- "Best prediction" has no meaning until the loss is specified.

2.3 From Known Distributions to Finite Data

What are we trying to get?

We never know $\mathbb{P}(X, Y)$. What we have is a finite pile of examples. We want the most direct possible way to imitate the Bayes rule from examples (look at nearby ones), to see exactly what "nearby" depends on, and to understand why this direct approach breaks down when inputs have many features.

The training sample

Definition 2.3.1 (Training dataset, i.i.d. assumption)**.** A *training dataset* is a finite collection

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$$

of N realised input–output pairs. Superscripts index examples, subscripts index components: $x_j^{(i)}$ is feature j of example i. In the standard supervised setting, the pairs are modelled as *independent and identically distributed* (i.i.d.) draws from $\mathbb{P}(X, Y)$: one draw does not alter another, and all follow the same distribution.

Reading the formula

$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ is a table with N rows. Row i holds the d features of example i and its label. With $N = 3$ houses and $d = 2$ features (area, rooms):

i	$x_1^{(i)}$ (area)	$x_2^{(i)}$ (rooms)	$y^{(i)}$ (price)
1	145	4	1.2
2	80	2	0.7
3	200	5	1.9

$x_1^{(2)} = 80$: the area of the second house. The i.i.d. assumption says the three rows are like three independent tickets drawn from the same urn. It can fail: consecutive days of a time series are dependent; houses in one street are dependent; and if the market changes, future draws come from a different urn (*distribution shift*).

The distinction is fundamental:

True distribution $\mathbb{P}(X, Y)$	Training data $\mathcal{D}$
unknown	observed finite sample

If the distribution were known, Definition 2.2.2 would give the optimal rule. Since we have only the sample, the task of learning is to use it to construct a rule that approximates what we would want under the unknown distribution. This move from a known distribution to finite observations is *the* central statistical step in supervised learning.

Estimating from nearby examples

To estimate the average price of houses "like this one," average the prices of the training houses most like this one.

The Bayes rules suggest a direct strategy. For regression we need the conditional mean at x. If many training examples had exactly the input x, we would average their outputs. Exact repetitions are rare when inputs contain continuous features, so we use examples whose representations are *close* to x.

Definition 2.3.2 (k-nearest neighbours)**.** Let $N_k(x)$ be the set of indices of the k training inputs nearest to x (for a chosen distance). The *kNN regression estimate* and the *kNN class-probability estimate* are

$$\widehat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}, \qquad \widehat{p}_k(Y = c \mid X = x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c).$$

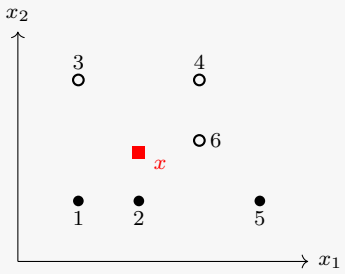
Under zero–one loss, the kNN classifier chooses $\arg\max_c \widehat{p}_k(Y = c \mid X = x)$: the *majority vote* among the neighbours.

Reading the formula

$\sum_{i \in N_k(x)}$ means "add up over the k neighbours." $\frac{1}{k}\sum y^{(i)}$ is their average label. $\frac{1}{k}\sum \mathbf{1}(y^{(i)} = c)$ counts how many neighbours have class c and divides by k: the *fraction* of neighbours in class c. If $k = 5$ and three neighbours are class B, then $\widehat{p}_5(Y = \mathrm{B} \mid x) = 3/5$, and majority vote says B.

Example 2.3.3 (kNN by hand)**.** Six training points in the plane, each with a class and a numerical value:

i	$x_1^{(i)}$	$x_2^{(i)}$	class	$y^{(i)}$
1	1	1	A	10
2	2	1	A	12
3	1	3	B	20
4	3	3	B	22
5	4	1	A	14
6	3	2	B	18



(filled: class A; open: class B.) Query $x = (2, 1.8)$. Squared Euclidean distances: to point 1: $1 + 0.64 = 1.64$; point 2: $0 + 0.64 = 0.64$; point 3: $1 + 1.44 = 2.44$; point 4: $1 + 1.44 = 2.44$; point 5: $4 + 0.64 = 4.64$; point 6: $1 + 0.04 = 1.04$. Order of closeness: $2, 6, 1, \{3, 4\}, 5$.

- $k = 1$: neighbour $\{2\}$. Class A; regression 12.
- $k = 3$: neighbours $\{2, 6, 1\}$, classes A, B, A. Estimated $\widehat{p}(\mathrm{A}) = 2/3$: class A; regression $(12 + 18 + 10)/3 = 13.3$.
- $k = 5$: neighbours $\{2, 6, 1, 3, 4\}$, classes A, B, A, B, B. $\widehat{p}(\mathrm{B}) = 3/5$: class B; regression $(12 + 18 + 10 + 20 + 22)/5 = 16.4$.

The decision flips from A to B as k grows: small k listens to the single closest point, large k averages over a wider region.

The meaning of "nearest" depends on the representation and on the distance used. The neighbourhood size k controls how local the estimate is: with small k the prediction follows individual observations and is sensitive to noise; with large k it averages over a broad region and can erase genuine local variation. k is a *hyperparameter*: it is chosen as part of model development (Section 2.6), not estimated by the averaging rule itself. Geometrically, for $k = 1$ the input space is partitioned into *Voronoi cells*: the cell of training point $x^{(i)}$ is the set of points closer to $x^{(i)}$ than to any other training point, and every query in that cell receives the label $y^{(i)}$.

Distance defines the neighbourhood

Definition 2.3.4 (Minkowski, Euclidean, Manhattan, Hamming distances)**.** For $x, z \in \mathbb{R}^d$ and $p \geq 1$, the *Minkowski distance* is

$$d_p(x, z) = \Big(\sum_{j=1}^{d} |x_j - z_j|^p \Big)^{1/p}.$$

$p = 1$ gives the *Manhattan* distance (sum of coordinate gaps), $p = 2$ the *Euclidean* distance (straight-line length), and $p \to \infty$ the largest coordinate gap. For binary vectors, the *Hamming* distance counts the positions where they differ. A *scaled, weighted* Euclidean distance is

$$d_w(x, z) = \sqrt{\sum_{j=1}^{d} w_j \Big(\frac{x_j - z_j}{s_j} \Big)^2},$$

where s_j is a scale (units, spread) and $w_j \geq 0$ a weight ($w_j = 0$ ignores feature j).

Reading the formula

For $x = (1, 2)$ and $z = (3, 5)$: coordinate gaps 2 and 3. Manhattan $d_1 = 2 + 3 = 5$ (walk along the streets). Euclidean $d_2 = \sqrt{2^2 + 3^2} = \sqrt{13} \approx 3.61$ (as the crow flies). Largest gap $d_\infty = 3$. Hamming distance between 10110 and 11100: positions 2 and 4 differ, so 2. In d_w, dividing by s_j converts each feature to "how many typical spreads apart," and w_j says how much that feature should count.

Example 2.3.5 (Scaling decides who the neighbours are)**.** Two customers, described by (income in USD, age in years): $P = (30{,}000, 25)$ and $Q = (31{,}000, 60)$. Query: $(30{,}800, 25)$, a 25-year-old earning 30,800. Raw Euclidean distances: to P: $\sqrt{800^2 + 0^2} = 800$; to Q: $\sqrt{200^2 + 35^2} = \sqrt{41{,}225} \approx 203$. The nearest neighbour is Q, a 60-year-old, because income differences of a few hundred dollars dwarf age differences of decades *in these units*. Standardise with typical spreads $s_{\text{income}} = 1000$, $s_{\text{age}} = 10$: $P \to (30.0, 2.5)$, $Q \to (31.0, 6.0)$, query $\to (30.8, 2.5)$. Now to P: 0.8; to Q: $\sqrt{0.04 + 12.25} \approx 3.5$. The nearest neighbour is P, the customer of the same age. Nothing about the customers changed; the choice of units silently chose the answer. Standardising each feature, $z_j = (x_j - \mu_j)/\sigma_j$ with μ_j, σ_j its mean and standard deviation on the training data, is the usual remedy.

Gotcha

Compute μ_j and σ_j on the *training* data only and reuse them for new inputs. Recomputing them on the test data leaks information and makes evaluation optimistic (Section 2.6).

The curse of dimensionality

kNN needs enough observations near the query. Two ways to see that high-dimensional inputs defeat this.

In high dimension, "nearby" points do not exist: everything is far from everything.

Proposition 2.3.6 (Grid count)**.** *Covering $[0, 1]^d$ with a grid of 10 positions per coordinate requires 10^d locations. Going from $d = 2$ to $d = 10$ raises this*

from 10^2 *to* 10^{10}.

Proof. Each of the d coordinates is chosen independently among 10 positions, giving $10 \times \cdots \times 10 = 10^d$ combinations. □

Proposition 2.3.7 (Probability of finding a neighbour)**.** *Let n points be uniformly distributed in the unit ball of $\mathbb{R}^d$ and let the query sit at its centre. Call a point a "true neighbour" if its distance to the query is below 0.1. The probability that a given point is a true neighbour is $p_d = (0.1)^d = 10^{-d}$, and the probability that at least one of the n points is a true neighbour is*

$$\mathbb{P}(\textit{at least one neighbour}) = 1 - \left(1 - 10^{-d}\right)^n.$$

Proof. A uniform point lands in a region with probability equal to the region's volume divided by the ball's volume. The ball of radius r in $\mathbb{R}^d$ has volume $V_d(r) = r^d V_d(1)$ (scaling every coordinate by r scales d-dimensional volume by r^d), so $p_d = V_d(0.1)/V_d(1) = 0.1^d$. The n points are independent, so the probability that *none* is a neighbour is $(1 - p_d)^n$, and the complement gives the claim. □

Reading the formula

With $n = 1000$ points:

d	1	2	3	4	5	6	10
$\mathbb{P}(\geq 1$ neighbour)	$\approx 100\%$	99.996%	63.2%	9.5%	1.0%	0.1%	10^{-5} %

For $d = 3$: $1-(1-0.001)^{1000} \approx 1-e^{-1} \approx 0.632$. For $d = 4$: $1-(1-10^{-4})^{1000} \approx 1 - e^{-0.1} \approx 0.095$. With a thousand data points and ten features, the chance that *any* training point is genuinely close to the query is one in ten million. The "neighbours" kNN uses are then far away, and the local averaging idea loses its justification.

This does not mean every high-dimensional problem is hopeless. It shows that *local* estimation cannot exploit high-dimensional inputs efficiently without additional structure, and motivates a different strategy: describe how predictions vary across the input space through a *shared* model and learn its adjustable quantities from all training examples at once (Section 2.4).

Computation

kNN is a *lazy* method: nothing is computed at training time; all the work happens at prediction time. The basic implementation stores all N inputs, memory $O(Nd)$, and computes N distances per query, each costing $O(d)$, for $O(Nd)$ per prediction. Tree structures help in low dimension; approximate nearest-neighbour search and batched distance computations help at scale.

Technique: kNN in practice

- Standardise features first (Example 2.3.5); one-hot categorical features (Example 2.1.2).
- Choose k on validation data (Section 2.6); odd k avoids ties in binary classification.

- For a hand computation, use *squared* Euclidean distances: same ordering, no square roots.
- Distrust kNN when d is large relative to N (Proposition 2.3.7).

Classic problems

- *In Example 2.3.3, what does $k = 6$ predict?* All points: $\widehat{p}(\mathrm{A}) = 3/6$: a tie; regression $(10 + 12 + 20 + 22 + 14 + 18)/6 = 16$. With $k = N$ every query gets the same answer: the global average.
- $n = 1000$, $d = 3$, *radius* 0.1*: expected number of true neighbours?* $np_d = 1000 \times 10^{-3} = 1$.
- *What is the* 1*-NN training error?* Zero (each training point is its own nearest neighbour), which says nothing about generalization; recall the memoriser of Chapter 1.

Summary: From distribution to data

- Data $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, modelled as i.i.d. draws from the unknown $\mathbb{P}(X, Y)$.
- kNN imitates the Bayes rule locally: average (regression) or vote (classification) over the k nearest training points.
- "Nearest" depends on the distance and on feature scaling; standardise.
- Curse of dimensionality: $\mathbb{P}(\text{a neighbour within } 0.1) = 1 - (1 - 10^{-d})^n$ collapses as d grows; local methods need shared structure to survive in high dimension.

2.4 Parameterized Models

What are we trying to get?

Since finite data cannot tell us what to do at every input, we restrict ourselves in advance to a *family* of candidate rules and let the data pick one member. We want to see what such a family looks like, what its "knobs" are, and which knobs are learned versus chosen by us.

Definition 2.4.1 (Parameterized model, hypothesis space)**.** A *parameterized model* is a function $f_\theta(x)$ whose behaviour is controlled by a vector of adjustable numbers $\theta \in \Theta$, the *parameters*. The set of functions the model can represent,

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\},$$

is the *hypothesis space*. The model *form* determines how parameters are shared across inputs; θ selects one member of the family. Choosing $\mathcal{H}$ introduces structure before any data are seen: an *inductive bias*.

Reading the formula

Think of f_θ as a machine with dials. θ is the dial settings; $\mathcal{H}$ is everything the machine can do across all settings. *Linear regression*: $f_\theta(x) = w^\top x + b$ with $\theta = (w, b)$. For $w = (2, -1)$, $b = 3$, and input $x = (1, 4)$: $f_\theta(x) = 2 \cdot 1 + (-1) \cdot 4 + 3 = 1$. Turning the dials to $w = (0, 1)$, $b = 0$ gives a different linear function, $f(x) = x_2$. Every function in $\mathcal{H}$ is a flat surface (a line in one dimension, a plane in two); no dial setting can produce a curve. That restriction is the inductive bias of linear regression.

Example 2.4.2 (Parameters versus model form)**.** A polynomial model in one variable, $f_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_p x^p$, makes the distinction clear. Changing the coefficients $\theta_0, \ldots, \theta_p$ selects another polynomial *of the same degree*: same form, different parameters. Changing the degree p changes the *family* being considered: with $p = 1$ only lines are available, with $p = 2$ parabolas as well. The coefficients are learned; the degree is a decision made before learning.

Definition 2.4.3 (Nested hypothesis spaces)**.** Hypothesis spaces $\mathcal{H}_1 \subseteq \mathcal{H}_2$ are *nested* when every function of the first is also available in the second. Polynomials of degree $\leq p$ are nested in polynomials of degree $\leq p + 1$ (set $\theta_{p+1} = 0$).

Classification models output probabilities

Definition 2.4.4 (Probability simplex, probabilistic classifier)**.** For C-class classification, a model estimates the vector of conditional class probabilities

$$\widehat{p}_\theta(x) = \big[\widehat{p}_\theta(Y = 1 \mid x),\ \ldots,\ \widehat{p}_\theta(Y = C \mid x)\big]^\top \in \Delta^{C-1},$$

where the *probability simplex* is

$$\Delta^{C-1} = \Big\{p \in \mathbb{R}^C : p_c \geq 0,\ \sum_{c=1}^{C} p_c = 1\Big\}.$$

A decision rule (Section 2.2) then converts this vector into an action.

Reading the formula

Δ^{C-1} is just "all lists of C nonnegative numbers adding to 1," i.e. all legitimate probability assignments over C classes. For $C = 3$: $(0.5, 0.3, 0.2)$ is in it; $(0.5, 0.5, 0.5)$ is not (sums to 1.5); $(1.2, -0.2, 0)$ is not (negative entry). A common way to build such a model is to compute C real-valued *scores* $s_{\theta,c}(x)$ and pass them through the *softmax* map $p_c = e^{s_c} / \sum_{c'} e^{s_{c'}}$: scores $(2, 0, -1)$ become $(7.39, 1, 0.37)/8.76 \approx (0.84, 0.11, 0.04)$, automatically nonnegative and summing to one.

Parameters and hyperparameters

Definition 2.4.5 (Parameters, hyperparameters)**.** *Parameters* are quantities learned from the training examples. *Hyperparameters* are choices made around that estimation: some define the hypothesis space (polynomial degree, number of layers), others control the learning procedure (number of neighbours k, step sizes).

$$\text{model form} + \text{hyperparameters} \;\to\; \mathcal{H} \;\to\; \theta \;\to\; \text{particular model.}$$

Model	Parameters (learned)	Hyperparameters (chosen)
kNN	none	k, the distance, feature scales
Linear regression	w, b	which features to include
Polynomial regression	coefficients $\theta_0, \ldots, \theta_p$	degree p
Neural network	all weights	layers, width, learning rate

Gotcha

kNN has hyperparameters but *no* parameters: nothing is fitted, the data themselves are the model. That is why it is called lazy, and why k cannot be "learned" by the averaging rule: with $k = 1$ the training error is always zero, so training data alone would always choose $k = 1$.

Summary: Parameterized models

- f_θ: a function with adjustable parameters; $\mathcal{H}$: all functions it can produce. Choosing $\mathcal{H}$ is choosing an inductive bias.
- Linear and polynomial regression; probabilistic classifiers output a vector in the simplex Δ^{C-1}.
- Parameters are learned; hyperparameters are chosen and define the family or the procedure.

2.5 Learning the Parameters from Data

What are we trying to get?

We have a family of candidate rules and a pile of examples. We want a principle that says which parameter values the data prefer, and we want to understand exactly what that principle can and cannot promise: it finds the best fit to the examples, which is not the same as the best rule for the world.

Two losses, two jobs

Learning a parameterized model requires a criterion for comparing parameter values. It is useful to distinguish two losses:

$$\ell_{\text{train}}: \text{ guides the learning of the model from data,}$$
$$\ell_{\text{dec}}: \text{ guides the choice of an action from inferred information.}$$

They can coincide, as when squared loss is used both to fit and to assess a regression. They can differ: a classifier may be *trained* with cross-entropy to estimate probabilities and then *decide* with zero–one loss or application costs.

Population risk and empirical risk

Definition 2.5.1 (Population risk, empirical risk, ERM)**.** For parameters θ, the *population risk* is the average training loss over the whole distribution,

$$R(\theta) = \mathbb{E}_{(X,Y)\sim\mathbb{P}}\big[\ell_{\text{train}}\big(Y, f_\theta(X)\big)\big],$$

and the *empirical (training) risk* is the average over the observed examples,

$$\widehat{R}_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell_{\text{train}}\big(y^{(i)}, f_\theta(x^{(i)})\big).$$

Empirical risk minimisation (ERM) chooses parameters that make the training risk as small as possible:

$$\widehat{\theta} \in \arg\min_{\theta\in\Theta} \widehat{R}_{\text{train}}(\theta).$$

The best parameters *in the population sense* are $\theta^*_{\mathcal{H}} \in \arg\min_\theta R(\theta)$; their population risk may exceed the Bayes risk when $\mathcal{H}$ does not contain the Bayes rule.

Reading the formula

Three quantities, in increasing order of how much we can compute them.

- $R(\theta)$: "if I used dial setting θ on *every* house that will ever exist, what would my average loss be?" This is what we care about and can never compute, because it needs $\mathbb{P}$.
- $\widehat{R}_{\text{train}}(\theta)$: "on the N houses in my table, what *was* my average loss?" Computable. With three examples whose squared errors under θ are $1, 4, 0$: $\widehat{R} = (1+4+0)/3 = 5/3$.
- $\widehat{\theta}$: the dial setting with the smallest $\widehat{R}_{\text{train}}$. The hat says "computed from this particular sample"; another sample would give another $\widehat{\theta}$.

Key insight

ERM replaces an average over the unknown world by an average over the sample. Everything that can go wrong in learning is contained in the gap between R and $\widehat{R}$, and the next two propositions say when that gap is harmless and when it is not.

Proposition 2.5.2 (For a fixed model, training risk is an honest estimate)**.** *Fix θ (chosen without looking at the data). If the examples are i.i.d. from $\mathbb{P}$, then $\mathbb{E}[\widehat{R}_{\text{train}}(\theta)] = R(\theta)$, and by the law of large numbers $\widehat{R}_{\text{train}}(\theta) \to R(\theta)$ as $N \to \infty$.*

Proof. $\widehat{R}_{\text{train}}(\theta)$ is the average of N terms $\ell_{\text{train}}(y^{(i)}, f_\theta(x^{(i)}))$, each an independent copy of the random variable $\ell_{\text{train}}(Y, f_\theta(X))$, whose mean is $R(\theta)$ by definition. The expectation of an average of terms with mean $R(\theta)$ is $R(\theta)$. The convergence statement is the law of large numbers for i.i.d. terms with finite mean. □

Proposition 2.5.3 (After minimisation, training risk is optimistic)**.** *Let $\widehat{\theta}$ be the ERM solution. Then*

$$\mathbb{E}[\widehat{R}_{\text{train}}(\widehat{\theta})] \;\leq\; \min_{\theta} R(\theta) \;\leq\; \mathbb{E}[R(\widehat{\theta})].$$

In words: the training risk of the fitted model underestimates, on average, both the best achievable population risk and the population risk of the fitted model itself.

Proof. Let θ^* minimise R. Since $\widehat{\theta}$ minimises $\widehat{R}_{\text{train}}$, $\widehat{R}_{\text{train}}(\widehat{\theta}) \leq \widehat{R}_{\text{train}}(\theta^*)$ for every sample. Taking expectations and using Proposition 2.5.2 for the fixed θ^*: $\mathbb{E}[\widehat{R}_{\text{train}}(\widehat{\theta})] \leq \mathbb{E}[\widehat{R}_{\text{train}}(\theta^*)] = R(\theta^*) = \min_\theta R(\theta)$. The second inequality holds because $R(\widehat{\theta}) \geq \min_\theta R(\theta)$ for every sample. □

Reading the formula

The fitted model has been *chosen because* it looks good on these particular examples. Part of its good training score is real, part is luck that will not recur. The proposition quantifies that "the training score is at least as flattering as the truth." A small training risk confirms a good fit to the observed examples; it is only an (optimistic) estimate of performance on unseen ones.

Proposition 2.5.4 (Bigger families fit the training data at least as well)**.** *If $\mathcal{H}_1 \subseteq \mathcal{H}_2$, then $\min_{f\in\mathcal{H}_2} \widehat{R}_{\text{train}}(f) \leq \min_{f\in\mathcal{H}_1} \widehat{R}_{\text{train}}(f)$. In particular, increasing the polynomial degree never increases the minimum training risk.*

Proof. The minimum over a larger set is at most the minimum over a subset, because every candidate of $\mathcal{H}_1$ is also a candidate in $\mathcal{H}_2$. □

Proposition 2.5.5 (Enough flexibility fits anything)**.** *Given N examples with distinct inputs $x^{(1)}, \ldots, x^{(N)} \in \mathbb{R}$, there is a polynomial of degree at most $N-1$ passing through all N points; its training risk under any loss with $\ell_{\text{train}}(y, y) = 0$ is zero.*

Proof. For each i, the polynomial $L_i(x) = \prod_{j\neq i} \frac{x - x^{(j)}}{x^{(i)} - x^{(j)}}$ has degree $N-1$, equals 1 at $x^{(i)}$ and 0 at every other $x^{(j)}$ (the denominators are nonzero because the inputs are distinct). Then $f(x) = \sum_{i=1}^{N} y^{(i)} L_i(x)$ has degree at most $N-1$ and $f(x^{(i)}) = y^{(i)}$ for all i, so every training loss term is $\ell_{\text{train}}(y^{(i)}, y^{(i)}) = 0$. □

Interpretation & discussion

Propositions 2.5.4 and 2.5.5 together explain why the training risk is a *poor guide for choosing model complexity*: making the family more flexible (higher degree, smaller k in kNN) can only lower the training risk, all the way to zero, even after performance on new examples has started to deteriorate. Parameter estimation and hyperparameter selection therefore need different evidence: training data estimate parameters *inside* each fixed configuration; separate validation data compare the fitted configurations (Section 2.6).

Example 2.5.6 (ERM solved by hand: least squares in one variable)**.** Some objectives have an analytical solution. Fit $f_\theta(x) = wx + b$ to the four points

$(0,1),(1,3),(2,2),(3,4)$ under squared loss. The training risk is $\widehat{R}(w,b) = \frac{1}{4}\sum_i (y^{(i)} - wx^{(i)} - b)^2$. Setting both partial derivatives to zero,

$$\frac{\partial \widehat{R}}{\partial b} = -\frac{2}{N}\sum_i (y^{(i)} - wx^{(i)} - b) = 0, \qquad \frac{\partial \widehat{R}}{\partial w} = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)} - b) = 0,$$

gives the *normal equations* $\sum y = w\sum x + Nb$ and $\sum xy = w\sum x^2 + b\sum x$, whose solution is

$$w = \frac{N\sum xy - \sum x \sum y}{N\sum x^2 - (\sum x)^2}, \qquad b = \frac{\sum y - w\sum x}{N}.$$

Here $N = 4$, $\sum x = 6$, $\sum y = 10$, $\sum x^2 = 14$, $\sum xy = 0 + 3 + 4 + 12 = 19$. So $w = (76 - 60)/(56 - 36) = 0.8$ and $b = (10 - 4.8)/4 = 1.3$. Predictions $1.3, 2.1, 2.9, 3.7$; residuals $-0.3, 0.9, -0.9, 0.3$; squared $0.09, 0.81, 0.81, 0.09$; training risk $1.8/4 = 0.45$.

Nested families in action. Degree 0 (a constant): best constant is the mean 2.5, residuals $-1.5, 0.5, -0.5, 1.5$, risk $5/4 = 1.25$. Degree 1: 0.45. Degree 3: by Proposition 2.5.5 a cubic passes through all four points, risk 0. The sequence $1.25, 0.45, \ldots, 0$ is non-increasing, exactly as Proposition 2.5.4 says, and it says nothing about which degree predicts a fifth point best.

Learning class probabilities with cross-entropy

Zero–one loss only notices which *class won. Cross-entropy notices* how much *probability the true class got, so it can guide small improvements.*

For classification, the natural decision loss is zero–one, $\ell_{01}(y,a) = \mathbf{1}(y \neq a)$, with decision $a(x) = \arg\max_c \widehat{p}_{\theta,c}(x)$. As a *training* loss it is unsuitable. Direct minimisation would use

$$\widehat{R}_{01}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\Big(y^{(i)} \neq \arg\max_c s_{\theta,c}(x^{(i)})\Big),$$

which (i) discards changes in confidence, (ii) is piecewise constant in θ, and (iii) has zero gradient wherever it is defined and no gradient where the decision flips.

Example 2.5.7 (Why the zero–one training risk cannot be optimised smoothly)**.** One feature, a threshold classifier $f_\theta(x) = 1$ if $x > \theta$, else 0. Data: $x = 1, 2, 3, 4$ with labels $0, 0, 1, 1$. Then $\widehat{R}_{01}(\theta)$ is $2/4$ for $\theta < 1$, $1/4$ for $1 \le \theta < 2$, 0 for $2 \le \theta < 3$, $1/4$ for $3 \le \theta < 4$, $2/4$ for $\theta \ge 4$: a staircase. Between the steps the slope is exactly 0, so "move θ in the direction that decreases the loss" gets no information; at the steps the function jumps. Any method that improves by small steps is blind. A useful training loss must vary smoothly with the model output and reward better probability estimates.

Definition 2.5.8 (Cross-entropy training loss)**.** Let $\widehat{p}_{\theta,c}(x) = \widehat{p}_\theta(Y = c \mid X = x)$ be the model's probability for class c. For an example $(x^{(i)}, y^{(i)})$, the *cross-entropy loss* is

$$\ell_{\mathrm{CE}}(y^{(i)}, \widehat{p}_\theta(x^{(i)})) = -\log \widehat{p}_{\theta,y^{(i)}}(x^{(i)}),$$

the negative log of the probability assigned to the class that actually occurred. The *empirical cross-entropy* is

$$\widehat{R}_{\mathrm{CE}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \widehat{p}_{\theta,y^{(i)}}(x^{(i)}), \qquad \widehat{\theta}_{\mathrm{CE}} \in \arg\min_\theta \widehat{R}_{\mathrm{CE}}(\theta).$$

For binary classification, writing $\widehat{p} = \widehat{p}_\theta(Y = 1 \mid x)$ and $y \in \{0, 1\}$, this is $\ell_{\mathrm{BCE}}(y, \widehat{p}) = -y \log \widehat{p} - (1 - y) \log(1 - \widehat{p})$.

Reading the formula

$\widehat{p}_{\theta, y^{(i)}}(x^{(i)})$ is a mouthful for a simple thing: "look up the probability the model gave to the *correct* class of example i." Then take minus its log. Natural logs:

probability given to the true class	0.9	0.6	0.1	0.01
loss $-\ln(\cdot)$	0.105	0.511	2.303	4.605

Three examples with true-class probabilities $0.9, 0.6, 0.1$ give $\widehat{R}_{\mathrm{CE}} = (0.105 + 0.511 + 2.303)/3 \approx 0.973$. The confident mistake (0.1) dominates. The binary formula is the same idea written for $y \in \{0, 1\}$: if $y = 1$ only the term $-\log \widehat{p}$ survives; if $y = 0$ only $-\log(1 - \widehat{p})$, the log-probability of the class that occurred.

Where does this formula come from? Encode the observed label of example i as a *one-hot* vector $t_c^{(i)} = \mathbf{1}(y^{(i)} = c)$: a probability vector that puts all its mass on the observed class. Its cross-entropy with the model's vector, in the sense of Definition 1.8.1, is $-\sum_c t_c^{(i)} \log \widehat{p}_{\theta,c}(x^{(i)}) = -\log \widehat{p}_{\theta, y^{(i)}}(x^{(i)})$, because only the term $c = y^{(i)}$ survives. The one-hot vector represents the *observed* label, not the unknown true distribution; averaging over many examples is what estimates the population quantity, as the next theorem shows.

Theorem 2.5.9 (Cross-entropy is minimised by the true probabilities)**.** *Let $p_c(x) = \mathbb{P}(Y = c \mid X = x)$ be the true conditional probabilities. At a fixed x, the conditional expected cross-entropy of the model's vector $\widehat{p}_\theta(x)$ is*

$$\mathbb{E}\big[-\log \widehat{p}_{\theta,Y}(x) \,\big|\, X = x \big] = H\big(p(x)\big) + \mathrm{D}_{\mathrm{KL}}\left(p(x) \,\|\, \widehat{p}_\theta(x)\right) \;\geq\; H\big(p(x)\big),$$

with equality iff $\widehat{p}_\theta(x) = p(x)$. Consequently the population cross-entropy risk decomposes as

$$R_{\mathrm{CE}}(\theta) = H(Y \mid X) + \mathbb{E}_X\big[\, \mathrm{D}_{\mathrm{KL}}\left(p(X) \,\|\, \widehat{p}_\theta(X)\right)\big],$$

where $H(Y \mid X) = \mathbb{E}_X[H(p(X))]$ is the conditional entropy, and R_{CE} is minimised, over all probability models, exactly by models that output the true conditional probabilities.

Proof. At fixed x, Y takes value c with probability $p_c(x)$, so $\mathbb{E}[-\log \widehat{p}_{\theta,Y}(x) \mid X = x] = -\sum_c p_c(x) \log \widehat{p}_{\theta,c}(x) = H(p(x), \widehat{p}_\theta(x))$, the cross-entropy of Definition 1.8.1. By equation (1.1) this equals $H(p(x)) + \mathrm{D}_{\mathrm{KL}}(p(x) \| \widehat{p}_\theta(x))$, and by Gibbs' inequality (Theorem 1.8.2) the divergence is ≥ 0 with equality iff the two vectors coincide. Averaging over X gives the decomposition of R_{CE}; the first term does not depend on θ and the second is ≥ 0 with equality iff $\widehat{p}_\theta(x) = p(x)$ for (almost) every x. □

Reading the formula

Two terms. $H(Y \mid X)$ is the uncertainty about the class that *remains even if you know x*: the problem's own noise, which no model can remove. The divergence term is *your* error: how far your probabilities sit from the true ones,

averaged over inputs. Training with cross-entropy pushes only the second term down. Recall the numbers of Example 1.8.3: with truth $(0.7, 0.3)$ the floor is 0.881 bits, a hedged $(0.5, 0.5)$ pays 0.119 extra, an overconfident $(0.9, 0.1)$ pays 0.222 extra.

Key insight

Cross-entropy is the unique combination of two facts: (i) the one-hot cross-entropy of an example is just $-\log$(probability of the observed class), which is computable from data; (ii) its expectation is $H + \mathrm{D}_{\mathrm{KL}}$, which is minimised at the truth (Gibbs). So minimising something we *can* compute drives the model toward something we *want*: calibrated probabilities, usable afterwards by any decision rule of Section 2.2.

Proposition 2.5.10 (Cross-entropy minimisation is maximum likelihood)**.** *Minimising $\widehat{R}_{\mathrm{CE}}(\theta)$ is equivalent to maximising the conditional likelihood of the observed labels, $L(\theta) = \prod_{i=1}^{N} \widehat{p}_{\theta, y^{(i)}}(x^{(i)})$.*

Proof. $\log L(\theta) = \sum_i \log \widehat{p}_{\theta, y^{(i)}}(x^{(i)}) = -N\, \widehat{R}_{\mathrm{CE}}(\theta)$. Since log is increasing and $-N < 0$, maximising L is the same as minimising $\widehat{R}_{\mathrm{CE}}$. □

Interpretation & discussion

Cross-entropy gives a graded assessment: predicting the observed class with probability 0.9 costs less than with 0.6, and a confident wrong prediction costs a lot. It is differentiable for positive probabilities, so with a smooth probability model gradient methods can optimise it (later in the course). Zero–one loss stays in the picture as the *decision* loss: train with cross-entropy, then decide with arg max or with a cost-sensitive threshold. The probabilities are learned once and can serve several decision rules.

Learning objective and learning algorithm

Definition 2.5.1 specifies a *learning objective*: which parameter values are preferred. A *learning algorithm* specifies how suitable values are found: some objectives have closed-form solutions (Example 2.5.6); most are solved approximately by iterative optimisation, producing a sequence $\theta_0, \theta_1, \ldots$. The complete sequence separates learning, inference and decision:

$$\mathcal{D} \xrightarrow{\ell_{\mathrm{train}}} \text{learned model } f_{\widehat{\theta}} \xrightarrow{\text{new } x} \text{inferred quantity} \xrightarrow{\ell_{\mathrm{dec}}} \text{action}.$$

Technique: Working with risks

- To compute an empirical risk by hand: one loss per example, then average. Keep a column of losses; the mistakes are usually in the arithmetic, not the concept.
- Squared-loss ERM for a line: use the normal-equation formulas of Example 2.5.6; check that residuals sum to zero (they must, from $\partial \widehat{R}/\partial b = 0$).
- Cross-entropy by hand: write the probability of the *true* class for each example, take $-\ln$, average. Never take the log of the largest probability by mistake.

- Training risk after fitting is optimistic (Proposition 2.5.3); never report it as the model's performance.

Classic problems

- *A model gives the true class probability* 0.5 *on every example. Empirical cross-entropy?* $-\ln 0.5 = 0.693$ nats, whatever N.
- *Degree-*5 *polynomial fitted to* 6 *points with distinct* x*: training risk?* Zero (Proposition 2.5.5).
- *Can a larger hypothesis space have larger minimum training risk?* No (Proposition 2.5.4). Can it have larger population risk for the fitted model? Yes, and that is the whole problem of overfitting.
- *Binary example,* $y = 0$*, model says* $\widehat{p} = 0.8$*. BCE loss?* $-\ln(1 - 0.8) = 1.609$.

Summary: Learning from data

- Population risk $R(\theta)$ (what we want, unknowable) versus empirical risk $\widehat{R}_{\text{train}}(\theta)$ (what we can compute). ERM minimises the latter.
- Training risk is honest for a fixed model (Proposition 2.5.2) and optimistic after fitting (Proposition 2.5.3); bigger families only lower it (Propositions 2.5.4, 2.5.5).
- Cross-entropy $-\log \widehat{p}_{\theta,y}(x)$ is a smooth training loss whose expectation is $H(Y \mid X) + \mathbb{E}[\mathrm{D}_{\mathrm{KL}}]$, minimised by the true probabilities (Theorem 2.5.9); it equals maximum likelihood.
- Objective (which θ is preferred) and algorithm (how to find it) are separate.

2.6 Evaluating a Learned Model

What are we trying to get?

Since training risk flatters the fitted model, we need evidence from examples that did *not* shape it. We want a disciplined way to split the data so that parameters, hyperparameters and the final performance estimate each rest on information not already used, and a way to do this when data are scarce.

Three subsets, three roles

Definition 2.6.1 (Training, validation and test sets)**.** The available data are partitioned into three subsets with different roles.

Subset	Main role
Training set $\mathcal{D}_{\text{train}}$	learn model parameters
Validation set $\mathcal{D}_{\text{val}}$	select models and hyperparameters
Test set $\mathcal{D}_{\text{test}}$	evaluate the final model, after all choices are fixed

Their sizes need not be equal; the defining difference is how information from

each subset is allowed to influence the learning procedure.

Every subset is informative about new data only until you use it to make a choice. Then it becomes part of "training" in disguise.

The training set enters parameter learning through ℓ_{train}. The validation set compares the fitted models and may select k in kNN, the degree of a polynomial, a representation, or a model family. Reusing validation results for many design decisions gradually adapts the development process to that set, so a third sample is reserved for the final assessment. The test set is used once, after parameters, hyperparameters and evaluation rules are fixed; consulting it repeatedly during development would turn it into another validation set and make its estimate optimistic, by exactly the mechanism of Proposition 2.5.3.

Proposition 2.6.2 (Held-out risk of a fixed model is honest)**.** *If the model f was chosen without using the examples of a held-out set of size M drawn i.i.d. from $\mathbb{P}$, then the held-out empirical risk $\frac{1}{M}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ has expectation equal to the population risk of f.*

Proof. Identical to Proposition 2.5.2: f is fixed relative to the held-out examples, so each term has mean equal to the population risk. □

The catch is the word *fixed*. Once we select, among several candidates, the one with the lowest validation risk, that minimum is optimistic (Proposition 2.5.3 with the validation set playing the role of the training set and the candidates playing the role of Θ). The test set restores honesty because nothing was selected on it.

Selecting hyperparameters

Definition 2.6.3 (Hyperparameter selection by validation)**.** Let λ denote a hyperparameter. For each candidate value, learn parameters on the training set,

$$\widehat{\theta}_\lambda = \arg\min_\theta \widehat{R}_{\text{train}}(\theta; \lambda),$$

then compare the fitted models on the validation set and keep the best,

$$\hat{\lambda} = \arg\min_\lambda \widehat{R}_{\text{val}}(\widehat{\theta}_\lambda; \lambda).$$

Reading the formula

Two nested "pick the best" operations, on two different datasets. *Inner:* for degree $\lambda = 2$, find the best parabola on the training points; for $\lambda = 3$, the best cubic; and so on. *Outer:* evaluate each of those fitted curves on the validation points and keep the degree whose fitted curve did best *there*. Concretely, suppose validation risks come out as

degree λ	0	1	2	3	5
training risk of $\widehat{\theta}_\lambda$	1.25	0.45	0.40	0.10	0.00
validation risk of $\widehat{\theta}_\lambda$	1.30	0.50	0.62	1.90	8.40

Training risk keeps falling (Proposition 2.5.4); validation risk is smallest at $\lambda = 1$. Choose the line, $\hat{\lambda} = 1$. The validation criterion may differ from ℓ_{train} when the application values something else (e.g. cost-weighted errors).

When data are limited: cross-validation

A fixed validation set leaves fewer observations for parameter learning and makes the selection depend on one particular split.

Definition 2.6.4 (K-fold cross-validation)**.** Divide the development data (training plus validation) into K folds of roughly equal size. For each candidate λ and each fold $j = 1, \dots, K$, train on the other $K - 1$ folds and compute the validation risk $\widehat{R}_{\text{val}}^{(j)}(\lambda)$ on fold j. The *cross-validation estimate* is the average

$$\widehat{R}_{\text{CV}}(\lambda) = \frac{1}{K} \sum_{j=1}^{K} \widehat{R}_{\text{val}}^{(j)}(\lambda),$$

and $\hat{\lambda}$ minimises it. Every development example serves as validation data exactly once. After selecting $\hat{\lambda}$, parameters are typically re-learned on all development data. The test set remains untouched until final evaluation. $K = N$ is *leave-one-out* cross-validation.

Reading the formula

Cut the deck into K piles. Round 1: hide pile 1, train on piles $2, \dots, K$, score on pile 1. Round 2: hide pile 2, and so on. Average the K scores. With $K = 3$ and candidates $k \in \{1, 3\}$ for kNN, suppose the fold scores are

	fold 1	fold 2	fold 3	$\widehat{R}_{\text{CV}}$
$k = 1$	0.40	0.20	0.60	0.40
$k = 3$	0.30	0.30	0.30	0.30

Choose $k = 3$. Notice that a single split using fold 2 alone would have chosen $k = 1$; averaging over folds protects against such accidents.

Technique: Splitting data without fooling yourself

- Decide the split *before* looking at any result. Typical: 60/20/20 or, with cross-validation, 80/20 development/test.
- Anything computed from data (feature scaling constants, vocabulary, imputed values) is computed on the training portion and *applied* to the others. Otherwise information leaks.
- Time series: split by time (train on the past, validate on the future); random splits leak the future into the past.
- Grouped data (several images of the same patient): keep a group entirely in one subset; otherwise the model is tested on patients it has seen.
- Touch the test set once. If you go back and change something afterwards, you need a new test set.

Gotcha

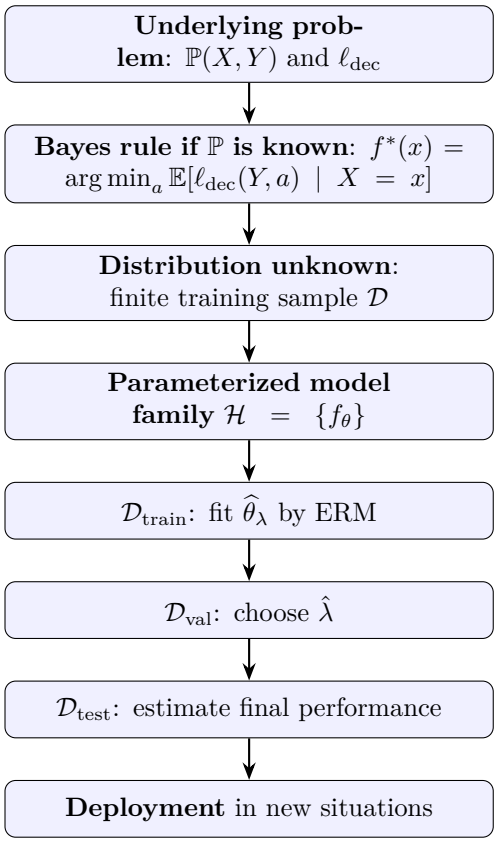
- "I chose k by minimising the validation error, so the validation error is my performance estimate." No: that minimum is optimistic (Proposition 2.5.3). Report the test error.

- Cross-validation selects hyperparameters; it does not by itself produce a final model. Refit on all development data with $\hat{\lambda}$.
- A validation set with 10 examples cannot distinguish a 12% error rate from an 18% one. Small held-out sets give noisy verdicts.

Summary: Evaluation

- Training data learn θ; validation data choose λ; test data estimate final performance once.
- Held-out risk is honest for a fixed model (Proposition 2.6.2) and optimistic after selecting among candidates; hence the third set.
- K-fold cross-validation averages K validation risks so that every example validates once; refit afterwards.

2.7 The Learning Problem in One Picture



The picture also clarifies the relationship with Chapter 1. Experience is represented by finite data; the hypothesis space specifies which rules can be considered; the loss measures the quality of their outputs. Training risk guides parameter estimation, validation performance guides model complexity, and test performance estimates how the completed procedure may behave on new observations. The four ingredients of Definition 1.6.1 appear as $\mathcal{D}$ (experience), $\mathcal{H}$ (model), ERM plus an optimiser (learning algorithm), and ℓ_{dec} evaluated on held-out data (evaluation criterion). Why finite samples provide reliable evidence about future performance is the subject of generalization, developed in Lecture 3.

Example 2.7.1 (The whole pipeline on one problem)**.** *Fraud detection.* Underlying problem: transactions X and fraud indicator Y; decision loss: $C_{\mathrm{FP}} = 5$ per blocked legitimate payment, $C_{\mathrm{FN}} =$ amount. Bayes rule: block iff $p(x) > 5/(5 + \text{amount})$; unknown, since p is unknown. Data: two years of labelled transactions, split by time into development (first 20 months) and test (last 4 months). Model family: a probabilistic classifier producing $\widehat{p}_\theta(x)$, with hyperparameters λ (representation, model size). Training: minimise empirical cross-entropy on $\mathcal{D}_{\mathrm{train}}$ for each λ. Validation: choose λ by cost-weighted error on $\mathcal{D}_{\mathrm{val}}$ (not by cross-entropy, because the application cares about money). Test: apply the Bayes threshold to $\widehat{p}_{\widehat{\theta}}$ on the final four months and report the total cost. Deployment: the threshold stays adjustable, because the decision policy is separate from the learned probabilities.

Notes on the original slides and notes

All formulas and numbers in the Lecture 2 material were rederived. The following points are worth attention.

- **"Mathematical proof" and "Proof" placeholder slides (12 and 17).** The slides leave the proofs of "squared loss → conditional mean" and "zero–one loss → most probable class" to the lecture. They are Theorems 2.2.4 and 2.2.8 here, together with the median and quantile cases the slides only tabulate.
- **Existence and uniqueness in the Bayes rule.** The notes write $f^*(x) \in \arg\min_a$ without conditions. The squared-loss result needs $\mathbb{E}[Y^2 \mid X = x] < \infty$; the median is generally *not* unique (Example 2.2.7: every value in $[100, 200]$); the argmax can tie. The "$\in$" in the notes is the right symbol; treat "the" Bayes rule as "a" Bayes rule.
- **Curse of dimensionality (slide 36).** The computation places the query at the centre of the ball; near the boundary the neighbourhood is partly outside the ball and the probability is smaller still, so the table is if anything optimistic. The notes' grid argument (10^d cells) and the slides' ball argument (10^{-d} per point) are two views of the same phenomenon (Propositions 2.3.6 and 2.3.7).
- **Slide 26 (kNN motivation).** "emprical" is a typo for "empirical"; "Ave$(Y \mid X \in N(x))$" is the same as the average in Definition 2.3.2.
- **Slide 14.** "Best prediction/prediction" is a duplicated word; the intended key point is "best prediction has no meaning until the loss is specified."
- **Cross-entropy and the true probabilities (slide 55, notes Section 4.2).** "The expected loss is minimised at $\widehat{p} = p$" is exactly right *over all probability vectors*; when the model family $\mathcal{H}$ cannot represent p, the minimiser within $\mathcal{H}$ is the member closest to p in the averaged KL sense of Theorem 2.5.9, which need not equal p.
- **Training risk as a guide to complexity (notes Section 5).** The notes assert that a more flexible family "can only improve or leave unchanged" the training fit and that some of the improvement is accidental. Propositions 2.5.4, 2.5.5 and 2.5.3 are the proofs.
- **Softmax.** The slides mention class scores $s_{\theta,c}$ on slide 50 without saying how scores become probabilities. The softmax map in the box after Defini-

tion 2.4.4 is the standard answer; it is not examinable from this lecture but makes the slide readable.

- **Slide 36 table,** $d = 1$**.** "$\approx 100\%$" is $1 - 0.9^{1000} \approx 1 - 1.7 \times 10^{-46}$; correct.

Chapter takeaways

Summary: From representations to decisions

- **Representation.** Objects become vectors $x \in \mathbb{R}^d$ by human choices; one-hot code categories; the representation bounds what can be learned.
- **Known distribution.** The Bayes rule $f^*(x) = \arg\min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$ is optimal (Theorem 2.2.3); squared loss gives the conditional mean, absolute the median, zero–one the most probable class, unequal costs the threshold $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$.
- **Finite data.** kNN imitates the Bayes rule locally; distance and scaling define "near"; the curse of dimensionality $(1 - (1 - 10^{-d})^n)$ kills locality in high dimension.
- **Models.** A hypothesis space $\mathcal{H} = \{f_\theta\}$ shares structure across inputs; parameters are learned, hyperparameters chosen.
- **ERM.** Minimise empirical risk; it is honest for fixed models and optimistic after fitting; cross-entropy trains calibrated probabilities because its expectation is $H(Y \mid X) + \mathbb{E}[\mathrm{D}_{\text{KL}}]$.
- **Evaluation.** Train / validate / test, or cross-validate; touch the test set once.

Chapter 3

From Fitting to Generalization

How to read the formulas in this chapter

This chapter reuses the symbols of Chapter 2 (R, $\widehat{R}$, $\mathcal{H}$, λ, $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{val}}$, $\mathcal{D}_{\text{test}}$) and adds the few below. As before, every formula is followed by a *Reading the formula* box.

Symbol	Meaning in words
$\widehat{f}_{\mathcal{D}}$	the model a fixed learning procedure produces when fed the dataset $\mathcal{D}$
$G(\widehat{f}_{\mathcal{D}})$	the *generalization gap*: population risk minus training risk of the same fitted model
$\mathbb{E}_{\mathcal{D}}[\cdot]$	an average over *many possible training sets* drawn from the same world
$\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\widehat{f}_{\mathcal{D}}(x)]$	the average prediction at x across those training sets
$\text{Bias}(x)$, $\text{Var}(x)$	systematic error and sample-to-sample spread of the prediction at x
ε, $\sigma^2(x)$	the noise in Y around the best possible prediction, and its variance
$\Omega(\theta)$	a *penalty*: a number saying how "complicated" the parameters θ are
$\lVert\theta\rVert_2^2 = \sum_j \theta_j^2$, $\lVert\theta\rVert_1 = \sum_j \lvert\theta_j\rvert$	the two standard penalties (squared length, and sum of absolute values)
$h \in \mathcal{C}$	one full *configuration* of hyperparameters, from a candidate set $\mathcal{C}$
$\widehat{f}_h^{(-j)}$	the model with configuration h fitted on all folds *except* fold j
TP, FP, TN, FN	counts of true/false positives and negatives on a test set

3.1 Generalization and Inductive Bias

What are we trying to get?

Chapter 2 taught us to *fit*: choose the parameters that make the training examples look good. But nobody deploys a model to re-predict its own training set. We want a precise name for "does the fit carry over to new cases," a clear reason why training performance cannot answer that question, and an honest account of what *does* decide the answer: the assumptions built into the learner.

Two averages of the same loss

Same loss, two populations of examples: the ones you have, and the ones you will meet.

Given $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, empirical risk minimisation picks $\widehat{\theta} \in \arg\min_\theta \widehat{R}_{\text{train}}(\theta)$. This solves a *fitting* problem. The purpose of learning is to perform well on new cases drawn from the environment in which the model will be used.

Definition 3.1.1 (Empirical risk, population risk, generalization gap)**.** For a model f, the *empirical risk* on a dataset $\mathcal{D}$ and the *population risk* are

$$\widehat{R}_{\mathcal{D}}(f) = \frac{1}{N}\sum_{i=1}^{N} \ell(y^{(i)}, f(x^{(i)})), \qquad R(f) = \mathbb{E}_{(X,Y)\sim\mathbb{P}}[\ell(Y, f(X))].$$

For the model $\widehat{f}_{\mathcal{D}}$ fitted on $\mathcal{D}$, the *generalization gap* is

$$G(\widehat{f}_{\mathcal{D}}) = R(\widehat{f}_{\mathcal{D}}) - \widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}}).$$

Reading the formula

Both terms of the gap refer to the *same* fitted model; only the source of the examples differs. $\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})$ is the score on the very examples that chose the model. $R(\widehat{f}_{\mathcal{D}})$ is the score on fresh cases from the world. A gap near 0 means training performance transfers; a large positive gap means the training score was flattering. If a spam filter makes 2% errors on its training emails and 9% on the emails of the next month, the gap is 0.07.

Why training performance is special

Training data play two roles when training performance is reported: they first influence *which* model is selected, and are then used again to *evaluate* that model,

$$\mathcal{D} \longrightarrow \widehat{f}_{\mathcal{D}} \longrightarrow \widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}}).$$

The procedure can therefore exploit accidental regularities in $\mathcal{D}$. A second sample $\mathcal{D}'$ from the same population would generally produce another fitted model $\widehat{f}_{\mathcal{D}'}$; this sampling variability is part of the learning problem, not a defect of the notation.

Proposition 3.1.2 (Training risk of a fitted model is optimistic)**.** *Let the learning procedure return $\widehat{f}_{\mathcal{D}}$ minimising $\widehat{R}_{\mathcal{D}}$ over a class $\mathcal{H}$, with $\mathcal{D}$ an i.i.d. sample. Then*

$$\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})] \;\le\; \min_{f\in\mathcal{H}} R(f) \;\le\; \mathbb{E}_{\mathcal{D}}[R(\widehat{f}_{\mathcal{D}})],$$

so the expected generalization gap is nonnegative: $\mathbb{E}_{\mathcal{D}}[G(\widehat{f}_{\mathcal{D}})] \ge 0$.

Proof. This is Proposition 2.5.3 of Chapter 2, restated. Let $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. Since $\widehat{f}_{\mathcal{D}}$ minimises the empirical risk, $\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ for every sample; taking expectations and using that $f^*_{\mathcal{H}}$ is fixed (Proposition 2.5.2), the right side has expectation $R(f^*_{\mathcal{H}})$. The second inequality holds pointwise because $R(\widehat{f}_{\mathcal{D}}) \ge \min_{\mathcal{H}} R$. Subtracting gives the sign of the expected gap. □

Interpretation & discussion

Low training loss is *necessary* evidence that optimisation has fitted the data, but *incomplete* evidence that learning has succeeded. Independent evaluation is needed to determine whether the fitted pattern persists beyond the observations that selected it. That is the entire justification for validation and test sets.

Stable and spurious regularities

Example 3.1.3 (Cows on grass, camels on sand)**.** Suppose cows in the training collection are photographed on grass and camels on sand. *Both* animal shape and background predict every training label. Shape is closely related to the intended task; background is a shortcut created by the way the images were collected. Training performance cannot tell the two cues apart, because both work on every observed image. A cow photographed on sand separates them: the shape cue says "cow," the background cue says "camel." This is the wolves-and-snow story of Chapter 1 (Example 1.7.4) seen from the fitting side: a spurious cue produces zero training loss and a large generalization gap.

Dataset composition is therefore *part of the learning method*: the examples determine which correlations the model can exploit. Similar shortcuts occur when a network segment or capture time predicts attacks, a hospital or scanner type identifies a diagnosis, or a customer identifier predicts an outcome in historical records. These variables may be predictive in the sample without describing a relationship that remains useful at deployment.

Technique: Weakening shortcuts

- Vary nuisance factors at *collection* time: several cameras, sites, seasons, devices.
- Use *data augmentation* only with transformations that are realistic and preserve the label (flip a cat photo, yes; flip a digit "6", no: it becomes a "9").
- Evaluate under deployment-like conditions (Section 3.6); a random split hides shortcuts that are present in every part of the collection.

Inductive bias

Definition 3.1.4 (Inductive bias)**.** A finite sample is compatible with many functions, including functions that agree on the sample and differ everywhere else (Proposition 1.7.2). Learning therefore requires an *inductive bias*: assumptions that make some solutions more plausible than others beyond the observed examples. It enters through

- the *data and representation*, which determine which cues are available;
- the *hypothesis space*, which determines which relationships can be expressed;
- *regularisation*, which favours some fitting solutions over others;
- the *optimisation algorithm*, which may prefer particular solutions when many parameters minimise the training objective.

Here "bias" means a preference among solutions; it is broader than the statistical bias of Section 3.3.

Key insight

Generalization is never a property of the data alone. It succeeds when the assumptions encoded by the learning procedure remain true in the deployment environment. Every technique in this chapter is a way of choosing, strengthening or checking those assumptions.

Summary: Generalization and inductive bias

- Empirical risk is observed but sample-dependent; population risk is wanted but unknown; the gap compares them for one fitted model.
- Training risk after fitting is optimistic in expectation (Proposition 3.1.2), because the data choose and then grade the model.
- Spurious cues fit the sample as well as stable ones; only new contexts separate them.
- Inductive bias (data, representation, hypothesis space, regularisation, optimiser) decides behaviour off the sample.

3.2 Model Capacity

What are we trying to get?

We want a vocabulary for "how flexible is this family of models," a proof that flexibility always looks good on training data, and a concrete demonstration that it does not always look good on new data.

Definition 3.2.1 (Capacity, nested families)**.** The *capacity* of a hypothesis space $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$ describes how varied the functions in it can be. It is not determined by parameter count alone: structure, parameter constraints, regularisation, optimisation, and the amount and geometry of the data all matter. Typical capacity-controlling hyperparameters are polynomial degree, tree depth, the number of units in a network, and the neighbourhood size k in kNN. Families are *nested* when $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \cdots \subseteq \mathcal{H}_M$: every function of a smaller family remains available in a larger one.

Proposition 3.2.2 (Training error favours capacity)**.** *For nested families* $\mathcal{H}_m \subseteq \mathcal{H}_{m+1}$,

$$\min_{f \in \mathcal{H}_{m+1}} \widehat{R}_{\text{train}}(f) \;\leq\; \min_{f \in \mathcal{H}_m} \widehat{R}_{\text{train}}(f).$$

Increasing capacity cannot make the best achievable training error worse.

Proof. A larger family can always reuse the best solution of a smaller one: the minimiser over $\mathcal{H}_m$ is a candidate in $\mathcal{H}_{m+1}$, so the minimum over $\mathcal{H}_{m+1}$ is at most its value. (Proposition 2.5.4 of Chapter 2.) □

Reading the formula

This is why training loss *alone* systematically favours richer families: it can only go down as you add flexibility. It does not follow that the population risk goes down. Additional functions can fit stable structure, but they can also fit accidental variation.

Underfitting and overfitting

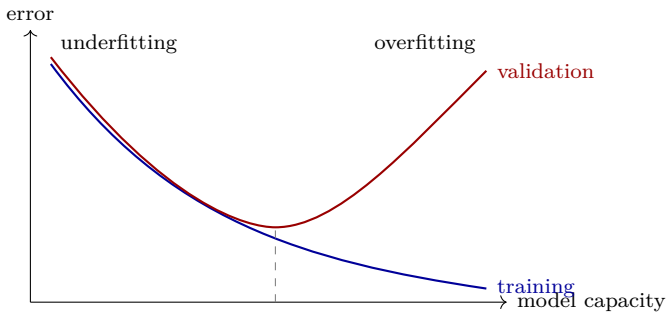
Definition 3.2.3 (Underfitting, overfitting)**.** An *underfitting* model is too restrictive to capture important structure: it usually has high training *and* high validation error. An *overfitting* model is too sensitive to the particular training sample: it may have very low training error and substantially higher validation error. The distinction is not whether the model is complicated in isolation, but whether its effective flexibility matches the available evidence.

Example 3.2.4 (Three polynomials on four points, judged on a fifth)**.** Chapter 2 (Example 2.5.6) fitted the points $(0,1),(1,3),(2,2),(3,4)$ with polynomials of degree 0, 1 and 3, with training risks 1.25, 0.45 and 0. Now a *validation* point arrives: $(4,5)$.

- Degree 0: the constant 2.5 predicts 2.5; squared error $(5-2.5)^2 = 6.25$.
- Degree 1: the line $0.8x + 1.3$ predicts 4.5; squared error 0.25.
- Degree 3: the interpolating cubic is $f(x) = 1 + 5.5x - 4.5x^2 + x^3$ (check: $f(1) = 3$, $f(2) = 2$, $f(3) = 4$); it predicts $f(4) = 1 + 22 - 72 + 64 = 15$; squared error $(5-15)^2 = 100$.

degree	0	1	3
training risk	1.25	0.45	0.00
validation error at $x = 4$	6.25	0.25	100

The constant underfits (bad everywhere). The cubic overfits: perfect on the four points it was shaped by, and wildly wrong one step outside them, because it bent itself around the noise in the sample. The line, which fits the training data worst of the last two, is the only one worth deploying.



For kNN the direction of the scale is reversed: small k gives small, highly adaptive neighbourhoods (low training error, high sensitivity), large k averages broadly (smoother rule, higher training error). Capacity must also be read relative to sample size: a family that is unstable with a small dataset may be well con-

strained by a much larger one, because more representative observations restrict the functions that can fit the sample while behaving arbitrarily elsewhere.

Gotcha

- "Zero training error" is not an achievement; with enough capacity it is guaranteed (Proposition 2.5.5). It is a warning to look at validation error.
- For kNN, *decreasing* k increases capacity. Students routinely get this backwards.

Summary: Model capacity

- Capacity: how varied the functions in $\mathcal{H}$ can be; controlled by degree, depth, width, k, and more.
- Nested families: training error never increases with capacity (Proposition 3.2.2); validation error typically falls then rises.
- Underfitting: high train and validation error. Overfitting: low train, high validation.

3.3 Bias and Variance

What are we trying to get?

"Sensitive to the sample" is a vague phrase. We want to make it a number, separate it from a second kind of error (being systematically wrong), and prove that squared prediction error splits exactly into noise plus these two quantities. That decomposition is the theoretical spine of the whole chapter.

A repeated-sampling thought experiment

Imagine rerunning the entire training procedure on many different samples from the same world, then looking at the cloud of predictions you would get at one input.

Imagine repeatedly drawing training sets $\mathcal{D}_1, \mathcal{D}_2, \dots$ from the same population and applying exactly the same learning procedure. The fitted functions $\widehat{f}_{\mathcal{D}_1}, \widehat{f}_{\mathcal{D}_2}, \dots$ need not coincide.

Definition 3.3.1 (Bias and variance of a learning procedure)**.** For squared-error regression let $f^*(x) = \mathbb{E}[Y \mid X = x]$ be the optimal regression function and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\widehat{f}_{\mathcal{D}}(x)]$ the average prediction of the procedure at x. The *pointwise bias* and *pointwise variance* are

$$\mathrm{Bias}(x) = \bar{f}(x) - f^*(x), \qquad \mathrm{Var}(x) = \mathbb{E}_{\mathcal{D}}\Big[\big(\widehat{f}_{\mathcal{D}}(x) - \bar{f}(x)\big)^2\Big].$$

Reading the formula

Fix one input x, say a particular house. Train a thousand times on a thousand different training sets and record the thousand predicted prices. $\bar{f}(x)$ is their average. *Bias* asks: is that average off from the true conditional mean price, in a consistent direction? *Variance* asks: how widely do the thousand predictions scatter around their own average? A model that always predicts "1 million" has zero variance and possibly large bias; a very flexible model may have tiny

bias and huge variance.

Restrictive models and inadequate features create bias; merely collecting more examples may not remove it if the family cannot express the relationship. High capacity, small datasets, irrelevant variables and unstable feature selection increase variance; more representative data reduce it.

Theorem 3.3.2 (Bias–variance decomposition)**.** *Assume* $Y = f^*(X) + \varepsilon$ *with* $\mathbb{E}[\varepsilon \mid X] = 0$ *and* $\mathrm{Var}(\varepsilon \mid X = x) = \sigma^2(x)$*, and assume the training sample* $\mathcal{D}$ *is independent of the test pair* (x, Y)*. Then at every fixed input* x*,*

$$\mathbb{E}_{\mathcal{D},\,Y|x}\Big[\big(Y - \widehat{f}_{\mathcal{D}}(x)\big)^2\Big] = \sigma^2(x) + \mathrm{Bias}(x)^2 + \mathrm{Var}(x).$$

Proof. **Step 1 (split off the noise).** Write $Y - \widehat{f}_{\mathcal{D}}(x) = \varepsilon + \big(f^*(x) - \widehat{f}_{\mathcal{D}}(x)\big)$, where $\varepsilon = Y - f^*(x)$. Squaring,

$$\big(Y - \widehat{f}_{\mathcal{D}}(x)\big)^2 = \varepsilon^2 + 2\varepsilon\big(f^*(x) - \widehat{f}_{\mathcal{D}}(x)\big) + \big(f^*(x) - \widehat{f}_{\mathcal{D}}(x)\big)^2.$$

Take expectations. $\mathbb{E}[\varepsilon^2 \mid x] = \sigma^2(x)$. In the cross term, ε is independent of $\mathcal{D}$ (the test noise has nothing to do with the training sample) and has conditional mean 0, so $\mathbb{E}\big[\varepsilon\,(f^*(x) - \widehat{f}_{\mathcal{D}}(x))\big] = \mathbb{E}[\varepsilon \mid x] \cdot \mathbb{E}_{\mathcal{D}}[f^*(x) - \widehat{f}_{\mathcal{D}}(x)] = 0$.

Step 2 (split the remaining error around the average prediction). Write $f^*(x) - \widehat{f}_{\mathcal{D}}(x) = \big(f^*(x) - \bar{f}(x)\big) + \big(\bar{f}(x) - \widehat{f}_{\mathcal{D}}(x)\big)$. Squaring and taking $\mathbb{E}_{\mathcal{D}}$: the first bracket is a constant, $-\mathrm{Bias}(x)$, whose square is $\mathrm{Bias}(x)^2$; the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by definition of $\bar{f}$, so the cross term $2\big(f^*(x) - \bar{f}(x)\big)\,\mathbb{E}_{\mathcal{D}}[\bar{f}(x) - \widehat{f}_{\mathcal{D}}(x)]$ vanishes; and the square of the second bracket averages to $\mathrm{Var}(x)$.

Adding the pieces: $\sigma^2(x) + \mathrm{Bias}(x)^2 + \mathrm{Var}(x)$. □

Key insight

Twice the same move as in Theorem 2.2.4: add and subtract a mean, square, and watch the cross term die because a mean-zero quantity multiplies a constant. First around the true regression function (killing the noise cross term), then around the average prediction (killing the sampling cross term).

Reading the formula

Three sources of error at one input.

- $\sigma^2(x)$: *irreducible noise*. Even the perfect predictor f^* makes this error, because Y has randomness that x does not explain.
- $\mathrm{Bias}(x)^2$: *systematic error* of the procedure, the part that repeating the experiment never averages away.
- $\mathrm{Var}(x)$: *sample sensitivity*, the part that a different training set would change.

Increasing capacity often reduces bias while increasing variance, producing the U-shaped validation curve. This is a useful diagnostic principle, not a universal law.

Example 3.3.3 (Bias and variance computed exactly)**.** The simplest regression problem: no input, $Y = \mu + \varepsilon$ with $\mathrm{Var}(\varepsilon) = \sigma^2$, so $f^* = \mu$. The training set is N draws $y^{(1)}, \ldots, y^{(N)}$. Three procedures:

- *Sample mean*, $\widehat{f} = \frac{1}{N}\sum_i y^{(i)}$. $\mathbb{E}_{\mathcal{D}}[\widehat{f}] = \mu$: bias 0. Variance of a mean of N independent draws: σ^2/N. Total error $\sigma^2 + 0 + \sigma^2/N$.
- *Ignore the data*, $\widehat{f} = 0$. Bias $-\mu$, variance 0. Total $\sigma^2 + \mu^2$.
- *Shrunk mean*, $\widehat{f} = \frac{1}{2} \cdot \frac{1}{N}\sum_i y^{(i)}$. $\mathbb{E}_{\mathcal{D}}[\widehat{f}] = \mu/2$: bias $-\mu/2$. Variance $\frac{1}{4}\sigma^2/N$. Total $\sigma^2 + \mu^2/4 + \sigma^2/(4N)$.

With $\sigma^2 = 1$ and $N = 4$:

bias^2 + variance	sample mean	ignore data	shrunk mean
$\mu = 1$	$0 + 0.25 = 0.25$	$1 + 0 = 1$	$0.25 + 0.0625 = 0.3125$
$\mu = 0.2$	$0 + 0.25 = 0.25$	$0.04 + 0 = 0.04$	$0.01 + 0.0625 = 0.0725$

When the truth is far from zero ($\mu = 1$) the unbiased procedure wins. When the truth is near zero ($\mu = 0.2$) both biased procedures beat it: they trade a little systematic error for a large reduction in variance. Regularisation (Section 3.4) is exactly this trade, made tunable.

Beyond the classical U-shape

Remark 3.3.4 (Double descent). Modern overparameterised models can display *double descent*. As capacity approaches the smallest level at which the training sample can be interpolated, validation error may increase sharply; beyond this *interpolation threshold*, further capacity can reduce validation error again. The second descent depends on regularisation, the optimiser, the data geometry and which interpolating solution is selected; it does not occur in every problem. An analogous phenomenon can occur over *training time* with a fixed number of parameters. The classical U-shape remains a valuable first model, but validation behaviour in modern systems must be examined rather than assumed.

Definition 3.3.5 (Three sources of excess error)**.** Relative to the Bayes risk, a learned model's extra risk decomposes conceptually into

- *approximation error*: the hypothesis space cannot express a good enough function;
- *estimation error*: finite data do not reliably identify the best function in the space;
- *optimisation error*: the algorithm fails to reach a good solution of its own objective.

Increasing capacity can reduce approximation error while increasing estimation error. Optimisation is studied later in the course.

Classic problems

- *A procedure has* $\mathrm{Bias}(x) = 0.3$, $\mathrm{Var}(x) = 0.05$, *noise* $\sigma^2 = 0.1$. *Expected squared error at* x*?* $0.1 + 0.09 + 0.05 = 0.24$.
- *Doubling the data leaves bias unchanged and halves the variance in Example 3.3.3. True in general?* The variance part often shrinks with N; the bias does not, because it comes from the family, not the sample size.
- *Can a procedure have zero bias and zero variance?* Only if it returns f^* regardless of the sample, i.e. if it already knows the answer.

- *1-NN regression: bias and variance?* Bias small (it follows the data closely), variance large (each prediction depends on one noisy neighbour).

Summary: Bias and variance

- Bias: average prediction minus truth, across training sets. Variance: spread of predictions across training sets.
- Squared error = noise + bias^2 + variance (Theorem 3.3.2), by adding and subtracting means twice.
- Capacity usually trades bias for variance; a little bias can buy a lot of variance (Example 3.3.3); double descent shows the trade is not the whole story.

3.4 Controlling Complexity

What are we trying to get?

If flexibility is the problem, we need dials that reduce it *without* throwing away parameters: penalties on parameter size, and stopping training early. We want to see exactly what a penalty does to the hypothesis space, why the L_1 penalty produces exact zeros, and how the strength of the penalty is chosen.

Complexity can be controlled by restricting the family (degree, depth), increasing k in kNN, selecting features, penalising parameters, stopping training early, or collecting more relevant data. Each choice expresses an assumption about which fitted solutions are likely to generalise.

Regularised empirical risk minimisation

Definition 3.4.1 (Regularised ERM)**.** Given a *penalty* $\Omega : \Theta \to [0, \infty)$ and a strength $\lambda \geq 0$,

$$\widehat{\theta}_\lambda \in \arg\min_{\theta \in \Theta} \left[\widehat{R}_{\text{train}}(\theta) + \lambda\, \Omega(\theta) \right].$$

Two standard penalties are

$$\Omega_2(\theta) = \|\theta\|_2^2 = \sum_j \theta_j^2 \quad (L_2,\ \text{ridge}), \qquad \Omega_1(\theta) = \|\theta\|_1 = \sum_j |\theta_j| \quad (L_1,\ \text{lasso}).$$

Reading the formula

The objective is now "fit the data *and* keep the parameters small." λ says how much a unit of penalty is worth in units of training loss: $\lambda = 0$ is plain ERM; large λ makes any sizeable coefficient too expensive, so the fit is pulled toward the origin. With two coefficients $(3, -4)$, $\Omega_2 = 9+16 = 25$ and $\Omega_1 = 3+4 = 7$.

Theorem 3.4.2 (A penalty is a soft constraint)**.** *Let* $\widehat{\theta}_\lambda$ *solve the penalised*

problem and put $c = \Omega(\widehat{\theta}_\lambda)$. *Then* $\widehat{\theta}_\lambda$ *also solves the* constrained *problem*

$$\min_\theta \ \widehat{R}_{\text{train}}(\theta) \quad \textit{subject to} \quad \Omega(\theta) \le c.$$

Consequently regularised ERM fits within the family $\mathcal{H}_c = \{f_\theta : \Omega(\theta) \le c\}$, *and these families are nested:* $c_1 < c_2 \Rightarrow \mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$.

Proof. Take any θ with $\Omega(\theta) \le c$. Optimality of $\widehat{\theta}_\lambda$ for the penalised problem gives $\widehat{R}(\widehat{\theta}_\lambda) + \lambda c \le \widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c$, hence $\widehat{R}(\widehat{\theta}_\lambda) \le \widehat{R}(\theta)$: no feasible θ beats $\widehat{\theta}_\lambda$, and $\widehat{\theta}_\lambda$ is itself feasible. Nesting is immediate from $\Omega(\theta) \le c_1 \Rightarrow \Omega(\theta) \le c_2$. □

Key insight

A stronger penalty confines the solution to a smaller *effective* family, even though the number of parameters has not changed. Capacity is about which functions can actually win the fitting contest, not about how many numbers the model stores. The precise map from λ to c depends on the problem.

Proposition 3.4.3 (Ridge in one variable: shrinkage). *For the model* $f_w(x) = wx$ *(no intercept) under squared loss with* L_2 *penalty, the regularised solution is*

$$\hat{w}_\lambda = \frac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2 + N\lambda},$$

which is the unpenalised least-squares slope multiplied by the factor $\dfrac{\sum_i (x^{(i)})^2}{\sum_i (x^{(i)})^2 + N\lambda} \in (0, 1]$: *the coefficient is* shrunk *toward* 0, *smoothly, never exactly to* 0 *for finite* λ.

Proof. The objective $J(w) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)})^2 + \lambda w^2$ is a convex quadratic in w. Setting $J'(w) = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)}) + 2\lambda w = 0$ gives $w\big(\sum_i (x^{(i)})^2 + N\lambda\big) = \sum_i x^{(i)} y^{(i)}$. The factor form follows by dividing numerator and denominator by $\sum_i (x^{(i)})^2$. □

Reading the formula

With $\sum x^2 = 14$, $\sum xy = 19$ and $N = 4$ (the numbers of Example 2.5.6, ignoring the intercept): $\lambda = 0$ gives $19/14 \approx 1.357$; $\lambda = 1$ gives $19/18 \approx 1.056$; $\lambda = 10$ gives $19/54 \approx 0.352$. The bigger the penalty, the closer to zero, but never zero.

Proposition 3.4.4 (Lasso in one variable: soft thresholding). *For* $z \in \mathbb{R}$ *and* $\lambda > 0$, *the minimiser of* $\frac{1}{2}(w - z)^2 + \lambda|w|$ *over* $w \in \mathbb{R}$ *is*

$$\hat{w} = \operatorname{sign}(z)\ \max\big(|z| - \lambda,\ 0\big).$$

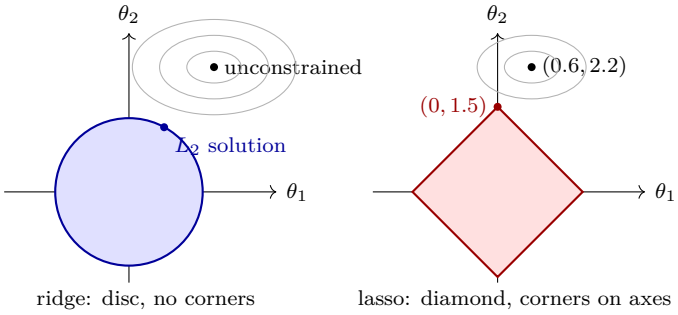
In particular $\hat{w} = 0$ *exactly whenever* $|z| \le \lambda$.

Proof. Suppose first $z > 0$. For $w \ge 0$ the objective is $\frac{1}{2}(w - z)^2 + \lambda w$, a convex parabola with unconstrained minimiser $w = z - \lambda$; if $z > \lambda$ this is positive and gives the minimum on $[0, \infty)$; if $z \le \lambda$ the parabola is increasing on $[0, \infty)$ and the minimum on that side is at $w = 0$. For $w < 0$ the objective is $\frac{1}{2}(w - z)^2 - \lambda w$,

which is decreasing in w toward 0 when $z > 0$ (its unconstrained minimiser $z + \lambda$ is positive), so no negative w beats $w = 0$. Combining: $\hat{w} = \max(z - \lambda, 0)$ for $z > 0$. The case $z < 0$ is symmetric, and $z = 0$ gives $\hat{w} = 0$. □

Reading the formula

z is where the unpenalised fit would put the coefficient. The L_1 penalty moves it toward zero by exactly λ, and if it would cross zero, it stops *at* zero. With $\lambda = 0.5$: $z = 2.2 \to 1.7$; $z = 0.4 \to 0$; $z = -3 \to -2.5$. Geometrically, the constraint region $\|\theta\|_1 \le c$ is a diamond whose corners lie on the coordinate axes, and an optimum often lands on a corner where some coordinates are exactly zero. The slide's two-dimensional example has unconstrained optimum $(0.6, 2.2)$ and lasso solution $(0, 1.5)$: the first feature is dropped from the model altogether. L_1 therefore performs *embedded feature selection*, although correlated features can make the selected subset unstable. L_2's region is a disc with no corners, so it shrinks everything and keeps everything.



Choosing the strength, and early stopping

The regularisation strength is a hyperparameter: for each candidate λ, training estimates $\widehat{\theta}_\lambda$; validation selects $\hat{\lambda} \in \arg\min_{\lambda \in \mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_\lambda)$; the test data stay outside this selection (Section 2.6).

Definition 3.4.5 (Early stopping)**.** Iterative optimisation produces a sequence $\widehat{\theta}_0, \widehat{\theta}_1, \ldots$. Early iterations often learn strong recurring patterns; later ones may fit exceptional observations or noise. *Early stopping* retains the checkpoint with smallest validation risk,

$$t^* = \arg\min_t \widehat{R}_{\text{val}}(\widehat{\theta}_t), \qquad \widehat{\theta}_{\text{ES}} = \widehat{\theta}_{t^*}.$$

Training time thereby controls effective capacity while the parameter count stays fixed. A *patience* p lets training continue p epochs without improvement before stopping, keeping the best checkpoint seen.

Reading the formula

Every epoch, compute the validation loss; remember the epoch where it was lowest; when you finally stop, roll back to that epoch's parameters. Patience protects against stopping on a temporary wobble of a noisy validation curve. It can still miss a later decrease under epoch-wise double descent; a longer horizon with checkpoint retention is safer when that is plausible, at the price of computation, and with no guarantee that a second descent will come.

Technique: Regularisation in practice

- Standardise features before penalising (Section 2.6); otherwise a feature measured in millimetres is penalised far more than the same feature in metres.
- Do not penalise the intercept.
- Search λ on a logarithmic grid $(10^{-3}, 10^{-2}, \ldots, 10^{2})$; the useful range spans orders of magnitude.
- Use L_1 when you believe few features matter and want to know which; L_2 when many features contribute a little.
- Early stopping is regularisation for free in any iterative method; always keep the best checkpoint, not the last.

Classic problems

- *Soft-threshold $z = 1.2$ with $\lambda = 1.5$.* $|z| \leq \lambda$: $\hat{w} = 0$.
- *Ridge with $\lambda \to \infty$?* All penalised coefficients $\to 0$; the model becomes the intercept alone (degree-0 fit).
- *Two identical features under L_2 versus L_1.* L_2 splits the weight equally between them; L_1 is indifferent among all splits and may pick either, which is the instability mentioned above.
- *Why is λ chosen on validation and not on training data?* Training risk is minimised at $\lambda = 0$ (Proposition 3.2.2 applied to the nested families $\mathcal{H}_c$).

Summary: Controlling complexity

- Regularised ERM: minimise training risk plus $\lambda\Omega(\theta)$; equivalent to fitting within a nested constrained family (Theorem 3.4.2).
- L_2 shrinks smoothly and keeps all features (Proposition 3.4.3); L_1 soft-thresholds and sets coefficients exactly to zero (Proposition 3.4.4).
- λ and the stopping epoch are hyperparameters chosen on validation data.

3.5 Feature and Model Selection

What are we trying to get?

Choosing *which inputs* to use is a model choice like any other, and it can go wrong in the same ways. We want to see why exhaustive search is impossible, what the practical searches do, why they must live inside the training pipeline, and why the validation set itself can be worn out by overuse.

Feature selection changes the representation supplied to the learner. It can remove irrelevant variables that increase estimation difficulty, reduce redundancy, improve interpretability, and exclude variables unavailable at decision time. It is a data-dependent model choice and must be evaluated accordingly.

Proposition 3.5.1 (Subset selection is combinatorial)**.** *With d features there are 2^d subsets. The idealised search $S^* \in \arg\min_{S\subseteq\{1,\dots,d\}} \widehat{R}_{\text{val}}(\widehat{f}_S)$, where $\widehat{f}_S$ uses only the features in S, requires 2^d fits. For $d = 30$, $2^{30} = 1{,}073{,}741{,}824$.*

Proof. Each feature is independently in or out of the subset: $2 \times \cdots \times 2 = 2^d$ choices. □

Definition 3.5.2 (Filters and greedy searches)**.**

- A *correlation filter* scores each variable separately, e.g. $s_j = |\widehat{\text{Corr}}(X_j, Y)|$, and keeps the top-scoring ones. Fast and model-independent, but it misses nonlinear relationships and interactions, and it ignores redundancy: two highly ranked features may carry the same information. Weak marginal correlation does not prove that a feature is useless jointly with others.
- *Forward selection* starts from $S = \emptyset$ and repeatedly adds the feature giving the largest validation improvement. *Backward elimination* starts from all features and repeatedly removes the least useful. Both are greedy: an early choice can block a better combination later; backward elimination also requires fitting the full model.

Example 3.5.3 (Two features that are useless alone and perfect together)**.** Let X_1, X_2 be independent fair coin flips (± 1) and $Y = X_1 X_2$. Each feature has correlation 0 with Y (knowing X_1 alone says nothing about the sign of the product), so a correlation filter discards both. Together they determine Y exactly. Filters see one variable at a time; interactions are invisible to them.

Gotcha

All data-dependent feature scoring and subset search belong *inside* the training pipeline. In cross-validation they must be repeated separately within every training fold. Otherwise information from a validation fold shapes the representation that is then evaluated on that fold: *leakage*, and an optimistic estimate.

The roles of the three subsets, and their overuse

Let h denote an entire hyperparameter configuration (features, model family, λ, k, thresholds), not only one number. The development loop fits $\widehat{f}_h$ on training data, measures validation performance, and updates the configuration: $\hat{h} \in \arg\min_{h\in\mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_h)$. The test set remains outside this loop.

Proposition 3.5.4 (The best of many validation scores is optimistic)**.** *Let $\widehat{R}_{\text{val}}(\widehat{f}_h)$, $h \in \mathcal{C}$, be validation risks of models fitted without the validation data, so that each is an unbiased estimate of $R(\widehat{f}_h)$ (Proposition 2.6.2). Then*

$$\mathbb{E}\Big[\min_{h\in\mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_h)\Big] \;\le\; \min_{h\in\mathcal{C}} \mathbb{E}\big[\widehat{R}_{\text{val}}(\widehat{f}_h)\big] = \min_{h\in\mathcal{C}} R(\widehat{f}_h),$$

with strict inequality as soon as the estimates are noisy and no single h is best in every sample.

Proof. For every $h_0 \in \mathcal{C}$, $\min_h \widehat{R}_{\text{val}}(\widehat{f}_h) \le \widehat{R}_{\text{val}}(\widehat{f}_{h_0})$ pointwise; take expectations, then take the minimum over h_0 of the right side. □

Reading the formula

Try a hundred configurations and report the lowest validation score: that number is the minimum of a hundred noisy estimates, and minima of noisy things are pulled downward. Every reported validation result also influences the next manual or automated design choice, so the development process gradually adapts to the validation set. The final test set protects against this accumulated adaptation; it does *not* repair it if its results are then used to continue development, because at that point it has become another validation set.

Definition 3.5.5 (K-fold cross-validation, restated for configurations)**.** Partition the development data into K folds. For configuration h and fold j, fit $\widehat{f}_h^{(-j)}$ without fold j and evaluate it on fold j; then

$$\widehat{R}_{\mathrm{CV}}(h) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}_{\mathrm{val}}^{(j)}(\widehat{f}_h^{(-j)}), \qquad \hat{h} \in \arg\min_{h\in\mathcal{C}} \widehat{R}_{\mathrm{CV}}(h).$$

Each observation validates once and trains $K-1$ times.

Cross-validation uses limited development data efficiently, at roughly K fits per configuration; its estimate still varies with the available sample; standard random folds assume exchangeable observations, so grouped or temporal dependence needs adapted folds (Section 3.6); and it does not replace final evaluation on an untouched test set.

Summary: Feature and model selection

- 2^d subsets make exhaustive selection impossible; filters and greedy searches are the practical tools, each blind to something.
- Selection is part of training: redo it inside every fold, or leak.
- The minimum of many validation scores is optimistic (Proposition 3.5.4); hence a test set used once.

3.6 Reliable Evaluation and Data Splitting

What are we trying to get?

An evaluation is only as honest as its split. We want a rule for choosing the split from the deployment question, metrics that do not hide minority-class failure, and a checklist of the ways information leaks from the future into the model.

The split should match deployment

Ask first: what kind of new *case will the model face? Then hold out exactly that kind of case.*

If the model must predict future events, the evaluation should preserve time order. If it must generalise to new users, devices, patients or organisations, entire groups should be held out. A random split is credible only when new deployment cases are reasonably approximated by independent draws from the same population.

Definition 3.6.1 (Random, stratified, grouped and temporal splits)**.**

- *Random*: each observation is assigned to a subset approximately independently.
- *Stratified*: random, but preserving approximate class proportions in every subset, which stabilises rare-class counts. Stratification must not override dependence: records from one device should not be scattered across subsets merely to match proportions.
- *Grouped*: every group (user, device, patient, network) lies entirely in one subset, so the learner cannot use identity-specific patterns and is evaluated on unseen groups.
- *Temporal*: train on the past, validate and test on later observations; this preserves the direction of prediction and reveals deterioration as behaviour, sensors or attack patterns change.

Example 3.6.2 (The same intrusion dataset, three verdicts)**.** Flows are recorded from 50 devices over 6 months. A *random* split puts flows from every device and every week into train and test; the model can learn "device 17 is the compromised laptop" and score 99.5%. A *grouped* split holds out 10 devices; the model must recognise attack *behaviour* and scores 96%. A *temporal* split trains on months 1 to 4 and tests on months 5 and 6, during which a new attack tool appeared; the model scores 90%. Only the third number describes what will happen next month. The first is not wrong arithmetic; it answers a question nobody asked.

Class imbalance and metrics

Proposition 3.6.3 (Accuracy hides minority failure)**.** *If a fraction π of the data is positive, the classifier that always predicts "negative" has accuracy $1-\pi$ and recall 0 on the positive class.*

Proof. It is correct on every negative and wrong on every positive: accuracy $= (1-\pi)$; it detects no positive, so $\mathrm{TP} = 0$ and recall $= 0/(0+\mathrm{FN}) = 0$. □

With $\pi = 0.01$ this is 99% accuracy and no attack ever detected.

Definition 3.6.4 (Metrics from the confusion matrix)**.** With attacks as the positive class, count true positives TP, false negatives FN, false positives FP and true negatives TN on the evaluation set. Then

$$\mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}, \quad \mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \quad \mathrm{Specificity} = \frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}},$$

$$\mathrm{BalancedAcc} = \frac{\mathrm{Recall}+\mathrm{Specificity}}{2}, \qquad F_1 = 2\,\frac{\mathrm{Precision}\cdot\mathrm{Recall}}{\mathrm{Precision}+\mathrm{Recall}}.$$

Reading the formula

Each metric answers one question. *Recall*: of the real attacks, what fraction did we catch? *Precision*: of the alarms we raised, what fraction were real? *Specificity*: of the benign flows, what fraction did we leave alone? *Balanced*

accuracy: average of the two per-class rates, immune to imbalance. F_1: the harmonic mean of precision and recall, high only when both are high; it ignores true negatives and treats the two symmetrically.
A detector on 10,000 flows with 50 attacks: TP = 40, FN = 10, FP = 90, TN = 9860.

metric	computation	value
accuracy	9900/10000	0.990
recall	40/50	0.80
precision	40/130	0.31
specificity	9860/9950	0.991
balanced accuracy	$(0.80 + 0.991)/2$	0.895
F_1	$2(0.31)(0.80)/(0.31 + 0.80)$	0.44

Accuracy says 99%; precision says two alarms in three are false. Which number matters depends on whether an analyst has to investigate each alarm.

Proposition 3.6.5 (F_1 lies between the smaller and the average of precision and recall)**.** *For* $P, R \in (0, 1]$*:* $\min(P, R) \leq F_1 \leq \frac{P+R}{2}$*, with* $F_1 = \frac{P+R}{2}$ *iff* $P = R$*.*

Proof. $F_1 = 2PR/(P + R)$. Lower bound: if $P \leq R$ then $F_1 = P \cdot \frac{2R}{P+R} \geq P$ because $2R \geq P + R$. Upper bound: $\frac{P+R}{2} - \frac{2PR}{P+R} = \frac{(P+R)^2-4PR}{2(P+R)} = \frac{(P-R)^2}{2(P+R)} \geq 0$, zero iff $P = R$. □

Interpretation & discussion

F_1 is dragged toward the *worse* of the two rates: precision 0.31 and recall 0.80 give $F_1 = 0.44$, far below their average 0.56. That is the point of a harmonic mean: it will not let a great recall hide a poor precision. No single metric is universally correct; metric and threshold choice should reflect the costs of missed attacks, false alarms and alert volume, exactly as in Theorem 2.2.9.

Data leakage

Definition 3.6.6 (Leakage)**.** *Leakage* occurs when development uses information that would not be available when a new deployment decision is made. Common forms:

- duplicate or near-duplicate events in several subsets;
- records from the same device, session or patient in training and test data;
- predictors created after the outcome or after a human investigation (a "ticket closed as intrusion" field);
- preprocessing fitted using validation or test observations;
- hyperparameters repeatedly chosen using test results.

Leakage creates an unrealistically favourable estimate of generalization.

Technique: Preprocessing belongs inside the training pipeline

For standardisation, the training set estimates

$$\hat{\mu}_j = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} x_j^{(i)}, \qquad \hat{\sigma}_j^2 = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} \left(x_j^{(i)} - \hat{\mu}_j\right)^2,$$

and the *same* $\hat{\mu}_j, \hat{\sigma}_j$ are reused unchanged on validation, test and deployment data. The same rule applies to imputation, feature selection, dimensionality reduction and resampling. In cross-validation each fold fits these operations on its own training portion only.

Leaking pipeline	Valid pipeline
all data → estimate μ_j, σ_j → split	split → estimate μ_j, σ_j on training → reuse them on validation, test, deployment

Classic problems

- TP = 30, FN = 70, FP = 0. *Precision, recall, F_1?* 1.0, 0.3, $2(0.3)/(1.3) \approx 0.46$.
- *Which split for "will this customer churn next quarter"?* Temporal: features from before the quarter, outcome after.
- *Which split for "does this model work on a hospital it has never seen"?* Grouped by hospital.
- *A feature "number of analyst comments" predicts intrusions perfectly. Why is it useless?* Comments are written after the investigation: leakage from the future.

Summary: Reliable evaluation

- Choose the split from the deployment question: temporal for future events, grouped for new entities, random only for i.i.d. cases; stratify without breaking groups or time.
- Accuracy hides minority failure (Proposition 3.6.3); use recall, precision, specificity, balanced accuracy, F_1 as the costs dictate.
- Leakage in any form inflates the estimate; fit every preprocessing step on training data only.

3.7 Diagnosis and Lab Preparation

Definition 3.7.1 (Learning curves)**.** *Learning curves* vary the amount of training data and plot training and validation performance against it.

Observed pattern	Likely issue	Possible response
Training and validation errors both high	underfitting	improve the representation or increase capacity
Training error low, validation error high	high variance	regularise, reduce capacity, or collect representative data
Errors close and both decrease as N grows	data-limited	additional representative examples are likely to help
Test performance degrades over time	distribution shift	update the data and use a time-aware evaluation protocol

"Errors close and decreasing" means the two curves have a small gap and both improve with more observations: the model is not overfitting strongly; the remaining limitation is the amount of evidence. Curves are diagnostic evidence, not proofs, and are read together with domain knowledge and error analysis.

Example 3.7.2 (Lab 1 at a glance)**.** The intrusion-detection lab audits a labelled-flow dataset (imbalance, missing or non-finite values, duplicates, conflicting labels, redundancy), pre-declares random-row, grouped and chronological train/validation/test protocols, fits baselines (majority class, depth-3 tree, logistic regression, kNN) under each protocol, probes metadata shortcuts and shuffled labels, selects the decision threshold on validation, and reports test recall, false-positive rate and false alarms per 100,000 benign flows. Every idea of this chapter appears there: the protocols are Section 3.6, the shortcut probes are Section 4.3, the threshold-on-validation rule is Section 3.5, and the metrics are Definition 3.6.4.

Notes on the original slides and notes

All formulas were rederived and all counts checked ($2^{30} = 1{,}073{,}741{,}824$; the metric definitions; the bias–variance identity). Points to note:

- **Bias–variance decomposition (notes eq. 8, slide 24).** The notes write $\mathrm{Var}(\varepsilon \mid X = x) = \sigma^2(x)$ while the slide writes a constant σ^2; both are fine, the proof never uses constancy. The identity also needs the training sample to be independent of the test noise, which the notes leave implicit; Theorem 3.3.2 states it.
- **"Moderate complexity: low bias, low variance" (slide 21).** This describes the sweet spot of the U-curve, not a general possibility; both terms are typically positive there, just small.
- **Typographic slips in the notes.** Section 4.2 has "widehatθ_1" (a rendering error for $\widehat{\theta}_1$); Section 5 has "For ($\subseteq \{1, \ldots, d\}$)" where the subset name S is missing; Section 1.2's parenthesised symbols "(D), (P)" are formatting artefacts, not new notation.
- **Soft-constraint view (notes 4.1, slide 29).** The notes say the correspondence between λ and c "depends on the problem" without stating it; Theorem 3.4.2 gives the exact statement: $c = \Omega(\widehat{\theta}_\lambda)$.
- **Lasso example (slide 31).** The numbers $(0.6, 2.2) \to (0, 1.5)$ are illustrative; with correlated features the lasso solution is not obtained by coordinatewise soft thresholding, which is why Proposition 3.4.4 is stated in one variable.

- **Slide 45.** "Always predicts normal" has recall 0 on attacks; its specificity is 1 and its balanced accuracy 0.5, which is the number that exposes it.
- **Slide 27.** "Optimization error will be developed in Lecture 6": not covered here.

Chapter takeaways

Summary: From fitting to generalization

- **Low training loss does not establish performance on new data:** the sample chooses and then grades the model.
- **Inductive bias** enters through data, representation, model family, regularisation and optimisation; it decides what happens off the sample.
- **Capacity** trades approximation ability against sensitivity to a finite sample; training error only ever rewards more of it.
- **Bias** is systematic error across repeated samples, **variance** is variation across them; squared error = noise + bias^2 + variance.
- **Regularisation**, feature selection and early stopping control effective capacity; validation selects their strength; L_1 zeroes coefficients, L_2 shrinks them.
- **Evaluation splits** must reproduce the novelty expected at deployment; metrics must expose minority-class failure; **leakage** invalidates the estimate.

Chapter 4

Discriminative and Generative Classification

How to read the formulas in this chapter

Chapter 2 taught you to read $\mathbb{P}(Y \mid X = x)$, losses and risks. This chapter adds a few symbols, all listed here; as before, every formula is followed by a *Reading the formula* box.

Symbol	Meaning in words
$p(y = c \mid x)$	the *posterior*: probability of class c once x is seen (the true one; p_θ or $\widehat{p}$ for a model's estimate)
$s_{\theta,c}(x)$	a real-valued *score* (logit) that a model gives to class c
$\sigma(s)$	the *sigmoid*, which turns one score into a probability in $(0, 1)$
$\mathrm{softmax}(s)_c$	turns C scores into C probabilities that sum to 1
$\pi_c = p(y = c)$	the *prior*: how frequent class c is before seeing any input
$p_\theta(x \mid y = c)$	the *class-conditional* distribution: which inputs class c produces (a density for continuous x, a mass function for discrete x)
$\phi = (\pi, \theta)$	all parameters of a generative model: priors plus class-conditional parameters
$\mathcal{N}(x; \mu, \Sigma)$	the Gaussian density with mean μ and covariance Σ
N_c	number of training examples of class c; $N = \sum_c N_c$
n_{cj}	count of token (word) j in the training documents of class c
α	a smoothing constant (pseudo-count) added to every count
π_t, π_d	class priors at training time and at deployment time
$\mathbf{1}(\cdot)$, log	indicator; natural logarithm unless stated

4.1 Two Routes to the Posterior

What are we trying to get?

Chapter 2 showed that once the posterior $p(y = c \mid x)$ is known, the best decision is a one-line rule. So the real work of classification is *estimating the posterior from labelled examples*. We want to understand that there are two fundamentally different ways to do that, what each one commits us to, and why neither wins in general.

The decision is known; the posterior is not

The Bayes rule is easy. The hard part is getting the probabilities it needs.

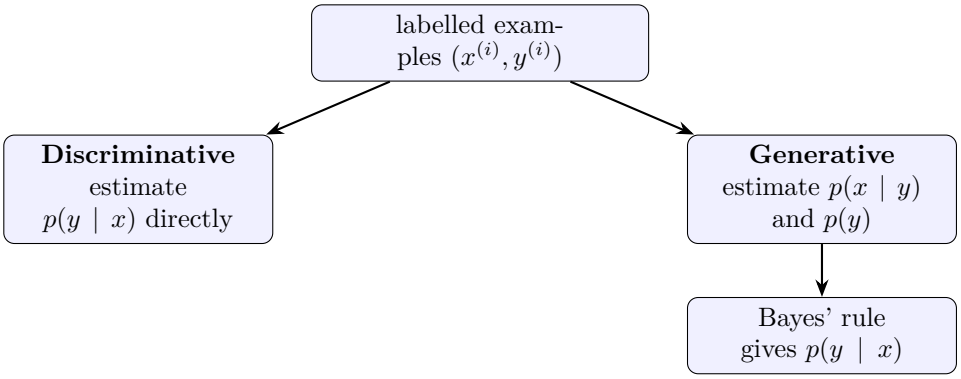
Let $Y \in \{1, \dots, C\}$ be a class label and $X \in \mathbb{R}^d$ a feature vector. If the posterior were known, Theorem 2.2.3 says to choose the action with the smallest conditional expected loss,

$$f^*(x) \in \arg\min_{a \in \mathcal{A}} \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x],$$

and under zero–one loss this is $f^*(x) = \arg\max_c p(y = c \mid x)$ (Theorem 2.2.8). The learning problem arises because $p(y = c \mid x)$ is unknown and must be estimated from labelled examples $\mathcal{D}_N = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$: for instance, is this network flow benign or malicious?

Definition 4.1.1 (Discriminative and generative classifiers)**.**

- A *discriminative* model directly approximates the posterior with a parameterized family $p_\theta(y = c \mid x)$; training produces the fitted posterior $p_{\widehat{\theta}}(y = c \mid x)$. It learns *where the label changes* as a function of the features.
- A *generative* classifier models the class prior $p(y = c)$ and the class-conditional distribution $p(x \mid y = c)$, and obtains the posterior through Bayes' rule. It learns *what each class looks like*, then compares how well each class explains the observed input.



Reading the formula

Both routes end with the same object, a posterior over classes. The difference is the *direction of modelling*. Discriminative: "given these features, what is the probability of attack?" Generative: "what do attack flows typically look like, what do benign flows look like, and how common is each? Now, which story fits this flow better?" The second route builds two "worlds" and asks which one this input belongs to.

Example 4.1.2 (Where each route shows up)**.**

- *Spam filtering.* Naive Bayes (generative) models the token frequencies of spam and of legitimate mail. It is cheap to estimate and still serves as a baseline or a first triage layer.
- *Fraud and intrusion screening.* Class-conditional models score how *compatible* an event is with normal behaviour and with known malicious behaviour, supporting both classification and anomaly triage. Modern systems combine this with richer discriminative and behavioural signals.

- *Image classification.* Deep discriminative networks dominate: modelling the full density of images per class is far harder than learning where the label changes.

Remark 4.1.3 ("Generative" is about direction, not task). A class-conditional model $p(x \mid y = c)$ can be used to *infer* the class of an observed x (classification), or, in principle, to *draw* a new x for a chosen class (generation). Classical generative classifiers and modern generative AI share this modelling direction; their model families and scales differ enormously.

Neither route is universally better. Their relative value depends on the amount of data, the credibility of the modelling assumptions, the representation, and the deployment objective. The rest of the chapter makes each route concrete and then compares them.

Summary: Two routes

- Decision rule known (Bayes); posterior unknown; must be learned from $\mathcal{D}_N$.
- Discriminative: model $p(y \mid x)$. Generative: model $p(x \mid y)$ and $p(y)$, then apply Bayes' rule.
- Same target, different commitments; which is better depends on data, assumptions and objective.

4.2 The Discriminative Route

What are we trying to get?

We want to see how a model that outputs *numbers* (scores) is turned into *probabilities* and then into *actions*, why the cross-entropy loss of Chapter 2 is exactly maximum likelihood here, and what a discriminative model cannot tell us.

Scores, probabilities, actions

A discriminative model often first produces a real-valued *score* (or *logit*) for each class. Scores are converted into posterior estimates and then into an action:

$$\{s_{\widehat{\theta},c}(x)\} \longrightarrow \{p_{\widehat{\theta}}(y = c \mid x)\} \longrightarrow \hat{y}(x).$$

The score model determines how inputs are *ranked*; the probability transformation quantifies *relative uncertainty* among classes; the decision loss determines the *action*.

Definition 4.2.1 (Sigmoid; binary logistic model)**.** The *sigmoid* (logistic) function is $\sigma(s) = \dfrac{1}{1+e^{-s}}$. For binary classification one score suffices, and the posterior estimate is

$$p_{\widehat{\theta}}(y = 1 \mid x) = \sigma(s_{\widehat{\theta}}(x)) = \frac{1}{1 + \exp[-s_{\widehat{\theta}}(x)]}.$$

Logistic regression is the special case of a linear score, $s_\theta(x) = w^\top x + b$, $\theta = (w, b)$.

Reading the formula

σ squashes any real number into $(0,1)$: $\sigma(0)=0.5$, $\sigma(2)\approx 0.88$, $\sigma(-2)\approx 0.12$, $\sigma(5)\approx 0.993$. A score of 0 means "no idea"; large positive means "confidently class 1"; large negative means "confidently class 0." With $w=(1,-2)$, $b=0.5$ and $x=(2,1)$: $s=2-2+0.5=0.5$ and $p(y=1\mid x)=\sigma(0.5)\approx 0.62$.

Proposition 4.2.2 (Properties of the sigmoid)**.** *For all $s\in\mathbb{R}$: (i) $0<\sigma(s)<1$; (ii) $\sigma(-s)=1-\sigma(s)$; (iii) σ is strictly increasing; (iv) $\sigma(s)>\frac{1}{2} \iff s>0$; (v) more generally, for $t\in(0,1)$, $\sigma(s)>t \iff s>\log\dfrac{t}{1-t}$.*

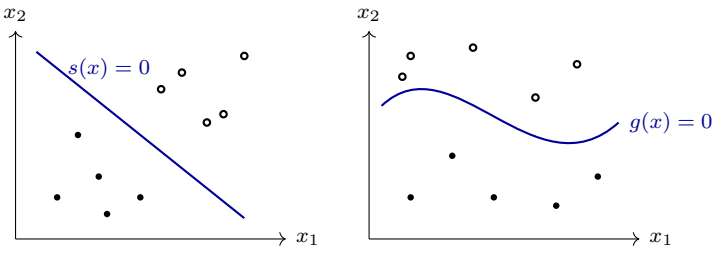
Proof. (i) $e^{-s}>0$ so the denominator exceeds 1. (ii) $1-\sigma(s)=\dfrac{e^{-s}}{1+e^{-s}}=\dfrac{1}{e^{s}+1}=\sigma(-s)$ after multiplying numerator and denominator by e^{s}. (iii) e^{-s} is strictly decreasing, so $1+e^{-s}$ is, so its reciprocal is strictly increasing. (v) $\sigma(s)>t \iff 1+e^{-s}<1/t \iff e^{-s}<(1-t)/t \iff -s<\log\frac{1-t}{t} \iff s>\log\frac{t}{1-t}$. (iv) is (v) with $t=\frac{1}{2}$, since $\log 1=0$. □

Theorem 4.2.3 (Decision boundaries of score models)**.** *Under zero–one loss, the binary logistic model decides $\hat{y}(x)=1 \iff \sigma(s_{\widehat{\theta}}(x))>\frac{1}{2} \iff s_{\widehat{\theta}}(x)>0$. For logistic regression the 0.5-posterior contour is the hyperplane $\hat{w}^{\top}x+\hat{b}=0$. Under costs $C_{\mathrm{FP}}, C_{\mathrm{FN}}$ (Theorem 2.2.9), the decision is $\hat{y}(x)=1 \iff s_{\widehat{\theta}}(x)>\log\dfrac{C_{\mathrm{FP}}}{C_{\mathrm{FN}}}$. For a nonlinear score $g_{\widehat{\theta}}$, the boundary is the surface $g_{\widehat{\theta}}(x)=0$ (shifted by the same constant under unequal costs).*

Proof. The zero–one Bayes decision compares $p(y=1\mid x)$ with $\frac{1}{2}$ (Theorem 2.2.8), and Proposition 4.2.2(iv) converts that into $s>0$; for a linear score, $s=0$ is a hyperplane. Under costs, Theorem 2.2.9 gives threshold $t=C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$, and Proposition 4.2.2(v) with $\frac{t}{1-t}=\frac{C_{\mathrm{FP}}}{C_{\mathrm{FN}}}$ gives $s>\log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$. □

Reading the formula

The probability threshold does *not* make the geometry linear; the score family does. A linear score gives a straight boundary; a curved score gives a curved boundary; changing the costs only *slides* the boundary along the score axis. Cyberattack example with $C_{\mathrm{FN}}=10$, $C_{\mathrm{FP}}=1$: the boundary moves from $s=0$ to $s=\log(1/10)\approx -2.3$, i.e. raise the alarm already when the score is mildly negative. Expensive missed attacks lower the threshold; expensive false alarms raise it. One probability model supports several policies.



linear score: hyperplane nonlinear score: curved boundary

Multiclass scores and softmax

Definition 4.2.4 (Softmax)**.** For scores $s = (s_1, \dots, s_C)$, the *softmax* map produces

$$p_\theta(y = c \mid x) = \mathrm{softmax}(s_\theta(x))_c = \frac{\exp[s_{\theta,c}(x)]}{\sum_{k=1}^{C} \exp[s_{\theta,k}(x)]}, \qquad c = 1, \dots, C.$$

Reading the formula

Exponentiate to make every score positive, then divide by the total so that the classes compete for probability one. Scores $(2, 0, -1)$: exponentials $(7.39, 1.00, 0.37)$, total 8.76, probabilities $(0.84, 0.11, 0.04)$. The largest score always gets the largest probability, and differences of scores are what matter: adding 100 to every score multiplies numerator and denominator by e^{100} and changes nothing.

Proposition 4.2.5 (Softmax facts)**.** *(i) The output lies in the simplex* Δ^{C-1} *and* $\arg\max_c p_\theta(y = c \mid x) = \arg\max_c s_{\theta,c}(x)$*. (ii) Adding the same constant to every score leaves all probabilities unchanged, so only* $C - 1$ *score* contrasts *are identifiable. (iii) With* $C = 2$*,* $\mathrm{softmax}(s)_1 = \sigma(s_1 - s_2)$*: softmax reduces to the sigmoid of the score difference.*

Proof. (i) Exponentials are positive and the entries are divided by their sum, so they are nonnegative and sum to 1; exp is increasing, so the ordering of scores is the ordering of probabilities. (ii) Replacing s_k by $s_k + a$ multiplies every exponential by e^a, which cancels between numerator and denominator. (iii) $\dfrac{e^{s_1}}{e^{s_1} + e^{s_2}} = \dfrac{1}{1 + e^{s_2 - s_1}} = \sigma(s_1 - s_2).$ □

Interpretation & discussion

Implementations still produce C scores rather than $C - 1$: every class then plays the same role, relabelling merely permutes outputs, and one vectorised output layer and loss serve any C. A reference class can remove the redundancy when identifiability matters (e.g. for interpreting coefficients).

Cross-entropy is conditional maximum likelihood

For example i, cross-entropy penalises the probability assigned to the true class:

$$\ell_{\mathrm{CE}}^{(i)}(\theta) = -\sum_{c=1}^{C} \mathbf{1}(y^{(i)} = c) \log p_\theta(y = c \mid x^{(i)}) = -\log p_\theta(y^{(i)} \mid x^{(i)}),$$

so probabilities $0.9, 0.5, 0.1$ on the true class cost about $0.11, 0.69, 2.30$: confident mistakes are punished hardest.

Proposition 4.2.6 (Cross-entropy equals negative conditional log-likelihood)**.** *Assuming the labels are conditionally independent given their inputs, the conditional likelihood is* $L_{\mathrm{cond}}(\theta) = \prod_{i=1}^{N} p_\theta(y^{(i)} \mid x^{(i)})$*, and*

$$-\log L_{\mathrm{cond}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(y^{(i)} \mid x^{(i)}) = N\, \widehat{R}_{\mathrm{CE}}(\theta).$$

Minimising empirical cross-entropy is therefore the same as maximising the conditional likelihood.

Proof. Independence turns the joint probability of the observed labels into a product; the logarithm turns the product into a sum; the sum divided by N is the empirical cross-entropy of Definition 2.5.8. Since $-\log$ is decreasing, maximising L_{cond} and minimising $\widehat{R}_{\text{CE}}$ have the same solutions. □

Reading the formula

"Conditional" likelihood means we only ask the model to explain the *labels given the inputs*, never the inputs themselves. Three examples with true-class probabilities $0.9, 0.5, 0.1$: likelihood $0.9 \times 0.5 \times 0.1 = 0.045$; negative log $= 0.105 + 0.693 + 2.303 = 3.10$; empirical cross-entropy $3.10/3 = 1.03$. Making the likelihood as large as possible and the average loss as small as possible are the same instruction.

Deep discriminative classifiers

Definition 4.2.7 (Feed-forward classifier)**.** A neural network computes a sequence of representations $h^{(0)} = x$, $h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$ for $\ell = 1, \dots, L-1$, with φ an elementwise nonlinearity; the final layer produces scores $s_\theta(x) = W_L h^{(L-1)} + b_L$, and softmax produces $p_\theta(y = c \mid x) = \text{softmax}(s_\theta(x))_c$. Cross-entropy can train all layers end to end.

Reading the formula

Each layer is "multiply by a matrix, add a vector, bend with φ." Logistic regression is the case $L = 1$: no hidden layers, scores linear in x. Adding layers lets the score be a curved function of x, so the boundary of Theorem 4.2.3 can curve. The representation the network learns is optimised for predicting the label, not for reconstructing or explaining the input distribution, which is exactly the point of the next subsection.

What the discriminative objective does not provide

A discriminative model ranks the classes it knows. It never asks whether the input makes sense.

A discriminative classifier focuses on predicting Y from X. This focus is usually an advantage, but it has consequences:

- the model need not learn whether an input x is *plausible*;
- a large $\max_c p_{\widehat{\theta}}(y = c \mid x)$ does not imply that x resembles the training data;
- the model provides no direct mechanism for sampling inputs or for marginalising missing features.

Softmax probabilities are *relative* comparisons among the modelled classes. If all classes explain an unfamiliar input poorly, one class can still receive a posterior near one.

Example 4.2.8 (Confident about nonsense)**.** A three-class network trained on cat, dog and horse photographs receives a photograph of a toaster. Its scores come out $(4.0, 0.5, -1.0)$, so softmax gives $(0.96, 0.03, 0.01)$: "cat, 96%." Nothing in the training objective ever asked the network "is this even an animal?" The number 0.96 measures how much better "cat" explains the toaster

than "dog" and "horse" do, which is all softmax can measure. Section 4.8 adds the missing check.

Remark 4.2.9 (Class imbalance is a different issue). If training and deployment share the same distribution, an imbalanced training prior is not an error: the fitted posterior *should* reflect that prevalence, and the boundary shifts accordingly (with a 5% positive class the 0.5 contour moves toward the positives). The practical difficulty is that severe imbalance may leave too few minority examples to estimate a reliable boundary. Weighting or resampling can help in that small-sample regime; with enough minority examples their benefit is limited and they may distort probability calibration. Section 4.7 separates this from a genuine *change* of prevalence between training and deployment.

Technique: Working with scores and probabilities

- To decide, compare the *score* with $\log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$; you never need to compute σ.
- To convert a probability back to a score: $s = \log \frac{p}{1-p}$ (the *log-odds*); $p = 0.9$ gives $s \approx 2.2$.
- Softmax by hand: subtract the largest score from all scores first (Proposition 4.2.5(ii)); it avoids huge exponentials and changes nothing.
- Never read $\max_c p(y = c \mid x)$ as "probability that the input is normal."

Summary: The discriminative route

- Scores → sigmoid/softmax → probabilities → decision; the score family fixes the boundary geometry, the costs only shift it (Theorem 4.2.3).
- Softmax is shift-invariant: $C - 1$ contrasts matter; with $C = 2$ it is the sigmoid of a difference.
- Cross-entropy = negative conditional log-likelihood (Proposition 4.2.6).
- Discriminative models do not model input plausibility; softmax confidence is relative.

4.3 The Generative Route

What are we trying to get?

We want a model that tells a *story* of how a labelled example came to be, to see how Bayes' rule reverses that story into a classifier, and to understand why fitting such a model asks more of the data than the discriminative route does.

A probabilistic story for labelled data

Definition 4.3.1 (Generative classifier)**.** A generative classifier imagines that an observation is produced in two steps: first choose a class, then generate features from that class,

$$Y \sim \mathrm{Categorical}(\pi), \qquad X \mid (Y = c) \sim p_\theta(\cdot \mid y = c),$$

with parameters $\phi = (\pi, \theta)$, $\pi_c = p(y = c)$. The joint model factorises as

$$p_\phi(x, y = c) = \pi_c \, p_\theta(x \mid y = c).$$

The *prior* π_c is the class frequency before the input is observed; the *class-conditional* density (or mass function) describes which inputs are plausible within class c.

Theorem 4.3.2 (Bayes' rule for classification)**.** *Under the model of Definition 4.3.1, the posterior is*

$$p_\phi(y = c \mid x) = \frac{\pi_c \, p_\theta(x \mid y = c)}{\sum_{k=1}^{C} \pi_k \, p_\theta(x \mid y = k)}.$$

Classification under zero–one loss therefore chooses the class with the largest prior-weighted likelihood $\pi_c \, p_\theta(x \mid y = c)$.

Proof. By definition of conditional probability, $p(y = c \mid x) = p(x, y = c)/p(x)$. The numerator is $\pi_c \, p_\theta(x \mid y = c)$ by the factorisation. The denominator is the marginal $p(x) = \sum_k p(x, y = k) = \sum_k \pi_k \, p_\theta(x \mid y = k)$ (sum over the mutually exclusive, exhaustive classes). The denominator does not depend on c, so the $\arg\max$ over c is decided by the numerator. □

Reading the formula

Numerator: "how common is class c" times "how well does class c explain this x." Denominator: the same quantity summed over all classes, which just rescales so that the posteriors add to 1. A rare class can still win if it explains the input much better. Network security: benign flows have prior 0.99 but give this flow likelihood 0.001; attacks have prior 0.01 but likelihood 0.20.

class	π_c	$p(x \mid y = c)$	$\pi_c \, p(x \mid y = c)$
benign	0.99	0.001	0.00099
attack	0.01	0.20	0.00200

$p(y = \text{attack} \mid x) = 0.002/(0.002 + 0.00099) \approx 0.67$. Attacks remain rare in the population, yet this particular flow is more compatible with the attack model, and the posterior says so.

Generative maximum likelihood

Proposition 4.3.3 (The generative likelihood splits)**.** *For independent labelled observations, the joint likelihood is* $L(\phi) = \prod_{i=1}^{N} p_\phi(x^{(i)}, y^{(i)})$ *and its negative logarithm is*

$$\begin{aligned} -\log L(\phi) &= -\sum_{i=1}^{N} \Big[\log \pi_{y^{(i)}} + \log p_\theta(x^{(i)} \mid y^{(i)}) \Big] \\ &= \underbrace{-\sum_{c=1}^{C} N_c \log \pi_c}_{\text{prior part}} \; \underbrace{-\sum_{c=1}^{C} \sum_{i:\, y^{(i)} = c} \log p_\theta(x^{(i)} \mid y = c)}_{\text{one term per class}}. \end{aligned}$$

Hence the prior and each class-conditional model can be fitted separately*: the*

prior from the class counts alone, and the model of class c from the examples of class c alone.

Proof. Take logs of the product and substitute the factorisation $p_\phi(x, y) = \pi_y\, p_\theta(x \mid y)$. Group the prior terms by class: the label c occurs N_c times, giving $N_c \log \pi_c$. Group the class-conditional terms by class likewise. The first group depends only on π, the second only on θ, and within the second each class's parameters appear only in that class's sum. □

Proposition 4.3.4 (Maximum-likelihood prior)**.** *The prior part is maximised, over the simplex, by the empirical class frequencies $\hat{\pi}_c = N_c/N$.*

Proof. Write $q_c = N_c/N$, a probability vector. Then $\sum_c N_c \log \pi_c = N \sum_c q_c \log \pi_c = -N\, H(q, \pi)$, the negative cross-entropy of Definition 1.8.1. By Gibbs' inequality (Theorem 1.8.2), $H(q, \pi) \geq H(q)$ with equality iff $\pi = q$. So the prior part is largest exactly at $\pi = q$. □

Key insight

Fitting the prior is Gibbs' inequality again: the cross-entropy between the empirical class distribution and the model prior is minimised when they coincide. Generative training asks each example to be explained *twice*: its class (through π) and its features (through $p_\theta(x \mid y)$). That is a stronger requirement than the discriminative route's "explain the class given the features," and it is both the strength (more structure, less data needed) and the weakness (more assumptions that can be wrong) of the generative route.

Interpretation & discussion

Two approximations should be kept separate. *Shape approximation*: the family $p_\theta(x \mid y)$ restricts the class-conditional distributions and may not contain the truth; this error does not vanish with more data. *Finite-sample estimation*: even within the family, $\hat{\phi}$ differs from the best population parameter because it is estimated from finite data; this error shrinks with more data. Both make the fitted posterior $p_{\hat{\phi}}(y = c \mid x)$ differ from the true $p(y = c \mid x)$.

Summary: The generative route

- Story: choose a class from π, generate features from $p_\theta(x \mid y)$; joint $= \pi_c\, p_\theta(x \mid y = c)$.
- Bayes' rule turns the story into a posterior; classification compares prior-weighted likelihoods (Theorem 4.3.2).
- The likelihood splits by class (Proposition 4.3.3); the MLE prior is the class frequency (Proposition 4.3.4).
- Errors come from a wrong family (persistent) and from finite data (shrinking).

4.4 Gaussian Class-Conditional Models

What are we trying to get?

For continuous features, the simplest class "world" is a bell-shaped cloud. We want the fitted parameters in closed form, one worked posterior computed by hand, and, most importantly, to see how an *assumption about covariance* becomes a *shape of the decision boundary*: shared covariance gives straight boundaries (LDA), separate covariances give curved ones (QDA).

Definition 4.4.1 (Multivariate Gaussian class model)**.** For $x \in \mathbb{R}^d$, the Gaussian class-conditional model is

$$p_\theta(x \mid y = c) = \mathcal{N}(x; \mu_c, \Sigma_c) = \frac{1}{(2\pi)^{d/2}|\Sigma_c|^{1/2}} \exp\Big[-\tfrac{1}{2}(x-\mu_c)^\top \Sigma_c^{-1}(x-\mu_c)\Big],$$

with mean $\mu_c \in \mathbb{R}^d$ and symmetric positive-definite covariance Σ_c. In one dimension this is $\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp[-(x-\mu)^2/(2\sigma^2)]$.

Reading the formula

The mean μ_c is the centre of the cloud of class c. The covariance Σ_c is its shape: a multiple of the identity gives circular contours; unequal diagonal entries stretch the circle into an axis-aligned ellipse; off-diagonal entries rotate it (correlated features). The quadratic form $(x-\mu_c)^\top \Sigma_c^{-1}(x-\mu_c)$ is the *squared Mahalanobis distance*: "how many standard deviations away from the centre, in the direction of x." The prefactor with $|\Sigma_c|$ makes the density integrate to one; a diffuse class (large determinant) spreads its probability thin.

Closed-form maximum likelihood

Theorem 4.4.2 (Gaussian MLE)**.** *Let N_c be the number of training examples of class c. The maximum-likelihood estimates are*

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\mu}_c = \frac{1}{N_c} \sum_{i:\, y^{(i)}=c} x^{(i)}, \qquad \hat{\Sigma}_c = \frac{1}{N_c} \sum_{i:\, y^{(i)}=c} (x^{(i)} - \hat{\mu}_c)(x^{(i)} - \hat{\mu}_c)^\top.$$

No iterative optimisation is needed.

Proof. The prior is Proposition 4.3.4. By Proposition 4.3.3 each class is fitted from its own examples, so fix a class and drop the index c; write its $n = N_c$ examples as $x^{(1)}, \dots, x^{(n)}$. We give the full proof in one dimension; the multivariate case follows the same two steps with vector calculus. The negative log-likelihood is

$$J(\mu, \sigma^2) = \frac{n}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x^{(i)} - \mu)^2.$$

Step 1 (mean). For fixed σ^2, $\partial J/\partial\mu = -\frac{1}{\sigma^2}\sum_i (x^{(i)} - \mu) = 0$ gives $\mu = \frac{1}{n}\sum_i x^{(i)}$, and J is a convex quadratic in μ, so this is the minimum. **Step 2 (variance).** With $\mu = \hat{\mu}$ and $S = \sum_i (x^{(i)} - \hat{\mu})^2$, put $v = \sigma^2$: $\partial J/\partial v = \frac{n}{2v} - \frac{S}{2v^2} = 0$ gives $v = S/n$; the derivative is negative for $v < S/n$ and positive beyond, so it is the minimum. In d dimensions the same computation with Σ in place of v gives $\hat{\mu}$ as the class mean and $\widehat{\Sigma}$ as the average outer product of the centred examples. □

Remark 4.4.3 (N_c versus $N_c - 1$). The denominator N_c is the maximum-likelihood denominator. The familiar *unbiased* sample covariance uses $N_c - 1$. For classification the difference is immaterial when N_c is large, but the two are different estimators and you should say which one you use.

Example 4.4.4 (One dimension, by hand)**.** Class 0: $\{0, 1, 2, 0, 1, 2\}$; class 1: $\{3, 4, 5\}$. Then $\hat{\pi}_0 = 6/9 = 2/3$, $\hat{\pi}_1 = 1/3$; $\hat{\mu}_0 = 6/6 = 1$, $\hat{\mu}_1 = 12/3 = 4$; $\hat{\sigma}_0^2 = (1+0+1+1+0+1)/6 = 2/3$, $\hat{\sigma}_1^2 = (1+0+1)/3 = 2/3$.
At $x = 2$: the two densities share the prefactor $1/\sqrt{2\pi \cdot 2/3}$, so compare exponentials. Class 0: $\exp[-(2-1)^2/(2 \cdot \frac{2}{3})] = \exp(-0.75) \approx 0.472$; class 1: $\exp[-(2-4)^2/(2 \cdot \frac{2}{3})] = \exp(-3) \approx 0.050$. Prior-weighted: $\frac{2}{3}(0.472) = 0.315$ and $\frac{1}{3}(0.050) = 0.017$. Normalising,

$$p_{\hat{\phi}}(y = 0 \mid x = 2) = \frac{0.315}{0.315 + 0.017} \approx 0.950, \qquad p_{\hat{\phi}}(y = 1 \mid x = 2) \approx 0.050.$$

The point $x = 2$ is one unit from the centre of class 0 and two units from the centre of class 1; both the distance and the larger prior of class 0 push the posterior toward class 0.

Example 4.4.5 (Where is the boundary in Example 4.4.4?)**.** With equal variances the log prior-weighted likelihoods are $\log \hat{\pi}_c - (x - \hat{\mu}_c)^2/(2 \cdot \frac{2}{3})$. Setting the two equal: $\log \frac{2}{3} - \frac{3}{4}(x-1)^2 = \log \frac{1}{3} - \frac{3}{4}(x-4)^2$, i.e. $\frac{3}{4}[(x-4)^2 - (x-1)^2] = \log \frac{1}{3} - \log \frac{2}{3} = -\log 2$, i.e. $\frac{3}{4}(15 - 6x) = -0.693$, giving $x \approx 2.65$. With equal priors the boundary would sit at the midpoint 2.5; the larger prior of class 0 pushes it toward class 1 by 0.15. This is LDA in one dimension: a single threshold.

Scores, LDA and QDA

Definition 4.4.6 (Generative class score)**.** Define $s_{\hat{\phi},c}(x) := \log \hat{\pi}_c + \log p_{\hat{\theta}}(x \mid y = c)$. Then, by Theorem 4.3.2,

$$\log p_{\hat{\phi}}(y = c \mid x) = s_{\hat{\phi},c}(x) - \log \sum_{k=1}^{C} \exp[s_{\hat{\phi},k}(x)],$$

so the fitted posterior is the softmax of the generative scores, and class prediction can be based on the scores alone (the log-normaliser is common to all classes).

Theorem 4.4.7 (LDA: shared covariance gives linear scores)**.** *Assume every class has the same covariance, $\widehat{\Sigma}_c = \widehat{\Sigma}$. Then, up to a term common to all classes,*

$$s_{\hat{\phi},c}(x) = x^\top \widehat{\Sigma}^{-1} \hat{\mu}_c - \tfrac{1}{2} \hat{\mu}_c^\top \widehat{\Sigma}^{-1} \hat{\mu}_c + \log \hat{\pi}_c,$$

which is affine in x. *Consequently the boundary between any two classes, $\{x : s_{\hat{\phi},c}(x) = s_{\hat{\phi},k}(x)\}$, is a hyperplane. This is* Linear Discriminant Analysis.

Proof. Take the log of the Gaussian density:

$$\log p(x \mid y = c) = -\tfrac{d}{2} \log(2\pi) - \tfrac{1}{2} \log |\widehat{\Sigma}| - \tfrac{1}{2} (x - \hat{\mu}_c)^\top \widehat{\Sigma}^{-1} (x - \hat{\mu}_c).$$

The first two terms do not depend on c (shared covariance). Expand the quadratic form:

$$(x-\hat{\mu}_c)^\top \widehat{\Sigma}^{-1}(x-\hat{\mu}_c) = x^\top \widehat{\Sigma}^{-1}x - 2x^\top \widehat{\Sigma}^{-1}\hat{\mu}_c + \hat{\mu}_c^\top \widehat{\Sigma}^{-1}\hat{\mu}_c,$$

using symmetry of $\widehat{\Sigma}^{-1}$ to combine the two cross terms. The term $x^\top \widehat{\Sigma}^{-1}x$ is common to all classes and drops out of any comparison; what remains, times $-\frac{1}{2}$, plus $\log \hat{\pi}_c$, is the displayed score. Two affine functions of x are equal on a hyperplane. □

Key insight

The only x-quadratic term, $x^\top \Sigma^{-1}x$, is the same for every class when Σ is shared, so it cancels in every comparison. The word *linear* in LDA describes this induced boundary, not the Gaussian density itself, which is anything but linear.

Proposition 4.4.8 (QDA: class-specific covariance gives quadratic scores)**.** *With separate covariances, up to a term common to all classes,*

$$s_{\hat{\phi},c}(x) = -\tfrac{1}{2}(x-\hat{\mu}_c)^\top \widehat{\Sigma}_c^{-1}(x-\hat{\mu}_c) - \tfrac{1}{2}\log|\widehat{\Sigma}_c| + \log \hat{\pi}_c.$$

The first term is a squared Mahalanobis distance to the class centre; the determinant term penalises diffuse classes. The class-dependent quadratic terms make the boundaries curved (conic sections in two dimensions). This is Quadratic Discriminant Analysis*.*

Proof. Same log density as above; now $\log|\widehat{\Sigma}_c|$ and the quadratic form both depend on c, and only $-\frac{d}{2}\log(2\pi)$ is common. Equality of two such scores is a quadratic equation in x. □

Reading the formula

LDA versus QDA is a decision about *how many numbers to estimate*. LDA: one shared covariance, $d(d+1)/2$ numbers, plus C means. QDA: one covariance per class, $C \cdot d(d+1)/2$ numbers. In two dimensions LDA can only draw straight lines between classes; QDA can draw ellipses, parabolas and hyperbolas. More flexibility, more data required. The geometry of the boundary is the visible fingerprint of the assumption you made about the class clouds.

Proposition 4.4.9 (Parameter count of a covariance)**.** *A symmetric $d \times d$ matrix has $d(d+1)/2$ distinct entries. For $d = 100$ that is 5050 per class.*

Proof. The diagonal has d entries; the strictly upper triangle has $\binom{d}{2} = d(d-1)/2$, and symmetry fixes the lower triangle. Total $d + d(d-1)/2 = d(d+1)/2$. With $d = 100$: $100 \cdot 101/2 = 5050$. □

If a class has few observations, its covariance estimate may be unstable or singular (not invertible, so the score cannot even be computed). Two responses: regularise the estimate, or impose stronger structure.

Definition 4.4.10 (Covariance shrinkage)**.** $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda\alpha I$ for $0 \le \lambda \le 1$ and a reference variance $\alpha > 0$ (e.g. the average diagonal entry). $\lambda = 0$ keeps the empirical covariance; $\lambda = 1$ removes all correlations and gives every

direction the same variance. Validation selects λ.

Reading the formula

Shrinkage blends the noisy, data-specific matrix with a stable, featureless one. For any $\lambda > 0$ the result is positive definite (a positive-definite matrix plus a positive multiple of the identity), so it can always be inverted. Taking $\lambda \to 1$ and then dropping the remaining correlations altogether leads to the next section.

Technique: Gaussian classifiers by hand

- Compute class counts, means, and variances (divide by N_c).
- Drop constants common to all classes before comparing: with equal variances compare $\log \hat{\pi}_c - (x - \hat{\mu}_c)^2/(2\sigma^2)$ only.
- To get a posterior, exponentiate the scores and normalise (softmax of the generative scores, Definition 4.4.6).
- Equal priors and equal variances in one dimension: the boundary is the midpoint of the means. A larger prior moves it away from that class; a larger variance in QDA bends the region.

Classic problems

- *Two classes, equal priors,* $\mu_0 = 0$, $\mu_1 = 4$, *shared* $\sigma^2 = 1$. *Boundary?* $x = 2$.
- *Same, but* $\pi_0 = 0.9$, $\pi_1 = 0.1$. Solve $-\frac{1}{2}x^2 + \log 0.9 = -\frac{1}{2}(x-4)^2 + \log 0.1$: $4x - 8 = \log 9 \approx 2.197$, so $x \approx 2.55$.
- $d = 10$ *features, QDA with* $C = 3$ *classes: how many covariance parameters?* $3 \times 55 = 165$; LDA: 55.
- *Class with* $N_c = 5$ *examples in* $d = 10$ *dimensions: what goes wrong?* The 10×10 covariance has rank at most 4: singular, not invertible. Shrink or use Naive Bayes.

Summary: Gaussian class models

- MLE in closed form: class frequencies, class means, class covariances with denominator N_c (Theorem 4.4.2).
- Fitted posterior = softmax of generative scores $\log \hat{\pi}_c + \log p(x \mid y = c)$.
- Shared covariance $\Rightarrow$ affine scores $\Rightarrow$ hyperplane boundaries (LDA, Theorem 4.4.7); separate covariances $\Rightarrow$ quadratic boundaries (QDA).
- A covariance costs $d(d+1)/2$ parameters; shrink it or simplify the model when data are scarce.

4.5 Naive Bayes

What are we trying to get?

When estimating a full joint distribution of the features is hopeless, we can pretend the features are independent *within each class*. We want to see exactly what this buys (one small model per feature, additive evidence, stability on little data), what it costs (double-counted evidence, overconfident probabilities), and how to fix the two failure modes that appear in practice: zero variances and zero counts.

Definition 4.5.1 (Naive Bayes)**.** Naive Bayes assumes the features are *conditionally independent given the class*:

$$p_\theta(x \mid y = c) = \prod_{j=1}^{d} p_\theta(x_j \mid y = c).$$

Taking logarithms, the generative score becomes *additive*:

$$s_{\hat{\phi},c}(x) = \log \hat{\pi}_c + \sum_{j=1}^{d} \log p_{\widehat{\theta}}(x_j \mid y = c).$$

Reading the formula

Read the score as a ledger: start with the prior evidence $\log \hat{\pi}_c$, then add one line of evidence per feature, each computed from a *one-dimensional* model of that feature within class c. Flow duration, packet counts and byte statistics may in reality be correlated within an attack class, so the assumption is strong. What it buys is that a difficult d-dimensional density estimation problem becomes d easy one-dimensional ones, each of which can be stable with limited data.

Interpretation & discussion

Two facts make Naive Bayes far more useful than its assumption suggests. First, a *misspecified density can still rank classes correctly*: getting the largest score right is a weaker requirement than getting every density right. Second, the one-dimensional models need not be Gaussian or even of the same family: a positive skewed feature may get a Gamma or log-normal, a count a Poisson, an irregular feature a histogram. The main risk is that *correlated features repeat the same evidence*: two nearly identical features count twice, so the classifier may still classify well while its probabilities become too extreme. Removing redundant features improves calibration.

Gaussian Naive Bayes and variance smoothing

For numerical features, Gaussian Naive Bayes models $x_j \mid (y = c) \sim \mathcal{N}(\hat{\mu}_{cj}, \hat{\sigma}^2_{cj})$, with the one-dimensional MLEs of Theorem 4.4.2 applied feature by feature. Two problems arise. If a fitted variance is 0 (all class-c values of feature j coincide), the density is undefined. If it is merely tiny, that one feature's narrow density dominates the whole class score.

Definition 4.5.2 (Variance smoothing)**.** Replace each fitted variance by $\tilde{\sigma}_{cj}^2 = \hat{\sigma}_{cj}^2 + \lambda s_j^2$ with $\lambda > 0$, where s_j^2 is a reference scale such as the overall training variance of feature j. A small λ prevents numerical instability; if its value materially affects predictions, select it on validation data.

Example 4.5.3 (A tiny variance takes over)**.** Two features, class c: feature 1 has $\hat{\sigma}^2 = 1$, feature 2 has $\hat{\sigma}^2 = 10^{-6}$ (every training example of class c had exactly the same value, say a port number). A test point whose feature 2 differs from that value by 0.01 contributes $-\frac{1}{2}(0.01)^2/10^{-6} = -50$ to the score of class c, dwarfing anything feature 1 can say. With $\lambda s_2^2 = 0.01$ added, the contribution becomes $-\frac{1}{2}(10^{-4})/(0.01) \approx -0.005$: feature 2 still counts, but no longer vetoes.

Variant	Feature model	Typical use
Gaussian	continuous Gaussian variable	measurements, flow statistics
Bernoulli	binary presence/absence	indicators
Multinomial	nonnegative event or token counts	documents, event counts
Hybrid	distribution chosen by feature type	mixed tabular data

A hybrid model multiplies contributions from different feature-specific distributions under one class prior; the prior appears once in the score, never once per feature.

Multinomial Naive Bayes for text

Definition 4.5.4 (Multinomial Naive Bayes)**.** Fix a vocabulary of V words and represent a document by its count vector $x = (x_1, \ldots, x_V)$. For class c, let $\theta_{cj} = p_\theta(\text{token} = j \mid y = c)$ with $\sum_j \theta_{cj} = 1$. Ignoring the multinomial coefficient (common to all classes),

$$p_\theta(x \mid y = c) \propto \prod_{j=1}^{V} \theta_{cj}^{x_j},$$

with the convention $0^0 = 1$. Repeated words contribute repeated evidence; absent words ($x_j = 0$) contribute the factor 1; word order is ignored.

Reading the formula

Imagine class c as a bag of words with proportions $\theta_{c\cdot}$; a document is produced by drawing its words from the bag. "free free prize" with vocabulary (free, win, prize, meeting, project, report) has $x = (2, 0, 1, 0, 0, 0)$ and likelihood $\propto \theta_{c,\text{free}}^2\, \theta_{c,\text{prize}}$. The same word twice is twice the evidence. The multinomial coefficient (how many orderings produce these counts) is the same for every class and cancels in Bayes' rule.

Example 4.5.5 (Why smoothing is needed)**.** Aggregate training counts, seven tokens per class:

	free	win	prize	meeting	project	report
spam	3	2	2	0	0	0
ham	0	0	0	3	2	2

Without smoothing, $\hat{\theta}_{cj} = n_{cj}/7$. The test message "free meeting" gives

$$p(x \mid \text{spam}) \propto \tfrac{3}{7} \cdot \tfrac{0}{7} = 0, \qquad p(x \mid \text{ham}) \propto \tfrac{0}{7} \cdot \tfrac{3}{7} = 0.$$

One unseen word zeroes an entire product, and here *both* products vanish, so the posterior $0/0$ is undefined. A single word never seen in training in a class would otherwise veto that class no matter how strong the other evidence.

Definition 4.5.6 (Additive smoothing)**.** With n_{cj} the training count of token j in class c and $\alpha > 0$,

$$\hat{\theta}_{cj} = \frac{n_{cj} + \alpha}{\sum_{\ell=1}^{V} n_{c\ell} + \alpha V}.$$

For a categorical feature x_j with K_j levels, with n_{cjv} the count of value v in class c and $n_{cj} = \sum_v n_{cjv}$,

$$p_{\hat{\theta}}(x_j = v \mid y = c) = \frac{n_{cjv} + \alpha}{n_{cj} + \alpha K_j}.$$

Proposition 4.5.7 (Smoothed estimates are proper and interpolate to uniform)**.** *For every $\alpha > 0$: (i) $\hat{\theta}_{cj} > 0$ for all j and $\sum_j \hat{\theta}_{cj} = 1$; (ii) as $\alpha \to 0$, $\hat{\theta}_{cj} \to n_{cj}/\sum_\ell n_{c\ell}$ (the unsmoothed estimate); (iii) as $\alpha \to \infty$, $\hat{\theta}_{cj} \to 1/V$ (uniform). Moreover $\hat{\theta}_{cj}$ is the estimate one obtains by adding α imaginary occurrences of every word to the class.*

Proof. (i) Numerators are $\geq \alpha > 0$; summing $n_{cj} + \alpha$ over j gives exactly the denominator. (ii) and (iii): divide numerator and denominator by α for (iii), or set $\alpha = 0$ in the limit for (ii). The last claim is the formula itself: counts $n_{cj} + \alpha$ over total $\sum_\ell n_{c\ell} + V\alpha$. □

Reading the formula

Pretend you saw every word α extra times in every class. With $\alpha = 1$ ("Laplace smoothing") and $V = 6$: spam counts become $(4, 3, 3, 1, 1, 1)$ over $7 + 6 = 13$, ham counts $(1, 1, 1, 4, 3, 3)$ over 13:

$$\hat{\theta}_{\text{spam}} = \tfrac{1}{13}(4, 3, 3, 1, 1, 1), \qquad \hat{\theta}_{\text{ham}} = \tfrac{1}{13}(1, 1, 1, 4, 3, 3).$$

Larger α pulls all estimates toward $1/6$; smaller α trusts the counts more.

Example 4.5.8 (The smoothed verdict on "free meeting")**.** With equal priors $\frac{1}{2}$, the unnormalised scores are

$$\exp[s_{\text{spam}}] = \tfrac{1}{2} \cdot \tfrac{4}{13} \cdot \tfrac{1}{13} = \tfrac{2}{169}, \qquad \exp[s_{\text{ham}}] = \tfrac{1}{2} \cdot \tfrac{1}{13} \cdot \tfrac{4}{13} = \tfrac{2}{169},$$

so $p(\text{spam} \mid x) = \frac{1}{2}$. Both words are now possible under both classes, and in

this symmetric message their evidence balances exactly.

Example 4.5.9 (A less symmetric message: "free prize"). $\exp[s_{\text{spam}}] = \frac{1}{2} \cdot \frac{4}{13} \cdot \frac{3}{13} = \frac{12}{338}$ and $\exp[s_{\text{ham}}] = \frac{1}{2} \cdot \frac{1}{13} \cdot \frac{1}{13} = \frac{1}{338}$, so $p(\text{spam} \mid x) = 12/13 \approx 0.92$. And "free free prize" (the word *free* twice): $\frac{1}{2}(\frac{4}{13})^2 \frac{3}{13} = \frac{48}{4394}$ versus $\frac{1}{2}(\frac{1}{13})^2 \frac{1}{13} = \frac{1}{4394}$, so $p(\text{spam} \mid x) = 48/49 \approx 0.98$: the repeated word counts twice, as the multinomial model says it should.

Remark 4.5.10 (Unseen categories). A categorical value that appears at validation or test time but never in training (a new port number, say) must have a defined representation: create an *other* level during preprocessing, include it among the K_j levels, and map every unseen value to it. Smoothing then gives it a small positive probability instead of a crash.

Technique: Naive Bayes by hand

- Work in logs when there are more than three or four features; products of small numbers underflow and are error-prone.
- Apply smoothing *before* normalising: add α to every count, add αV (or αK_j) to the total.
- Compare unnormalised scores first; normalise only if a posterior is asked for.
- Count the prior exactly once, not once per feature.
- Remove near-duplicate features (e.g. bytes and packets when they are proportional) to avoid double counting.

Classic problems

- $V = 4$, *class counts* $(5, 0, 3, 2)$, $\alpha = 1$. *Smoothed probabilities?* $(6, 1, 4, 3)/14$.
- *Same counts,* $\alpha = 100$*: roughly?* Close to $(1, 1, 1, 1)/4$: $(105, 100, 103, 102)/410 \approx (0.256, 0.244, 0.251, 0.249)$.
- *Bernoulli NB with features (has link, from known sender): spam rates* $(0.9, 0.1)$*, ham rates* $(0.2, 0.8)$*, priors* 0.5*. An email with a link from a known sender?* Spam: $0.5 \cdot 0.9 \cdot 0.1 = 0.045$; ham: $0.5 \cdot 0.2 \cdot 0.8 = 0.08$. Ham, with posterior $0.08/0.125 = 0.64$.
- *Why can Gaussian NB have a boundary that is not linear even though each feature is Gaussian?* Class-specific variances per feature make the scores quadratic in x (a diagonal QDA).

Summary: Naive Bayes

- Assume conditional independence of features given the class; the score becomes prior evidence plus one additive term per feature.
- One-dimensional models per feature: Gaussian, Bernoulli, multinomial, or a hybrid; choose by feature type and validate.
- Fix zero variances by adding λs_j^2; fix zero counts by additive smoothing

$(n + \alpha)/(\text{total} + \alpha V)$ (Proposition 4.5.7).

- Correlated features double-count evidence: ranking may survive, calibration suffers.

4.6 Comparing the Two Routes

What are we trying to get?

We want a fair statement of when each route is preferable, phrased in terms of assumptions and data, not slogans.

Question	Discriminative	Generative
Modelled quantity	posterior $p_\theta(y \mid x)$ or a boundary	prior π_c and class model $p_\theta(x \mid y = c)$
Input assumptions	usually fewer	usually stronger
Primary objective	predict the class	describe class-specific inputs
Missing features	need separate treatment	can sometimes be marginalised
Sampling inputs	usually unavailable	possible in principle
Raw high-dimensional input	often practical	full density estimation is hard

Interpretation & discussion

Generative structure can *reduce estimation error*: a restricted description of each class has few parameters, so with little labelled data it can be fitted reliably, provided the assumptions are approximately right. A wrong class-conditional family creates *approximation bias* that does not vanish with more data. A sufficiently flexible discriminative model may eventually reach lower classification error because it spends no capacity modelling aspects of X that are irrelevant to Y. Schematically, error versus labelled data often shows the generative curve lower at small N and the discriminative curve lower at large N, but this is not a universal law: the crossing point, and whether it exists, depends on the data distribution, the two families, regularisation and optimisation. The right question is not "which route is better" but "which assumptions are credible, and which deployment objective matters."

Example 4.6.1 (Same data, two verdicts)**.** Spam data with 200 labelled emails and a 10,000-word vocabulary. Multinomial Naive Bayes needs to estimate $2 \times 10{,}000$ smoothed word probabilities, each from simple counts: it works reasonably from day one. Logistic regression on the same counts must locate a 10,000-dimensional hyperplane from 200 points: badly underdetermined, prone to overfitting. With 2 million labelled emails, the picture flips: logistic regression (or a network) learns interactions such as "*free* is spammy unless *invoice* is nearby," which the independence assumption forbids Naive Bayes from ever representing.

Summary: Comparing the routes

- Generative: more assumptions, less data needed, persistent bias if wrong, extra abilities (missing features, sampling, plausibility).
- Discriminative: fewer assumptions, better with much data, nothing about the input distribution.
- Decide by credibility of assumptions and by the deployment objective, then validate.

4.7 Class Imbalance and Prior Shift

What are we trying to get?

Two situations are constantly confused: a class that is rare, and a class whose frequency *changes* between training and deployment. We want to separate them, and for the second, derive the exact correction for both routes.

Definition 4.7.1 (Calibration)**.** A probabilistic classifier is *calibrated* if, among cases assigned probability u, approximately a fraction u are positive. Calibration concerns the interpretation of probabilities; the choice of a decision threshold from costs concerns the action taken from them. These are separate questions.

If training and deployment share the same joint distribution, an imbalanced training prior is not an error: the fitted posterior should reflect that prevalence. The difficulty is finite minority data: with few positives, the minority distribution or the boundary is poorly estimated, and weighting or resampling may help estimation. Otherwise their benefit is limited and the resulting probabilities may need recalibration.

Definition 4.7.2 (Class-prior shift)**.** Between training (t) and deployment (d), the class-conditional distributions are unchanged but the priors differ:

$$p_d(x \mid y = c) = p_t(x \mid y = c), \qquad \pi_{d,c} \neq \pi_{t,c}.$$

Example: attacks form 20% of the training data but 5% of deployment traffic.

Theorem 4.7.3 (Generative correction)**.** *Under prior shift, the deployment posterior is obtained by keeping the fitted class-conditional models and replacing the training priors by the deployment priors:*

$$p_d(y = c \mid x) = \frac{\pi_{d,c}\, p_{\widehat{\theta}}(x \mid y = c)}{\sum_{k=1}^{C} \pi_{d,k}\, p_{\widehat{\theta}}(x \mid y = k)}.$$

No class-conditional model needs to be refitted.

Proof. Bayes' rule (Theorem 4.3.2) at deployment uses π_d and $p_d(x \mid y)$; by Definition 4.7.2 the latter equals $p_t(x \mid y)$, which is what the fitted $p_{\widehat{\theta}}(x \mid y)$ estimates. Substituting gives the formula. □

Theorem 4.7.4 (Discriminative correction, binary case)**.** *Let* $r_t(x) = p_t(y = 1 \mid x)$ *and* $r_d(x) = p_d(y = 1 \mid x)$*, and write* $\pi_t = p_t(y = 1)$*,* $\pi_d = p_d(y = 1)$*. Under prior shift,*

$$r_d(x) = \frac{(\pi_d/\pi_t)\, r_t(x)}{(\pi_d/\pi_t)\, r_t(x) + \dfrac{1-\pi_d}{1-\pi_t}\,[1 - r_t(x)]}.$$

Proof. **Step 1 (odds form of Bayes' rule).** At training time, $\dfrac{r_t(x)}{1-r_t(x)} = \dfrac{\pi_t\, p(x \mid y=1)}{(1-\pi_t)\, p(x \mid y=0)}$: divide the two numerators of Theorem 4.3.2; the common denominator cancels. Likewise at deployment with π_d.

Step 2 (the likelihood ratio is shared). Under prior shift, $p(x \mid y = 1)/p(x \mid y = 0)$ is the same at both times. Solving the training odds for it and substituting into the deployment odds,

$$\frac{r_d(x)}{1-r_d(x)} = \frac{\pi_d/(1-\pi_d)}{\pi_t/(1-\pi_t)} \cdot \frac{r_t(x)}{1-r_t(x)}.$$

Step 3 (solve for r_d). If odds $= A/B$ then probability $= A/(A+B)$. With $A = \frac{\pi_d}{\pi_t} r_t(x)$ and $B = \frac{1-\pi_d}{1-\pi_t}(1 - r_t(x))$, the odds above equal A/B, hence $r_d = A/(A+B)$, which is the displayed formula. □

Reading the formula

The correction *reweights the two hypotheses*: multiply the training probability of class 1 by "how much rarer class 1 became" (π_d/π_t), multiply the probability of class 0 by "how much commoner class 0 became" $((1-\pi_d)/(1-\pi_t))$, then renormalise. Attacks 20% → 5%: $\pi_d/\pi_t = 0.25$ and $(1-\pi_d)/(1-\pi_t) = 0.95/0.80 = 1.1875$. A flow with training posterior $r_t = 0.5$ gets $A = 0.125$, $B = 0.59375$, so $r_d = 0.125/0.71875 \approx 0.17$. What looked like a coin flip in training is a 17% attack probability in deployment, because attacks are four times rarer there. The correction assumes r_t is a *calibrated* training posterior; the generative route makes the prior an explicit component, the discriminative route can only correct after the fact and depends on the quality of its probabilities.

Gotcha

- Imbalance ≠ shift. Imbalance with enough minority data needs no correction at all; shift needs one even with abundant data.
- Reweighting or resampling the training set *changes the effective prior*: the fitted posterior then corresponds to the resampled prior and must be corrected back with Theorem 4.7.4 if calibrated probabilities are wanted.
- Cost-sensitive thresholds (Theorem 2.2.9) and prior correction are different operations; apply the correction first, then the threshold.

Summary: Imbalance and prior shift

- Rare class with enough data: the posterior should reflect the prevalence; with too little data, weighting or resampling helps estimation but may hurt calibration.

- Prior shift: same class-conditionals, new priors. Generative: swap the priors (Theorem 4.7.3). Discriminative: reweight the odds by π_d/π_t and $(1-\pi_d)/(1-\pi_t)$ (Theorem 4.7.4), which requires calibrated training probabilities.

4.8 Unfamiliar Inputs and Combined Systems

What are we trying to get?

Security systems meet inputs that belong to no known class. We want a mechanism that can say "I do not recognise this," and to see how modern systems combine the two routes, including generative language models used for classification.

Posterior confidence is not familiarity

A high value of $\max_c p_{\widehat{\theta}}(y = c \mid x)$ only means that one modelled class outranks the others. It does not establish that x resembles the training data. A new attack mechanism may lie far from every known class while still receiving a confident known-class prediction.

Definition 4.8.1 (Reject option via a compatibility check)**.** Choose the most probable known class $c^* = \arg\max_c p_{\hat{\phi}}(y = c \mid x)$, then compare the fitted class likelihood with a class-specific threshold:

$$\hat{y} = \begin{cases} c^* & \text{if } \log p_{\widehat{\theta}}(x \mid y = c^*) \geq \tau_{c^*}, \\ \text{reject} & \text{otherwise.} \end{cases}$$

Reading the formula

Two questions instead of one. *Which known class fits best?* (the posterior). *Does even the best class fit well enough?* (the likelihood against a threshold). Only a generative model can answer the second, because only it has a notion of what inputs of class c look like. The threshold τ_c should be validated on data that reproduce the intended novelty, e.g. attack mechanisms excluded from training. A low likelihood is evidence of incompatibility, not a guarantee of detection: high-dimensional likelihoods behave counterintuitively, and a misspecified class model may give low likelihood to perfectly valid cases.

Example 4.8.2 (Combining both routes)**.** A discriminative model ranks the known classes for an observed flow, producing $p_{\widehat{\theta}_d}(y = c \mid x)$. A generative or anomaly model computes $\log p_{\widehat{\theta}_g}(x \mid y = c)$ for the predicted class. The final policy accepts the known class, requests human review, or rejects the input as unfamiliar. The discriminative model is good at the ranking; the generative model is the plausibility check it lacks.

Generative language models and classification

An autoregressive language model is trained *generatively* by factorising a token sequence,

$$p_\theta(w_{1:T}) = \prod_{t=1}^{T} p_\theta(w_t \mid w_{<t}),$$

its pretraining objective being next-token prediction (Chapter 1, Example 1.5.4). A *sequence-classification head* instead models $p_\psi(y \mid w_{1:T})$ and minimises the discriminative objective $L_{\text{disc}}(\psi) = -\sum_i \log p_\psi(y^{(i)} \mid w^{(i)}_{1:T_i})$: one label for the entire sequence, not one per token. Alternatively, a prompt can ask the model to generate or score label tokens directly. A single system thus combines generative pretraining with a discriminative downstream objective.

Summary: Unfamiliar inputs and combined systems

- Softmax confidence compares known classes only; a reject option needs a likelihood check (Definition 4.8.1) with validated thresholds.
- Modern systems combine a discriminative ranker with a generative compatibility model.
- Language models: generative pretraining (next token) plus a discriminative classification head or prompted label scoring.

4.9 Model Choice and Lab Preparation

Three questions organise model choice.

1. *Can a credible class-conditional model $p_\theta(x \mid y)$ be stated?* If so, its structure may reduce the amount of labelled data required.
2. *Is the main objective accurate prediction with many labelled examples?* A flexible discriminative model avoids unnecessary assumptions about the input distribution.
3. *Is the raw input complex and high-dimensional?* Learn a representation first (Chapter 1), then compare simple classification heads; that is more credible than modelling the raw density.

Deployment-like validation decides between credible alternatives. The labels "generative" and "discriminative" do not replace empirical comparison under the intended use conditions.

Example 4.9.1 (Lab 2 at a glance)**.** The lab compares the routes on cybersecurity flow data: Gaussian, multinomial and hybrid Naive Bayes for numerical, service and mixed feature sets, against logistic regression and a decision tree; then limited training data (20% of each class), deployment prior shift ($\pi_d = 0.05$, corrected by swapping priors for the generative models and by Theorem 4.7.4 for the discriminative ones), and multiclass classification with and without destination-port information. Every model must use the same fixed stratified split and the same rows, so that differences reflect the models rather than the data.

Notes on the original slides and notes

All formulas and numbers were rederived: the rare-attack posterior 0.67, the one-dimensional Gaussian example ($\hat{\sigma}^2 = 2/3$ for both classes, posterior 0.950), the covariance count 5050, the smoothed word probabilities and the $\frac{1}{2}$ posterior for "free meeting," and the prior-shift formula. All are correct. Points to note:

- **The slide deck is duplicated.** The Lecture 4 PDF contains its first 34 slides twice: pages 35–68 repeat pages 1–34 verbatim (title, objectives, two routes, discriminative route, Gaussian models, variance smoothing). The genuinely new material resumes at page 69 ("Beyond Gaussian Naive Bayes"). Nothing is lost, but do not look for new content in the repeated block.
- **Cross-entropy table (notes Section 2.4, slide 14).** "$0.9, 0.5, 0.1 \to 0.11, 0.69, 2.30$" uses natural logarithms: $-\ln 0.9 = 0.105$, $-\ln 0.5 = 0.693$, $-\ln 0.1 = 2.303$.
- **Covariance MLE denominator (notes Section 4.1).** The notes correctly stress N_c rather than $N_c - 1$. The proof of the one-dimensional case is Theorem 4.4.2; the multivariate case is stated without the matrix calculus.
- **LDA score (notes eq. 21, slide 28).** The score is affine in x only after dropping $x^\top \Sigma^{-1} x$ and $\log|\Sigma|$, both common to all classes *because* the covariance is shared; Theorem 4.4.7 shows the cancellation explicitly.
- **Shrinkage (notes eq. 24).** The reference variance α is unspecified in the notes; any positive scale works, and the average diagonal entry of $\widehat{\Sigma}$ is a common choice. The point is positive-definiteness for any $\lambda > 0$.
- **Multinomial likelihood (notes eq. 29).** The proportionality hides the multinomial coefficient, which cancels in Bayes' rule because it depends only on x, not on the class. The convention $0^0 = 1$ is needed for absent words.
- **Prior-shift correction (notes eq. 34).** The formula is right; the derivation through odds is Theorem 4.7.4. It silently requires r_t to be calibrated, as the notes remark afterwards.

Chapter takeaways

Summary: Discriminative and generative classification

- **Decisions** depend on the posterior and the loss; the posterior must be *learned*.
- **Discriminative** models estimate $p(y \mid x)$: scores, sigmoid/softmax, cross-entropy as conditional maximum likelihood; the score family fixes the boundary geometry.
- **Generative** classifiers model $p(x \mid y)$ and $p(y)$, then apply Bayes' rule; the likelihood splits by class and the MLE prior is the class frequency.
- **Gaussian** class models: closed-form MLE; shared covariance gives linear boundaries (LDA), separate covariances quadratic ones (QDA); covariance costs $d(d+1)/2$ parameters.
- **Naive Bayes** trades a strong independence assumption for stability; smooth variances and counts.

- **Trade-off:** generative structure helps with little data if credible; misspecification is persistent; flexible discriminative models win with much data.
- **Imbalance** is an estimation issue; **prior shift** needs a posterior correction (swap priors, or reweight odds).
- **Confidence is not familiarity**: add a likelihood-based reject option; combine routes; language models pair generative pretraining with discriminative heads.

Chapter 5

Learning by Combining Models

How to read the formulas in this chapter

Chapters 3 and 4 dealt with *one* fitted model: its bias and variance, its posterior estimate, its validation. This chapter fits *many* models and combines them. The new symbols are listed here; as before, every formula is followed by a *Reading the formula* box.

Symbol	Meaning in words
M	number of fitted models (trees, classifiers) in the ensemble
$\widehat{f}_m(x)$, $\widehat{p}_m(y = c \mid x)$	prediction, or posterior estimate, of the m-th fitted model
$\widehat{f}_{\text{ens}}(x)$	the ensemble prediction (average or vote)
σ^2, ρ	error variance of one model; correlation between the errors of two models
$\mathcal{D}^*_m$	the m-th *bootstrap sample*: N draws with replacement from $\mathcal{D}$
$\text{count}(i)$	how many times observation i appears in one bootstrap sample
$\mathcal{M}_i$	the *out-of-bag* models for observation i: those whose bootstrap sample omits i
$S_1, \ldots, S_J$; θ_j	the regions (leaves) of a tree and the constant predicted in each
$I(S)$	impurity of a set of observations (Gini for classification, squared error for regression)
$\widehat{p}_c(S)$	fraction of observations in S with class c
m_{try}	number of randomly chosen candidate features offered at each split of a random forest
Δ_t, MDI_j, I_j	credit of split t; mean-decrease-in-impurity and permutation importance of feature j
$F_m(x)$, $T_m(x)$, η	boosted ensemble after m rounds; the m-th tree; the learning rate (shrinkage)
$r_m^{(i)}$	residual, or pseudo-residual, of observation i before round m
$w_{q,m}$, $S_{q,m}$, Q_m	output of leaf q of tree m; its region; number of leaves of tree m
$g^{(i)}$, $h^{(i)}$	first and second derivative of the loss of observation i with respect to its current logit
G_q, H_q	sums of $g^{(i)}$ and $h^{(i)}$ over the observations in leaf q
λ, γ	XGBoost penalties: on squared leaf outputs; per leaf
z_m, $h_{\hat{\gamma}}$	stacking: base-model predictions and the fitted meta-model
$g_m(x)$, $a_m(x)$	mixture of experts: gate weight and gate score of expert m

5.1 Why Combine Models?

What are we trying to get?

Chapter 3 showed that a flexible model trained on one sample has high *variance*: a different sample would have produced a noticeably different fit. We want to see why *averaging several fits* attacks exactly this problem, under what condition it works (their errors must differ), and which three ways of producing different fits are in common use.

The wisdom of the crowd

Many rough guesses, averaged, can beat each rough guess.

At a livestock fair in 1906, 787 visitors guessed the dressed weight of an ox. Individual guesses varied widely, yet their mean, 1197 lb, was within one pound of the true weight, 1198 lb (Galton, *Vox Populi*, 1907). The same idea is used to label data: when three annotators look at a network event and two say *malicious* while one says *benign*, the majority label is kept, and strong disagreements are sent to an expert. Replace "annotators" by "fitted models" and you have an *ensemble*: several models produce several predictions, and a vote or an average produces one.

Two conditions make a crowd wise. The members must be *competent*: averaging random noise gives noise. And their errors must *differ*: if everybody makes the same mistake, the mistake survives the average. Three copies of the same model are not an ensemble; they have the same strengths and the same mistakes.

One fitted model can be unstable

Same algorithm, different sample, different boundary: that is variance.

Train the same learning algorithm on two different samples from the same population and the two fitted decision boundaries differ. An unstable learner fits the signal *and* the sample-specific noise; the noise part changes from sample to sample. In the language of Definition 3.3.1, the prediction at a fixed x has a large $\mathrm{Var}(x)$. Where positive and negative errors occur in different places for different fits, averaging the fits cancels part of the fluctuation and leaves the shared signal.

Key insight

Combining models helps when the component models are useful *and* their errors differ. Shared errors do not cancel. Every ensemble method is a recipe for producing competent models whose errors are as different as possible, plus a rule for combining them.

Three families

The methods of this chapter differ in *how* they obtain diversity and *how* they combine.

Family	How diversity is obtained	How predictions are combined
Voting / averaging (§5.2)	different algorithms, settings, features or initialisations, usually all fitted to the *same* data	fixed vote or average
Bagging / random forest (§5.4–5.5)	randomised *observations* (bootstrap samples) and randomised *candidate features*	fixed vote or average
Boosting (§5.7–5.9)	each new model targets the *current errors* of the ensemble	successive additive corrections

Two further architectures replace the fixed combination rule by a learned one: *stacking* (§5.12) learns one global combination of the base predictions, and a *mixture of experts* (§5.13) learns an input-dependent combination.

Summary: Why combine

- A single flexible fit has high variance: it reflects the sample it was trained on.
- Averaging competent models with *different* errors cancels part of the fluctuation; shared errors remain.
- Diversity comes from different models (voting), different data (bagging, random forests) or different targets (boosting); the combination is a vote, an average or a sum of corrections.

5.2 Averaging and Voting

What are we trying to get?

We want the two elementary combination rules, to see when hard and soft voting disagree and what each one throws away, and to *prove* how much averaging reduces variance as a function of the correlation between the component errors.

The rules

Definition 5.2.1 (Averaging; hard and soft voting)**.** Let $\widehat{f}_1, \dots, \widehat{f}_M$ be fitted models.

- *Regression (averaging):* $\widehat{f}_{\text{ens}}(x) = \dfrac{1}{M} \displaystyle\sum_{m=1}^{M} \widehat{f}_m(x)$.
- *Hard voting:* each classifier outputs a class $\hat{y}_m(x)$; the ensemble outputs the most frequent class.
- *Soft voting:* each classifier outputs posterior estimates; the ensemble averages them and takes the largest average,

$$\widehat{p}_{\text{ens}}(y = c \mid x) = \frac{1}{M} \sum_{m=1}^{M} \widehat{p}_m(y = c \mid x), \quad \hat{y}(x) = \arg\max_c \widehat{p}_{\text{ens}}(y = c \mid x).$$

Reading the formula

Hard voting counts heads; soft voting pools confidence. A classifier that says 0.49 and one that says 0.05 cast the *same* hard vote (class 0) but contribute very differently to a soft vote. Soft voting therefore uses strictly more information: the *margin* by which each model preferred its class. The price is that this information must be trustworthy: the probabilities should be reasonably calibrated (Chapter 3) and computed under the same class prior, otherwise one overconfident model dominates the average.

Example 5.2.2 (Hard and soft votes disagree)**.** Three classifiers evaluate the same flow with threshold 0.5:

model	$\widehat{p}(y=0 \mid x)$	$\widehat{p}(y=1 \mid x)$	prediction
A	0.51	0.49	0
B	0.52	0.48	0
C	0.05	0.95	1

Hard voting predicts class 0 (two votes to one). Soft voting averages $\widehat{p}(y=1 \mid x) = (0.49+0.48+0.95)/3 = 0.64$ and predicts class 1. The soft vote used the fact that A and B were coin flips while C was nearly certain; the hard vote discarded it. Whether that is right depends on whether C's 0.95 deserves to be believed.

Gotcha

Soft voting assumes comparable, reasonably calibrated probabilities under the same prior. A model that outputs 0.95 whenever it leans towards class 1 will dominate every average. Options: calibrate it (Chapter 3), weight the models, or fall back to hard voting, which caps each model at one vote and so is robust to overconfidence at the cost of discarding margins.

How much does averaging help? The variance formula

Averaging M independent errors divides the variance by M. Correlated errors leave a floor.

Fix an input x and let $e_m = \widehat{f}_m(x) - f(x)$ be the error of model m at x, viewed as a random variable over training samples. Suppose every error has the same variance and every pair the same correlation.

Theorem 5.2.3 (Variance of an average of correlated errors)**.** *Let $e_1, \dots, e_M$ have $\mathrm{Var}(e_m) = \sigma^2$ for all m and $\mathrm{Corr}(e_m, e_{m'}) = \rho$ for all $m \neq m'$. Then the error of the average, $\bar{e} = \frac{1}{M}\sum_m e_m$, satisfies*

$$\mathrm{Var}(\bar{e}) = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big).$$

In particular: $\rho = 0$ gives σ^2/M; $\rho = 1$ gives σ^2 (averaging changes nothing); and as $M \to \infty$ the variance tends to $\rho\sigma^2$, not to zero.

Proof. Expand the variance of a sum:

$$\mathrm{Var}\Big(\sum_{m=1}^{M} e_m\Big) = \sum_{m=1}^{M} \mathrm{Var}(e_m) + \sum_{m \neq m'} \mathrm{Cov}(e_m, e_{m'}) = M\sigma^2 + M(M-1)\rho\sigma^2,$$

because there are M variance terms and $M(M-1)$ ordered pairs, each with covariance $\rho\sigma^2$. Dividing by M^2,

$$\begin{aligned}\mathrm{Var}(\bar{e}) &= \frac{M\sigma^2 + M(M-1)\rho\sigma^2}{M^2} = \frac{\sigma^2}{M} + \frac{(M-1)\rho\sigma^2}{M} \\ &= \sigma^2\Big(\rho + \frac{1}{M} - \frac{\rho}{M}\Big) = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big).\end{aligned}$$

The three special cases follow by substitution and by letting $M \to \infty$. □

Key insight

Only the $M(M-1)$ cross terms carry the correlation. The M diagonal terms shrink like $1/M$ after division by M^2; the cross terms do not, because there are M^2 of them. The part of the error that all models *share* is immune to averaging.

Reading the formula

Read ρ as "how much of the error direction is shared". With $\rho = 0.2$ and $M = 25$ trees the variance is $\sigma^2(0.2 + 0.8/25) = 0.232\sigma^2$, not $\sigma^2/25 = 0.04\sigma^2$. Adding more trees moves $0.032\sigma^2$ towards zero but never touches the floor $0.2\sigma^2$. A useful ensemble therefore needs *accurate* components (σ^2 small) whose errors are *sufficiently different* (ρ small). Every later idea in this chapter, bootstrap samples, random feature subsets, sequential corrections, is a way of lowering ρ without raising σ^2 too much.

Interpretation & discussion

Theorem 5.2.3 is about variance only. Averaging does not remove bias: if every model is missing the same important feature, every error has the same sign and ρ is close to 1. This is the formal content of "shared errors do not cancel".

Where does diversity come from when all models see the same data?

Voting and averaging usually fit every model to the *same* training set. Diversity can still come from different learning algorithms (logistic regression, a tree, a nearest-neighbour rule), different settings or random initialisations, or different feature subsets. But because every model sees the same observations, they can all learn the same sample-specific pattern and make correlated errors. The next sections manufacture diversity more deliberately, by changing the *data* each model sees.

Summary: Averaging and voting

- Regression: average predictions. Classification: hard voting (count classes) or soft voting (average posteriors).
- Soft voting uses margins and needs calibrated, comparable probabilities; hard voting is robust to overconfidence but discards information.
- Var $= \sigma^2(\rho + (1-\rho)/M)$: independent errors give σ^2/M; correlation leaves a floor $\rho\sigma^2$ that no number of models removes.

5.3 Decision Trees as Base Learners

What are we trying to get?

Trees are the base learner of almost every ensemble in practice. We want to define a tree precisely as a partition with one constant per region, prove which constant is optimal in a leaf, state the greedy splitting rule with the Gini impurity, work one split by hand, and understand why trees are at once well suited to ensembles (flexible, fast, scale-free) and in need of them (high variance).

A tree is a partition with one number per region

Definition 5.3.1 (Decision tree)**.** A tree with J leaves partitions the feature space into disjoint regions $S_1, \dots, S_J$ and assigns one prediction θ_j to each region:

$$T(x) = \sum_{j=1}^{J} \theta_j \, \mathbf{1}[x \in S_j].$$

Each region is reached by a sequence of binary tests of the form $x_j \le t$ (go left) or $x_j > t$ (go right). The *depth* of a leaf is the number of tests on its path; the *maximum depth* and the *minimum leaf size* (fewest training observations allowed in a leaf) are the usual size controls.

Reading the formula

Exactly one indicator is 1 for any x, so $T(x)$ is simply "the constant of the region x falls in". A tree is a piecewise-constant function whose pieces are axis-aligned boxes. For classification θ_j is a class or a vector of class proportions; for regression it is a number.

Ideally one would choose the regions and their constants *jointly*,

$$\min_{J, \{S_j, \theta_j\}} \sum_{i=1}^{N} \ell\Big(y^{(i)}, \sum_{j=1}^{J} \theta_j \mathbf{1}[x^{(i)} \in S_j]\Big) + \lambda J,$$

with λJ a penalty on the number of leaves. The search over all partitions is infeasible; two simplifications make trees practical. First, for *fixed* regions the best constant in each leaf has a closed form. Second, the regions are chosen *greedily*, one split at a time.

Proposition 5.3.2 (Optimal leaf constant)**.** *For fixed regions the objective separates by leaf, so*

$$\theta_j^* = \arg\min_{\theta} \sum_{i:\, x^{(i)} \in S_j} \ell(y^{(i)}, \theta).$$

For squared-error loss θ_j^* is the mean *of the $y^{(i)}$ in leaf j; for absolute-error loss it is their* median*; for zero–one loss it is the* majority class*.*

Proof. Each training point lies in exactly one leaf, so the sum over i splits into J sums, the j-th of which involves only θ_j; minimise each separately. For squared error, $\frac{d}{d\theta} \sum_i (y^{(i)} - \theta)^2 = -2 \sum_i (y^{(i)} - \theta) = 0$ gives $\theta =$ mean (the finite-sample

version of Theorem 2.2.4). For absolute error the derivative is $\sum_i \operatorname{sign}(\theta - y^{(i)})$, which changes sign at the median. For zero–one loss the leaf should predict the class that is wrong on the fewest points, the majority. □

Greedy recursive splitting and the Gini impurity

Definition 5.3.3 (Impurity; Gini impurity; greedy split)**.** An *impurity* $I(S)$ measures how mixed the observations in a node S are. For classification with class proportions $\widehat{p}_c(S)$ the *Gini impurity* is

$$I(S) = 1 - \sum_{c=1}^{C} \widehat{p}_c(S)^2.$$

For regression the within-node squared error $\sum_{i \in S}(y^{(i)} - \bar{y}_S)^2$ plays the same role. At a node S, a candidate split is a feature j and threshold t giving $S_L = \{i \in S : x_j^{(i)} \le t\}$ and $S_R = S \setminus S_L$. The *greedy rule* chooses the split with the largest impurity reduction,

$$(j^*, t^*) = \arg\max_{j,t} \Big[I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R) \Big],$$

then repeats inside each child until a stopping rule (depth, leaf size, no useful split) is reached.

Reading the formula

$\sum_c \widehat{p}_c^2$ is the probability that two observations drawn from the node have the same class; 1 minus it is the probability they differ. A pure node ($\widehat{p}_c = 1$ for one class) has $I = 0$; two balanced classes give $I = 1 - \frac{1}{4} - \frac{1}{4} = 0.5$, the maximum for $C = 2$. The reduction formula compares the parent's impurity with the *size-weighted* average of the children's: a child that is pure but tiny counts little. Greedy means "best immediate reduction"; it does not look ahead and need not find the globally best tree.

Example 5.3.4 (Choosing a split by hand)**.** Eight observations, sorted by the candidate feature x:

x	1	2	3	4	5	6	7	8
y	0	0	0	1	0	1	1	1

The parent has four of each class, $I(S) = 1 - (\frac{1}{2})^2 - (\frac{1}{2})^2 = 0.50$.

- $x \le 3.5$: left $\{0,0,0\}$, $I = 0$; right $\{1,0,1,1,1\}$, $I = 1 - 0.8^2 - 0.2^2 = 0.32$. Weighted: $\frac{3}{8} \cdot 0 + \frac{5}{8} \cdot 0.32 = 0.20$. Reduction $0.50 - 0.20 = 0.30$.
- $x \le 4.5$: left $\{0,0,0,1\}$, $I = 1 - 0.75^2 - 0.25^2 = 0.375$; right $\{0,1,1,1\}$, $I = 0.375$. Weighted 0.375. Reduction 0.125.

The greedy rule prefers $x \le 3.5$: it gives a pure left leaf and the larger immediate reduction, even though $x \le 4.5$ splits the data in half.

Why trees in ensembles

Flexible and cheap, but one early split decides everything below it.

Trees capture nonlinear effects and feature interactions (a test on x_1 followed by a test on x_2 *is* an interaction), need no scaling of numerical features (only the order of values matters), fit complex patterns when grown deep, and are fast enough to fit hundreds of times. Their weakness is the one that averaging cures: a small change in the data can alter an early split, and everything below that split is rebuilt. Deep trees are therefore high-variance learners. Bootstrap samples make different trees; averaging makes the prediction stable while preserving the trees' ability to model nonlinearity and interactions.

Summary: Trees

- A tree is a partition into boxes with one constant per box; the optimal constant is the leaf mean (squared error), median (absolute error) or majority class.
- Splits are chosen greedily by impurity reduction; Gini $I = 1 - \sum_c \widehat{p}_c^2$ for classification, squared error for regression.
- Trees are flexible, scale-free and fast, but deep trees have high variance, which is exactly what ensembles reduce.

5.4 Bagging and Out-of-Bag Evaluation

What are we trying to get?

We want a way to give the *same* learner *different* training sets without throwing data away. Bootstrap sampling does this; we prove that each bootstrap sample omits about 36.8% of the observations, and we turn those omitted observations into a free, honest validation set: the out-of-bag estimate.

Bootstrap samples

Splitting $\mathcal{D}$ into M disjoint parts would give each model only N/M observations, too few unless N is huge. Instead each model is trained on a *perturbed copy* of the whole dataset.

Definition 5.4.1 (Bootstrap sample; bagging)**.** A *bootstrap sample* $\mathcal{D}^*$ of a training set $\mathcal{D}$ with N observations consists of N draws from $\mathcal{D}$ *with replacement*. Let $\text{count}(i)$ be the number of times observation i appears in $\mathcal{D}^*$; observations with $\text{count}(i) = 0$ are *out of bag* for that sample. *Bagging* (bootstrap aggregating) draws M bootstrap samples $\mathcal{D}_1^*, \dots, \mathcal{D}_M^*$, fits the same base learner to each, and aggregates:

$$\widehat{f}_{\text{bag}}(x) = \frac{1}{M}\sum_{m=1}^{M} \widehat{f}_m(x), \qquad \widehat{p}_{\text{bag}}(y = c \mid x) = \frac{1}{M}\sum_{m=1}^{M} \widehat{p}_m(y = c \mid x).$$

Example 5.4.2 (One bootstrap sample)**.** From $\mathcal{D} = \{1, \dots, 8\}$, eight draws with replacement might give $\mathcal{D}_1^* = \{2, 5, 2, 8, 1, 5, 7, 2\}$. Then

observation i	1	2	3	4	5	6	7	8
$\text{count}(i)$	1	3	0	0	2	0	1	1

Observation 2 is seen three times and 5 twice; 3, 4 and 6 are out of bag.

Different count patterns give different empirical distributions and so different fitted models.

Theorem 5.4.3 (Out-of-bag fraction)**.** *The probability that a given observation is absent from a bootstrap sample of size N is*

$$\mathbb{P}(\text{count}(i) = 0) = \left(1 - \frac{1}{N}\right)^N \xrightarrow[N\to\infty]{} e^{-1} \approx 0.368.$$

Hence a large bootstrap sample contains about $1 - e^{-1} \approx 63.2\%$ of the distinct *training observations, and about 36.8% are out of bag for each fitted model.*

Proof. Each draw picks observation i with probability $1/N$, so misses it with probability $1 - 1/N$. The N draws are independent, so the probability of missing i every time is $(1 - 1/N)^N$. The limit is the standard $\lim_{N\to\infty}(1 + x/N)^N = e^x$ with $x = -1$. Already for $N = 8$ the value is $(7/8)^8 = 0.344$; for $N = 100$ it is 0.366. □

Reading the formula

Two thirds in, one third out, for *every* model, and a *different* third each time. The "in" part is what makes the models different; the "out" part is what the next definition exploits.

Out-of-bag prediction and error

Definition 5.4.4 (Out-of-bag prediction)**.** For training observation i, let $\mathcal{M}_i = \{m : i \notin \mathcal{D}_m^*\}$ be the set of models whose bootstrap sample omits i; by Theorem 5.4.3, $|\mathcal{M}_i| \approx 0.368M$. The *out-of-bag (OOB) prediction* uses only those models,

$$\widehat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\mathcal{M}_i|} \sum_{m \in \mathcal{M}_i} \widehat{f}_m(x^{(i)}),$$

or, for classification, the average of their probabilities (or their vote). The *OOB error* is the loss of the OOB predictions over all training observations.

Reading the formula

For observation i, the OOB models never saw i, so their prediction at $x^{(i)}$ is a genuine unseen-data prediction, even though i is in the training set. Doing this for every i gives a validation estimate *without* holding anything out: the bootstrap already held out a third of the data for each model. It is a slight underestimate of the full ensemble's quality, since only about a third of the trees vote for each point.

Interpretation & discussion

The OOB error estimates the generalisation error of one fixed ensemble and can compare *ensemble-wide* choices: base-learner complexity, number of models, feature set. Two cautions. Repeated OOB-based choices overfit the OOB estimate just as repeated validation overfits a validation set (Chapter 3, Def-

inition 3.5.5): the final performance still needs an untouched test set. And the OOB estimate is tied to the training distribution; it says nothing about deployment shift. Bootstrap sampling provides internal validation cases, not a final test set.

One tree-size setting for the whole ensemble

Technique: Tune the ensemble, not the trees

Treat the tree-size setting ξ (maximum depth, minimum leaf size) as an *ensemble* hyperparameter: $\mathcal{D}_m^* \to \widehat{T}_m(x;\xi)$ with the *same* ξ for every m. For each candidate ξ, fit a complete bagged ensemble and compute its OOB error (or a validation error); pick the best ξ. Bagging usually uses deep trees, since averaging removes their variance; with noisy or limited data the OOB error may favour smaller trees or larger leaves. Tuning each tree separately would optimise the wrong target (a single tree's accuracy rather than the average's) and would push every tree towards the same strong, correlated fit.

Summary: Bagging

- Bootstrap: N draws with replacement; $(1 - 1/N)^N \to e^{-1}$, so about 63.2% distinct observations in, 36.8% out, for each model.
- Bagging fits the same learner to M bootstrap samples and averages or votes.
- OOB prediction for i uses only the $\approx 0.368M$ models that never saw i; the OOB error is a free internal validation estimate, but not a substitute for a test set.
- Tree size is one ensemble-level hyperparameter chosen by OOB or validation error.

5.5 Random Forests

What are we trying to get?

Bagged trees can still be too alike: a very strong feature ends up at the root of most of them and their errors stay correlated. We want the one extra randomisation that fixes this, its main hyperparameter m_{try}, and the resulting trade-off between tree strength and tree correlation in the light of Theorem 5.2.3.

Bagged trees can remain too similar

If one feature is very strong, almost every bootstrap sample will choose it near the root, and the trees that result have similar shapes and make similar mistakes. In Theorem 5.2.3 that is a large ρ: more trees stop helping early. The remedy is to *randomise the candidate features* at each split so that different trees are forced to start differently.

If x_1 is the best split everywhere, every bootstrap tree starts with x_1.

Definition 5.5.1 (Random forest)**.** For each tree $m = 1, \dots, M$:

1. draw a bootstrap sample $\mathcal{D}_m^*$;
2. at *each node*, select m_{try} of the d features uniformly at random;
3. choose the best split (Definition 5.3.3) using only those candidates;
4. grow the tree subject to leaf-size or depth constraints.

Aggregate the M trees by averaging probabilities or voting. A common starting value for classification is $m_{\text{try}} \approx \sqrt{d}$ (regression implementations often use $d/3$); with $d = 16$ each node receives four random candidates. A *new* subset is drawn at every node.

Reading the formula

Step 2 is the only difference from bagging. Setting $m_{\text{try}} = d$ recovers bagged trees. Small m_{try} means that at many nodes the strongest feature is simply not on offer, so an alternative is used, the trees take different shapes, and their errors decorrelate. The strong feature is not lost: it is available at a fraction m_{try}/d of the nodes and will be used deeper down.

Example 5.5.2 (Probability that a feature is offered)**.** With $d = 25$ and $m_{\text{try}} = 5$, a given feature is among the candidates at a node with probability $5/25 = 0.2$ (choosing 5 of 25 without replacement, the chance that a particular one is included is $\binom{24}{4}/\binom{25}{5} = 5/25$). It is *absent* from the root of one tree with probability 0.8, and from the roots of all $M = 10$ trees with probability $0.8^{10} \approx 0.107$. So in a ten-tree forest about 2 roots use it, while a bagged forest might use it at all 10.

Strength versus correlation

Large m_{try}	Small m_{try}
strongest features reused at most nodes	alternative features get considered
individual trees strong (small σ^2)	individual trees weaker (larger σ^2)
trees similar, errors correlated (large ρ)	trees less correlated (small ρ)
	too few candidates can give a poor split

Interpretation & discussion

Theorem 5.2.3 says the forest variance is $\sigma^2(\rho + (1-\rho)/M)$. Feature randomisation raises σ^2 a little and lowers ρ a lot; for moderate M the product improves. This is why a forest of individually *weaker* trees can beat a forest of individually *stronger* ones: the target is the average's error, not any tree's.

Controls

Hyperparameter	Main effect
number of trees M	stability and computation; OOB error decreases then plateaus; more trees stabilise rather than deepen the model
m_{try}	tree strength versus correlation; start near $\sqrt{d}$ for classification
minimum leaf size	prevents very small terminal regions
maximum depth	limits the number of successive splits

Technique: Forest-level selection

Tune m_{try}, minimum leaf size and, if useful, maximum depth by OOB performance or a separate validation set; each candidate setting requires fitting a complete forest. Increase the number of trees until the OOB error stabilises; beyond that, additional trees cost computation without changing the model's capacity.

Summary: Random forests

- Bagging plus a random subset of m_{try} candidate features at every node; $m_{\text{try}} = d$ is bagging.
- Feature randomisation weakens each tree slightly but decorrelates the trees; the forest improves because ρ falls faster than σ^2 rises.
- Start at $m_{\text{try}} \approx \sqrt{d}$; tune at forest level by OOB error; add trees until the OOB error plateaus.

5.6 Feature Importance

What are we trying to get?

A forest of hundreds of trees is not readable. We want two numerical summaries of *which features the fitted forest relies on*: one computed from the training process (mean decrease in impurity) and one computed by breaking a feature and watching the error (permutation importance), with the biases of each and the one interpretation both forbid.

Mean decrease in impurity

Definition 5.6.1 (Split credit; MDI)**.** For a node t containing N_t of the N training observations, split into children of sizes N_L, N_R, the *credit* of the split is its impurity reduction weighted by the fraction of data it acts on,

$$\Delta_t = \frac{N_t}{N}\Big[I(S_t) - \frac{N_L}{N_t} I(S_L) - \frac{N_R}{N_t} I(S_R)\Big].$$

The credit goes to the feature used at t. The *mean decrease in impurity* of feature j adds all credits of splits on x_j and averages across trees,

$$\text{MDI}_j = \frac{1}{M} \sum_{m=1}^{M} \sum_{t \in T_m:\ \text{split}(t)=j} \Delta_t,$$

usually normalised so that $\sum_j \text{MDI}_j = 1$.

Reading the formula

The bracket is the impurity reduction of Definition 5.3.3. The factor N_t/N says: a reduction achieved on all the data is worth more than the same reduction on 5% of it. The root has $N_t/N = 1$; deeper nodes have smaller weights but can still earn more credit through a larger reduction. The normalised

MDI is a share: "this feature produced 75% of all the impurity the forest removed".

Example 5.6.2 (Credit earned by one split)**.** A root node holds 10 flows, 5 attacks and 5 benign, Gini 0.50. The test "destination port = 22?" sends 4 flows left (0 attacks, 4 benign, Gini 0) and 6 right (5 attacks, 1 benign, Gini $1-(5/6)^2-(1/6)^2 = 10/36 = 0.278$). Weighted child impurity: $\frac{4}{10}(0) + \frac{6}{10}(0.278) = 0.167$. Since $N_t/N = 1$, $\Delta_t = 1 \cdot (0.50 - 0.167) = 0.333$, credited to destination port.

Example 5.6.3 (From credits to forest-level MDI)**.** Two trees produce the following credits. Tree 1: destination port 0.18, flow duration 0.06. Tree 2: destination port 0.12, total packets 0.04. Averaging over the two trees (a feature absent from a tree contributes 0 there): port $(0.18 + 0.12)/2 = 0.15$, duration $0.06/2 = 0.03$, packets $0.04/2 = 0.02$. The means sum to 0.20; dividing gives MDI = 0.75, 0.15, 0.10. Destination port produced the largest total impurity reduction.

Gotcha

MDI is free (it is computed during training) but biased: a feature with many possible split points (a continuous feature, a high-cardinality categorical) has more chances to find *some* reduction, by signal or by noise, and tends to be over-credited. MDI also measures the *training* fit, not generalisation.

Permutation importance

Definition 5.6.4 (Permutation importance)**.** For a fitted forest and a feature j:

1. compute the original OOB (or validation) error $\widehat{\mathrm{Err}}_{\mathrm{original}}$;
2. shuffle the values of x_j among the OOB observations, keeping every other column and the labels fixed;
3. predict again and measure $\widehat{\mathrm{Err}}_{\mathrm{shuffled}(j)}$.

The importance is the increase, $I_j = \widehat{\mathrm{Err}}_{\mathrm{shuffled}(j)} - \widehat{\mathrm{Err}}_{\mathrm{original}}$. No tree is retrained. In a forest each tree has its own OOB sample and its own shuffled copy; the increases are averaged across trees.

Reading the formula

Shuffling destroys the *information* in x_j (its link with the label and the other features) while preserving its marginal distribution, so the trees still receive plausible values and are not thrown off by out-of-range inputs. If the error barely moves, the forest was not relying on x_j; if it jumps, it was. Because this uses OOB or validation data and the fitted model as is, it measures reliance in *prediction*, not in training.

Example 5.6.5 (Shuffling destination port)**.** Four OOB flows with ports $80, 22, 443, 53$ become, after one shuffle, $53, 443, 80, 22$; duration, packets and

labels do not move. On 100 OOB flows the forest originally makes 10 errors (rate 0.10). Shuffling destination port gives 18 errors (0.18, increase 0.08); flow duration 13 (0.13, increase 0.03); total packets 11 (0.11, increase 0.01). Destination port matters most *for this fitted forest*: breaking it adds eight errors.

Gotcha

Two readings that importance does *not* support.

- **Correlated features mask one another.** If x_a and x_b carry the same information, shuffling x_a alone hurts little because the trees that use x_b (and the forest as a whole) still have the information. Both features then look less important than the information they share. Permute them jointly, or drop a column and refit, to see the shared contribution.
- **Importance is not causality.** A high score means the fitted model relies on the feature. The feature may be a proxy for the real cause (destination port stands in for the service being attacked), a consequence of the outcome recorded after the fact (leakage, Chapter 3), or an artefact of how the data were collected. Intervening on it would not change the outcome.

Summary: Feature importance

- MDI: sum of $\frac{N_t}{N} \times$(impurity reduction) over a feature's splits, averaged over trees, normalised; fast, training-based, biased towards features with many split points.
- Permutation importance: error increase on OOB/validation data after shuffling one feature; no retraining; measures reliance of the fitted model.
- Correlated features mask each other; importance does not imply causality.

5.7 Boosting for Regression

What are we trying to get?

Bagging fits many models *independently* and averages. Boosting asks a different question: *what should the next model learn from the current mistakes?* We want the additive model this leads to, a proof that for squared error "fit the next tree to the residuals" is exactly minimising the loss, the gradient-boosting algorithm, one iteration by hand, and the three hyperparameters that jointly control overfitting.

A chain of specialists

Tree 1 gets the trend. Tree 2 fixes what tree 1 got wrong. Tree 3 fixes what is still wrong.

The observations stay fixed. A first learner makes a baseline prediction, right in general but wrong in places. Compute the *leftovers*, the residuals $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$. Train a second learner exclusively to predict those residuals; its job is to correct the first. A third learner is trained on what remains after the first two, and so on. Each added learner changes the ensemble's prediction where the current

errors are, and nowhere else. Bagging builds models in parallel; boosting adds corrections sequentially.

Definition 5.7.1 (Additive boosted model)**.** A boosted ensemble after m rounds is

$$F_m(x) = F_{m-1}(x) + \eta\, T_m(x),$$

where T_m is the new tree fitted in round m and $\eta \in (0, 1]$ is the *learning rate* (shrinkage). Unrolled, $F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} T_m(x)$: a constant plus a scaled *sum* of trees.

Reading the formula

Compare with bagging's $\frac{1}{M}\sum_m \widehat{f}_m$: there, every model predicts the *whole* target and the average smooths them; here, every tree predicts a *correction* and the corrections are summed. Averaging reduces variance; summing corrections reduces the remaining loss (mostly bias) one step at a time. The learning rate η scales each step down; $\eta = 1$ applies the full correction, $\eta = 0.1$ one tenth of it.

Theorem 5.7.2 (Squared error: boosting fits residuals)**.** *With squared-error loss* $L = \frac{1}{2}\sum_{i=1}^{N} \left(y^{(i)} - F_m(x^{(i)})\right)^2$ *and the update* $F_m = F_{m-1} + \eta T_m$*,*

$$L = \frac{1}{2}\sum_{i=1}^{N} \Big(\underbrace{y^{(i)} - F_{m-1}(x^{(i)})}_{r_m^{(i)}} - \eta T_m(x^{(i)}) \Big)^2,$$

so minimising L *over* T_m *is the same as fitting* ηT_m *to the current residuals* $r_m^{(i)}$ *by least squares. In practice* T_m *is fitted to* $r_m^{(i)}$ *and* η *is applied afterwards in the update.*

Proof. Substitute $F_m(x^{(i)}) = F_{m-1}(x^{(i)}) + \eta T_m(x^{(i)})$ into L and group $y^{(i)} - F_{m-1}(x^{(i)})$, which is a fixed number at round m because F_{m-1} is already fitted. The expression is then the least-squares objective of a regression whose *targets* are the residuals and whose *predictor* is ηT_m. □

Key insight

At round m everything built so far is frozen. The only moving part is T_m, and the loss depends on it only through "residual minus correction". The next tree therefore sees a plain regression problem whose targets are the current errors.

The gradient boosting algorithm for regression

Definition 5.7.3 (Gradient boosting for regression)**.**

1. Initialise with the best constant, $F_0(x) = \arg\min_w \sum_i \ell(y^{(i)}, w)$; for squared error this is the mean $\bar{y}$ (Proposition 5.3.2).
2. For $m = 1, \ldots, M$:
   (a) compute the pseudo-residuals $r_m^{(i)} = -\partial \ell(y^{(i)}, F)/\partial F\big|_{F=F_{m-1}(x^{(i)})}$; for squared error this is the raw error $r_m^{(i)} = y^{(i)} - F_{m-1}(x^{(i)})$;

(b) fit a regression tree T_m to the targets $r_m^{(i)}$, creating leaf regions $S_{1,m}, \dots, S_{Q_m,m}$;

(c) in each leaf, set the output to the mean residual, $w_{q,m} = \frac{1}{|S_{q,m}|} \sum_{i \in S_{q,m}} r_m^{(i)}$;

(d) update $F_m(x) = F_{m-1}(x) + \eta T_m(x)$, where $T_m(x) = \sum_q w_{q,m} \mathbf{1}[x \in S_{q,m}]$.

Reading the formula

The word *gradient*: $-\partial \ell / \partial F$ is the direction in which changing the prediction at $x^{(i)}$ decreases the loss fastest. For squared error $\ell = \frac{1}{2}(y - F)^2$ the negative derivative is $y - F$, the residual, which is why "fit the residuals" and "fit the negative gradient" coincide here. For other losses they differ, and the general recipe is: fit the tree to the negative gradient, then choose each leaf's output to minimise the actual loss (§5.8).

Example 5.7.4 (One boosting round by hand)**.** Three observations with learning rate $\eta = 0.1$:

ID	x	y	F_0	$r_1 = y - F_0$
1	1.2	10	20	-10
2	4.5	20	20	0
3	7.1	30	20	$+10$

Initialise: $F_0 = \text{mean}(10, 20, 30) = 20$. *Residuals:* as tabulated. *Fit T_1:* a split at $x < 2.85$ puts ID 1 in leaf S_1 and IDs 2, 3 in leaf S_2. *Leaf weights:* $w_{1,1} = -10$, $w_{2,1} = (0 + 10)/2 = +5$. *Update:* for ID 3, $F_1(7.1) = 20 + 0.1 \cdot 5 = 20.5$, closer to 30; for ID 1, $F_1(1.2) = 20 - 0.1 \cdot 10 = 19$. The new residuals are $-9, -0.5, 9.5$; the next tree is fitted to those.

Why many shallow trees rather than one deep tree

One deep tree	Boosted shallow trees
fits the whole rule at once	tree 1 isolates the loudest pattern; later trees target leftovers
hierarchical lock-in: early splits force rigid boundaries, deep splits only refine the sub-region they sit in	*global adaptability*: each new tree may adjust the boundary anywhere in feature space
a poor early split cannot be revised and easily amplifies noise	shrinkage η enforces gradual convergence

Instead of committing to an architecture in one step, boosting builds a flexible decision surface incrementally. This is the bias–variance trade-off of Chapter 3 in a new guise: each shallow tree has high bias and low variance, and the sum of many of them lowers the bias while η and early stopping keep the variance in check.

Overfitting and the interconnected triad

Boosting can continue indefinitely and the training loss keeps decreasing; after some number of rounds the validation loss starts rising because new trees fit noise.

Training loss keeps falling forever; validation loss turns up. Stop there.

Early stopping (Definition 3.4.5) selects the number of rounds at the validation minimum. Three hyperparameters control capacity *jointly*:

- **Number of rounds M.** Too small underfits; too large overfits.
- **Learning rate η.** Smaller steps need more rounds to reach the same fit: roughly $\eta \propto 1/M$. Smaller η usually generalises slightly better at a higher computational cost.
- **Tree depth or maximum leaves Q.** Limits the order of feature interactions a tree can express (depth 1: additive effects only; depth 2: pairwise interactions). Shallow trees, depth 2 to 6, usually generalise best.

Technique: Do not tune in isolation

The three are coupled: halving η shifts the optimal M up; deeper trees reach the validation minimum in fewer rounds. Use a grid or randomised search over (η, Q), with M chosen by early stopping on validation or by cross-validation (Definition 3.5.5) for each candidate, and compare configurations by their validation loss at the selected round.

Summary: Boosting for regression

- $F_m = F_{m-1} + \eta T_m$: a sum of corrections, not an average of predictions.
- For squared error, minimising the loss over T_m is least-squares fitting of the residuals; the pseudo-residual is the negative gradient of the loss.
- Initialise at the mean; each leaf outputs the mean residual; scale by η.
- Many shallow trees avoid the lock-in of one deep tree; M, η, depth are tuned together with early stopping.

5.8 Boosting for Binary Classification

What are we trying to get?

Probabilities live in $(0, 1)$ and cannot be summed with tree outputs. We want the fix (boost the *logit*), the gradient and Hessian of the cross-entropy loss with respect to the logit (proved), why the leaf output no longer has a closed form, and the second-order (Newton) shortcut that restores one, first without and then with regularisation.

Boost in logit space

Definition 5.8.1 (Boosting the logit; binary cross-entropy)**.** The ensemble produces an unconstrained score $F^{(i)} \in (-\infty, \infty)$ for observation i and maps it to a probability through the sigmoid (Definition 4.2.1),

$$\widehat{p}^{(i)} = \sigma(F^{(i)}) = \frac{1}{1 + e^{-F^{(i)}}}.$$

The loss is the binary cross-entropy (Definition 2.5.8) written as a function of the logit,

$$\ell^{(i)}(y^{(i)}, F^{(i)}) = -\Big[y^{(i)} \ln \widehat{p}^{(i)} + (1 - y^{(i)}) \ln(1 - \widehat{p}^{(i)})\Big].$$

Trees are added to F: $F_m = F_{m-1} + \eta T_m$, and $\widehat{p}$ is recomputed from F_m.

Reading the formula

Adding a tree output to a probability could give 1.3. Adding it to a logit is always legal, and the sigmoid brings the result back inside $(0, 1)$. This is exactly the discriminative route of Chapter 4, with the score $s_\theta(x)$ now a sum of trees instead of $w^\top x + b$.

Theorem 5.8.2 (Gradient and Hessian of the cross-entropy in the logit)**.** *With $\widehat{p} = \sigma(F)$,*

$$g = \frac{\partial \ell}{\partial F} = \widehat{p} - y, \qquad h = \frac{\partial^2 \ell}{\partial F^2} = \widehat{p}(1 - \widehat{p}).$$

Proof. First, $\sigma'(F) = \sigma(F)(1 - \sigma(F))$: writing $\sigma = (1 + e^{-F})^{-1}$, $\sigma' = e^{-F}(1 + e^{-F})^{-2} = \sigma \cdot \frac{e^{-F}}{1+e^{-F}} = \sigma(1 - \sigma)$. Then by the chain rule

$$\begin{aligned} \frac{\partial \ell}{\partial F} = \frac{\partial \ell}{\partial \widehat{p}} \frac{d\widehat{p}}{dF} &= \Big(-\frac{y}{\widehat{p}} + \frac{1-y}{1-\widehat{p}} \Big) \widehat{p}(1 - \widehat{p}) \\ &= -y(1 - \widehat{p}) + (1 - y)\widehat{p} = -y + y\widehat{p} + \widehat{p} - y\widehat{p} = \widehat{p} - y. \end{aligned}$$

Differentiating once more, $\partial(\widehat{p} - y)/\partial F = d\widehat{p}/dF = \widehat{p}(1 - \widehat{p})$. □

Reading the formula

The *negative* gradient $-g = y - \widehat{p}$ is the classification analogue of the residual: how far the current probability is from the label. The Hessian $h = \widehat{p}(1 - \widehat{p})$ is the variance of a Bernoulli with parameter $\widehat{p}$: it peaks at 0.25 when $\widehat{p} = 0.5$ (the model is unsure, the loss curves most) and vanishes when $\widehat{p} \to 0$ or 1 (confident, loss flat in F). For $y = 1$ and a very negative F (confidently wrong), $-g \to 1$ is at its largest while $h \to 0$: a large residual with almost no curvature. Keep this picture; it explains both the Newton step and why it needs regularisation.

Basic gradient boosting and the line-search bottleneck

Definition 5.8.3 (Gradient boosting for classification)**.** At round m:

1. pseudo-residuals $r_m^{(i)} = -\partial \ell^{(i)}/\partial F_{m-1}^{(i)} = y^{(i)} - \widehat{p}_{m-1}^{(i)}$;
2. grow a regression tree on $r_m^{(i)}$ by squared-error splits, giving leaves $S_{1,m}, \dots, S_{Q_m,m}$ (*split search determines the regions*);
3. with the structure fixed, choose each leaf output by a *line search* on the true loss,
$$w_{q,m} = \arg\min_{w} \sum_{i \in S_{q,m}} \ell^{(i)}(y^{(i)}, F_{m-1}^{(i)} + w)$$
(*line search determines the outputs*);
4. $T_m(x) = \sum_q w_{q,m} \mathbf{1}[x \in S_{q,m}]$ and $F_m = F_{m-1} + \eta T_m$.

For squared error step 3 had the closed form "mean residual". For log loss it has none: $\sum_i \ell(y^{(i)}, F^{(i)} + w)$ is a sum of log-sigmoids in w and must be minimised

numerically, leaf by leaf, with repeated loss evaluations. This is the *line-search bottleneck*. The remedy is to replace the loss by a local quadratic.

The Newton step: a quadratic approximation gives a closed form

Theorem 5.8.4 (Newton leaf value)**.** *Expand the loss of observation i to second order around the current logit,*

$$\ell^{(i)}(y^{(i)}, F_{m-1}^{(i)} + w) \;\approx\; \ell^{(i)}(y^{(i)}, F_{m-1}^{(i)}) + g^{(i)} w + \tfrac{1}{2} h^{(i)} w^2.$$

Minimising the sum of these approximations over a leaf $S_{q,m}$ gives

$$w_{q,m} = \frac{\sum_{i \in S_{q,m}} (-g^{(i)})}{\sum_{i \in S_{q,m}} h^{(i)}} = \frac{\sum_{i \in S_{q,m}} (y^{(i)} - \widehat{p}_{m-1}^{(i)})}{\sum_{i \in S_{q,m}} \widehat{p}_{m-1}^{(i)} (1 - \widehat{p}_{m-1}^{(i)})}.$$

Adding an L_2 penalty $\frac{\lambda}{2} w^2$ to the leaf objective gives the regularised *value*

$$w_{q,m} = \frac{\sum_{i \in S_{q,m}} (-g^{(i)})}{\sum_{i \in S_{q,m}} h^{(i)} + \lambda}.$$

Proof. The sum over the leaf, $\sum_i \left[\ell^{(i)} + g^{(i)} w + \frac{1}{2} h^{(i)} w^2\right] + \frac{\lambda}{2} w^2$, is a quadratic in the single number w with positive leading coefficient $\frac{1}{2}(\sum_i h^{(i)} + \lambda)$, since $h^{(i)} > 0$. Its derivative $\sum_i g^{(i)} + (\sum_i h^{(i)} + \lambda) w$ vanishes at $w = -\sum_i g^{(i)} / (\sum_i h^{(i)} + \lambda)$. Setting $\lambda = 0$ gives the unregularised step; substituting Theorem 5.8.2 gives the explicit form. □

Reading the formula

Numerator: the total residual in the leaf, how far and in which direction the probabilities are off. Denominator: the total curvature, how fast the loss bends. A Newton step is "residual divided by curvature": in a leaf full of unsure observations ($h \approx 0.25$ each) the step is moderate; in a leaf of confidently-wrong observations ($h \approx 0$) the denominator collapses and the step *explodes*. That is the risk: small leaves with tiny $\sum h$ give enormous outputs and extreme overfitting. The λ in the denominator is a floor that caps the step; it matters most exactly where $\sum h$ is smallest.

Example 5.8.5 (Newton leaf value by hand)**.** A leaf holds $(y, \widehat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$. Gradients $g = \widehat{p} - y$: $-0.8, -0.6, +0.8$, so $G = \sum g = -0.6$. Hessians $h = \widehat{p}(1 - \widehat{p})$: $0.16, 0.24, 0.16$, so $H = 0.56$. Unregularised leaf value $-G/H = 0.6/0.56 = 1.071$; with $\lambda = 1$, $0.6/1.56 = 0.385$. With $\eta = 0.1$ the logit actually moves by 0.0385 (regularised). The positive sign raises all three probabilities, helping the two positives more than it hurts the negative.

Interpretation & discussion

Both GBM variants still build the *tree structure* by unweighted squared-error splits on the raw residuals $y - \widehat{p}$, and only afterwards compute Newton outputs and apply λ to the frozen leaves. The split search is blind to the curvature information that the leaf values use. XGBoost's contribution (§5.9) is to use one objective for both.

Summary: Boosting for classification

- Boost the logit F; $\widehat{p} = \sigma(F)$; loss is binary cross-entropy.
- $g = \widehat{p} - y$ (so the pseudo-residual is $y - \widehat{p}$); $h = \widehat{p}(1-\widehat{p})$, maximal at $\widehat{p} = 0.5$, zero when confident.
- Exact leaf outputs need a numerical line search; a second-order expansion gives the Newton step $w = \sum(-g)/\sum h$.
- Small $\sum h$ makes the step explode; the regularised step $\sum(-g)/(\sum h+\lambda)$ caps it.

5.9 XGBoost: One Objective for Structure and Leaf Values

What are we trying to get?

We want the single regularised second-order objective that XGBoost uses to evaluate a *whole tree*, to show (by completing the square) that it is a weighted least-squares problem with a new target and a new weight, to derive the optimal leaf value and the *gain* of a split, and to see precisely what the two penalties λ and γ do.

The objective

Definition 5.9.1 (XGBoost tree objective)**.** A proposed tree with leaves $S_{1,m}, \dots, S_{Q_m,m}$ and outputs $w_{q,m}$ is scored by

$$\mathcal{L}_m = \sum_{q=1}^{Q_m} \sum_{i:\, x^{(i)} \in S_{q,m}} \left[g^{(i)} w_{q,m} + \tfrac{1}{2} h^{(i)} w_{q,m}^2 \right] + \gamma Q_m + \frac{\lambda}{2} \sum_{q=1}^{Q_m} w_{q,m}^2,$$

with $g^{(i)}, h^{(i)}$ the first and second derivatives of the loss at the current prediction $F_{m-1}(x^{(i)})$.

Reading the formula

Three parts. The double sum is the second-order Taylor approximation of the total loss change produced by the tree (the constant $\sum_i \ell^{(i)}$ is dropped since it does not depend on the tree). γQ_m charges γ per leaf. $\frac{\lambda}{2}\sum_q w_q^2$ charges for large outputs. The *same* number scores the structure (through which observations share a leaf) and the outputs; XGBoost therefore uses gradients, Hessians and regularisation *while growing* the tree, not after.

Theorem 5.9.2 (The objective is a weighted regression)**.** *For observation i in leaf q,*

$$g^{(i)} w_q + \tfrac{1}{2} h^{(i)} w_q^2 = \tfrac{1}{2} h^{(i)} \left(r_{\mathrm{XGB}}^{(i)} - w_q \right)^2 + \mathit{const}, \quad r_{\mathrm{XGB}}^{(i)} = -\frac{g^{(i)}}{h^{(i)}} = \frac{y^{(i)} - \widehat{p}_{m-1}^{(i)}}{\widehat{p}_{m-1}^{(i)}(1 - \widehat{p}_{m-1}^{(i)})}.$$

Hence

$$\mathcal{L}_m = \sum_q \sum_{i \in S_{q,m}} \tfrac{1}{2} h^{(i)} \big(r_{\mathrm{XGB}}^{(i)} - w_{q,m}\big)^2 + \gamma Q_m + \frac{\lambda}{2} \sum_q w_{q,m}^2 + \textit{constant.}$$

Proof. Complete the square: $\frac{1}{2}h(w+g/h)^2 = \frac{1}{2}hw^2+gw+g^2/(2h)$, so $gw+\frac{1}{2}hw^2 = \frac{1}{2}h(w-(-g/h))^2 - g^2/(2h)$, and $-g^2/(2h)$ does not involve w. Substituting $g = \widehat{p} - y$ and $h = \widehat{p}(1-\widehat{p})$ gives the explicit target. □

Reading the formula

Each observation asks the tree to output $r_{\mathrm{XGB}} = -g/h$, the one-step Newton correction *for that observation alone*, and its request is weighted by h. An unsure observation (h large) has a modest target and a loud voice; a confidently-wrong one has a huge target ($(y-\widehat{p})/(\widehat{p}(1-\widehat{p})) \approx 1/\widehat{p}$ when $y=1$, $\widehat{p} \to 0$) but a whisper of a weight. Target and weight must always be read together: their product, $h \cdot r_{\mathrm{XGB}} = -g = y - \widehat{p}$, is bounded by 1, which is why the leaf value stays finite. The tree is grown by *weighted* squared-error splits on these targets, which is what "XGBoost uses the Hessian in the split search" means.

Leaf values and the gain of a split

Theorem 5.9.3 (Optimal leaf value and split gain)**.** *For a leaf q let $G_q = \sum_{i \in S_q} g^{(i)}$ and $H_q = \sum_{i \in S_q} h^{(i)}$. The leaf's contribution to $\mathcal{L}_m$ is $\phi_q(w) = G_q w + \frac{1}{2}(H_q+\lambda)w^2$, minimised at*

$$w_q^* = -\frac{G_q}{H_q + \lambda}, \qquad \phi_q(w_q^*) = -\frac{1}{2}\,\frac{G_q^2}{H_q+\lambda}.$$

Splitting a parent with (G,H) into children (G_L,H_L) and (G_R,H_R), where $G = G_L + G_R$ and $H = H_L + H_R$, changes the objective by $-$gain, with

$$\text{gain} = \frac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\Big] - \gamma.$$

The split is accepted only when gain > 0. *After the tree is grown, $F_m = F_{m-1} + \eta T_m$.*

Proof. Summing the per-observation terms over a leaf gives $\sum_i g^{(i)} w + \frac{1}{2}\sum_i h^{(i)} w^2 + \frac{\lambda}{2}w^2 = G_q w + \frac{1}{2}(H_q+\lambda)w^2$. Its derivative $G_q + (H_q+\lambda)w$ vanishes at w_q^*; substituting, $\phi_q(w_q^*) = -G_q^2/(H_q+\lambda) + \frac{1}{2}G_q^2/(H_q+\lambda) = -\frac{1}{2}G_q^2/(H_q+\lambda)$. Replacing one leaf by two changes the data-fit part from $\phi_{\mathrm{parent}}(w^*)$ to $\phi_L(w_L^*) + \phi_R(w_R^*)$, a decrease of $\frac{1}{2}[G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - G^2/(H+\lambda)]$, and raises the leaf count from Q_m to $Q_m + 1$, so the penalty rises by $\gamma(Q_m+1) - \gamma Q_m = \gamma$. □

Reading the formula

$-\frac{1}{2}G^2/(H+\lambda)$ is "how much loss this leaf can remove with its best output"; more negative is better. A split is worth taking when the two children can jointly remove more than the parent could, by more than the price γ of the extra leaf. The two penalties act differently: λ sits inside every denominator

and *shrinks* all leaf outputs and all three terms, most strongly where H is small; γ is a flat per-split threshold that *discourages* splits whose improvement is marginal. In one line: λ shrinks leaf outputs; γ discourages unnecessary splits.

Example 5.9.4 (Does the split pay for its leaf?)**.** Parent $(G, H) = (-4, 3)$, children $(G_L, H_L) = (-3, 1)$ and $(G_R, H_R) = (-1, 2)$, $\lambda = 1$, $\gamma = 0.2$. Parent: $w^* = 4/4 = 1$, $\phi = -\frac{1}{2} \cdot 16/4 = -2$. Left: $w^* = 3/2$, $\phi = -\frac{1}{2} \cdot 9/2 = -2.25$. Right: $w^* = 1/3$, $\phi = -\frac{1}{2} \cdot 1/3 = -0.167$. Children together -2.417; the data fit improves by 0.417; gain $= 0.417 - 0.2 = 0.217 > 0$, accept. The split would be rejected for any $\gamma \geq 5/12 \approx 0.417$.

The four leaf-value mechanisms side by side

Algorithm	Structure target (split search)	Leaf value mechanism
Basic boosting	raw error $y - \widehat{p}$	exact numerical line search
Unregularised GBM	raw error $y - \widehat{p}$	Newton step $\sum(-g)/\sum h$
Regularised GBM	raw error $y - \widehat{p}$	regularised step $\sum(-g)/(\sum h + \lambda)$
XGBoost	Newton target $r_{\text{XGB}} = -g/h$, weighted by h	same regularised step, found in the same weighted pass that chose the splits; gain includes γ

XGBoost unifies structure selection and weight optimisation into a single pass over one objective, which is both faster and better regularised.

Further controls and implementation

Beyond λ and γ, XGBoost offers *row subsampling* (a random fraction of observations per tree), *feature subsampling* (by tree, level or split, the random-forest idea inside boosting), *minimum child weight* (reject a split if either child's H is too small, which directly prevents the exploding step), and *tree-size limits* (maximum depth or leaves). Its speed comes from engineering: histogram-based split search, sparse and missing-value handling, parallel split evaluation across features, and GPU support. Boosting rounds themselves remain sequential.

Summary: XGBoost

- One objective: second-order loss approximation $+ \gamma Q + \frac{\lambda}{2} \sum w_q^2$, used for splits *and* outputs.
- Completing the square: a weighted least-squares problem with target $-g/h$ and weight h; their product $y - \widehat{p}$ is bounded.
- $w_q^* = -G_q/(H_q + \lambda)$; gain $= \frac{1}{2}[G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - G^2/(H + \lambda)] - \gamma$; accept if positive.
- λ shrinks outputs (continuous), γ charges a fixed price per extra leaf (discrete); row/feature subsampling and minimum child weight add further control.

5.10 Multi-Class Boosting and Other Libraries

What are we trying to get?

We want to extend logit boosting from two classes to K: one tree per class per round, class-specific gradients and Hessians, and the diagonal approximation of the softmax Hessian that lets the K trees be fitted separately. Then a map of the other names you will meet.

The softmax space

For K classes the ensemble maintains a vector of logits $F_1, \dots, F_K$ and maps it to probabilities with the softmax (Definition 4.2.4), $\widehat{p}_k = e^{F_k} / \sum_j e^{F_j}$. The loss is the multi-class cross-entropy with one-hot label y_k, $\ell = -\sum_k y_k \ln \widehat{p}_k$. Instead of one tree per round, the algorithm builds K trees per round, one per logit.

Proposition 5.10.1 (Class-specific derivatives)**.** *For one observation,*

$$g_k = \frac{\partial \ell}{\partial F_k} = \widehat{p}_k - y_k, \qquad \frac{\partial^2 \ell}{\partial F_k \, \partial F_r} = \begin{cases} \widehat{p}_k(1 - \widehat{p}_k) & k = r, \\ -\widehat{p}_k \widehat{p}_r & k \neq r. \end{cases}$$

Using only the diagonal, $h_k = \widehat{p}_k(1 - \widehat{p}_k)$, the XGBoost target for class k is $r_{\mathrm{XGB},k} = -g_k/h_k = (y_k - \widehat{p}_k)/(\widehat{p}_k(1 - \widehat{p}_k))$, formally identical to the binary case.

Proof. Write $\ell = -\sum_k y_k F_k + \ln \sum_j e^{F_j}$ (since $\ln \widehat{p}_k = F_k - \ln \sum_j e^{F_j}$ and $\sum_k y_k = 1$). Then $\partial \ell / \partial F_k = -y_k + e^{F_k} / \sum_j e^{F_j} = \widehat{p}_k - y_k$. Differentiating $\widehat{p}_k = e^{F_k} / \sum_j e^{F_j}$ with respect to F_r by the quotient rule gives $\widehat{p}_k(\mathbf{1}[k = r] - \widehat{p}_r)$, which is $\widehat{p}_k(1 - \widehat{p}_k)$ on the diagonal and $-\widehat{p}_k \widehat{p}_r$ off it. □

Reading the formula

The softmax *couples* the classes: raising F_1 lowers every other $\widehat{p}_k$, which is what the off-diagonal terms $-\widehat{p}_k \widehat{p}_r$ record. Keeping the full Hessian would make the K leaf values of one round depend on each other. Dropping the off-diagonal terms, $H \approx \mathrm{diag}(\widehat{p}_1(1 - \widehat{p}_1), \dots, \widehat{p}_K(1 - \widehat{p}_K))$, makes the approximate objective a *sum* of K independent binary-style objectives, so one tree can be fitted per class. The coupling is restored at the end of the round when the softmax is recomputed jointly from all updated logits.

Technique: One multi-class round

(1) From the current scores $F_{m-1,1}, \dots, F_{m-1,K}$ compute all class gradients and Hessians. (2) Fit one class-specific tree for each of the K separated objectives; these K fits, and the split evaluation inside each, can run in parallel. (3) Update all K scores and recompute the softmax probabilities before round $m+1$. Rounds stay sequential because round $m+1$ needs the updated probabilities. Multi-class boosting does K times the work per round; parallelism reduces elapsed time, not total work.

Other boosting approaches and libraries

Name	Type	Main emphasis
AdaBoost	a boosting *algorithm*	increases the weights of misclassified observations so that the next weak classifier focuses on them; the ensemble is a weighted vote
LightGBM	gradient-boosted-tree *library*	histogram-based splits and leaf-wise growth for large datasets
CatBoost	gradient-boosted-tree *library*	ordered treatment of categorical features designed to limit target leakage

Gotcha

AdaBoost is a different boosting *algorithm* (reweight observations, combine classifiers by weighted vote). XGBoost, LightGBM and CatBoost are *libraries* implementing gradient-boosted trees with different engineering choices. Do not list them as four algorithms of the same kind.

Summary: Multi-class boosting

- K logits, softmax, cross-entropy; K trees per round.
- $g_k = \widehat{p}_k - y_k$; the softmax Hessian couples classes through $-\widehat{p}_k\widehat{p}_r$; its diagonal $\widehat{p}_k(1-\widehat{p}_k)$ separates the K objectives.
- AdaBoost is an algorithm; XGBoost, LightGBM, CatBoost are libraries.

5.11 Bagging versus Boosting, and When Ensembles Do Not Help

What are we trying to get?

We want the two families side by side, a decision procedure that starts from the *source* of the error rather than from a favourite method, and the honest list of situations in which no ensemble will rescue a model.

Question	Bagging / random forest	Boosting
training	parallel randomised fits	sequential corrections
main mechanism	average unstable predictions (variance)	reduce the current loss (mostly bias)
typical trees	deep or moderately deep	shallow
diversity source	bootstrap rows, feature subsets	changing residuals or gradients
main controls	tree strength versus correlation (m_{try}, leaf size)	learning rate, depth, rounds
noise sensitivity	generally robust	can focus heavily on difficult or noisy cases
parallelisation	natural across trees	limited by sequential dependence

These are tendencies: either method can affect both bias and variance.

Technique: Start from the source of error

1. Is one strong learner *unstable* across training samples? Try bagging or a random forest.
2. Can many small corrections improve a *structured residual pattern*? Try gradient-boosted trees.
3. Do credible but different model families make *complementary* errors? Try validated voting or averaging.
4. Is the dominant error caused by *missing information* or *deployment shift*? An ensemble alone is unlikely to solve it.

Gotcha

Ensembles do not help when

- all models are missing the same important information (every error shares a direction; $\rho \approx 1$);
- the models are too similar and make the same mistakes;
- boosting starts fitting noise, outliers or incorrect labels (it treats every residual as signal);
- validation data are reused so often that the choices overfit them;
- test or deployment data differ substantially from the training data.

Additional models also cost computation, memory and system complexity, which Lab 3 measures explicitly.

Summary: Choosing

- Bagging and forests average parallel, randomised, deep fits; boosting sums sequential, shallow corrections.
- Diagnose first: instability, structured residuals, complementary families, or missing information and shift.
- No ensemble repairs shared missing information or a distribution shift.

5.12 Stacking

What are we trying to get?

Voting fixes the combination rule in advance. We want to *learn* it: a meta-model that takes the base predictions as inputs, and the one procedural rule that keeps this honest, out-of-fold predictions.

Definition 5.12.1 (Stacking)**.** Base models $m = 1, \ldots, M$ (e.g. logistic regression, a random forest, gradient boosting) produce predictions z_m, usually numbers or class probabilities. A *meta-model* h_γ maps them to the final prediction, $\hat{y} = h_{\hat{\gamma}}(z_1, \ldots, z_M)$. For binary classification with $z_m^{(i)} = \widehat{p}_m(y = 1 \mid x^{(i)})$

a common meta-model is logistic regression,

$$\widehat{p}^{(i)}_{\text{stack}} = \sigma\Big(\beta_0 + \sum_{m=1}^{M} \beta_m z^{(i)}_m\Big), \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} \ell(y^{(i)}, \widehat{p}^{(i)}_{\text{stack}}),$$

fitted on the true labels and on *out-of-fold* base predictions.

Reading the formula

Soft voting is the special case $\beta_m = 1/M$ (up to the sigmoid). Stacking lets the data decide the weights: a large positive β_m gives model m more influence; a small or negative one down-weights an unhelpful or redundant model. The meta-model is kept simple, a linear model or a logistic regression with regularisation, because it is fitted on only M inputs and must not overfit.

Training and prediction are different

Technique: Out-of-fold predictions

Training the meta-model. Split the training set into K folds. For each fold, fit every base model on the other folds and predict the held-out fold. Join the held-out predictions from all folds: every training observation now has a prediction $z^{(i)}_m$ from a model that never saw it. Fit h_γ on these $(z^{(i)}, y^{(i)})$.
Predicting a new observation. Refit each base model on *all* the training data, compute $(z_1, \ldots, z_M)$ for x_{new}, pass them through the fitted meta-model.

Gotcha

Never train the meta-model on *in-sample* base predictions. A deep tree or forest predicts its own training observations almost perfectly, so the meta-model would learn to trust the models that overfit the most, and its own fit would be unrealistically optimistic. This is leakage of training information into the combination rule. Out-of-fold predictions are the cure. They are not the same as OOB predictions (Definition 5.4.4): OOB uses the bootstrap by-product inside a bagged ensemble; out-of-fold uses an explicit fold split and works for any base model.

Summary: Stacking

- A meta-model learns one global combination of base predictions; logistic regression with regularisation is typical.
- Train it on out-of-fold predictions; predict with base models refitted on all data.
- In-sample base predictions leak and favour overfitting models.

5.13 Mixture of Experts

What are we trying to get?

Stacking learns one combination rule for all inputs. We want the architecture where the combination depends on x: a gate decides, for each input, how much to trust each expert, and gate and experts are trained together.

Definition 5.13.1 (Mixture of experts)**.** Experts $p_m(y \mid x)$, $m = 1, \dots, M$, are combined by a *gating model* $g_m(x) \geq 0$ with $\sum_m g_m(x) = 1$:

$$p(y \mid x) = \sum_{m=1}^{M} g_m(x)\, p_m(y \mid x).$$

A linear gate scores expert m by $a_m(x) = \beta_m^\top x + b_m$ (a small network for nonlinear routing) and normalises with a softmax, $g_m(x) = e^{a_m(x)} / \sum_r e^{a_r(x)}$. *Soft routing* uses all experts with their weights; *top-k routing* evaluates only the k experts with the largest scores.

Reading the formula

Compare with stacking's $\hat{y} = h(z_1, \dots, z_M)$: there the weights β_m are the same for every x; here $g_m(x)$ changes with the input, so one expert can own one region of feature space and another a different one. Top-k routing is also a computational device: with M large experts and $k = 2$, only two are evaluated per input (*conditional computation*), which is how very large language and vision models use mixtures of experts.

Technique: Joint training

Minimise the negative log-likelihood of the mixture,

$$-\sum_{i=1}^{N} \log \Big[\sum_{m=1}^{M} g_m(x^{(i)})\, p_m(y^{(i)} \mid x^{(i)}) \Big],$$

jointly over the gate parameters and the expert parameters. Experts improve on the observations that receive high gate weight; the gate learns which expert does well where; repeated joint updates make the experts specialise. The routing regions are not specified in advance.

Gotcha

Expert collapse. An expert that receives more gate weight early learns faster, becomes better, and attracts still more inputs, until one expert handles almost everything and the others are idle. Load-balancing penalties that encourage the gate to spread its weight are used to counter this.

Summary: Mixture of experts

- $p(y \mid x) = \sum_m g_m(x) p_m(y \mid x)$ with softmax gate weights that depend on x.

- Gate and experts trained jointly on the mixture likelihood; soft or top-k routing.
- Differs from stacking by input-dependent weights and joint training; risk of expert collapse.

5.14 Where Ensembles Are Used, and Lab 3

Method	Representative applications	Why it is used
Voting / averaging	medical prediction, sensor fusion, deep-network ensembles	combines different models; more stable predictions
Random forest	intrusion detection, remote sensing, bioinformatics, tabular problems	strong baseline with nonlinearity, interactions, little preprocessing
Gradient-boosted trees	credit scoring, fraud detection, churn, ranking on tabular data	high predictive accuracy from successive corrections
Stacking	data-science competitions, systems combining model families	learns when one model should complement another
Mixture of experts	large language and vision models, multi-domain prediction	specialisation and conditional computation

These are common examples; the appropriate method still depends on the data, the cost and the validation results.

Example 5.14.1 (Lab 3 at a glance)**.** Tree ensembles on the cybersecurity flow data of Labs 1 and 2, with one fixed stratified split and identical rows for every model.

Experiment	Models	Question
Binary classification	bagged trees, random forest	how much does aggregation stabilise the model?
Boosting	gradient boosting, XGBoost	how do rate, depth and rounds interact?
Model comparison	best Lab 2 models and both ensembles	which tree errors are corrected?
Multiclass	random forest, boosting	which attack classes remain difficult?
Port information	with and without destination port	does service information help?
Efficiency	fit time, prediction time, model size	what does higher accuracy cost?

Notes on the original slides

All numbers were rechecked: the variance $0.232\sigma^2$ for $M = 25$, $\rho = 0.2$; the Gini reductions 0.30 and 0.125 of the split example; the MDI credit 0.333 and the normalised MDI 0.75/0.15/0.10; the permutation increases 0.08/0.03/0.01; the boosting round $F_1(7.1) = 20.5$; the derivatives $g = \widehat{p} - y$, $h = \widehat{p}(1 - \widehat{p})$; and the gain formula. All are correct. Points to note when reading the deck:

- **Slide order versus concept order.** The slides introduce bagging (slides 16–22) before decision trees (23–27). This chapter defines trees first (§5.3) because bagging's tree-size hyperparameter and everything in random forests presuppose them.

- **Sign conventions.** The slides use *both* $r = y - \widehat{p}$ (the negative gradient, slides 51, 54) and $g = \widehat{p} - y$ (the gradient, slides 53, 57, 63, where it is written $g_k = -(y_k - \widehat{p}_k)$). They are consistent: $r = -g$. Always ask "gradient or negative gradient?" before reading a formula.
- **Slide 45, Step 5.** Only ID 3 is updated on the slide; the full update is $F_1 = (19, 20.5, 20.5)$ with new residuals $(-9, -0.5, 9.5)$, as in Example 5.7.4.
- **Slide 60, "Variance Target".** The XGBoost structure target is the Newton pseudo-target $r_{\text{XGB}} = -g/h$; "variance" refers to the weight $h = \widehat{p}(1-\widehat{p})$, the Bernoulli variance, not to the target itself.
- **Slide 62, "K independent trees per round".** They are independent only *within* a round and only under the diagonal Hessian approximation (slide 64); the rounds remain sequential.
- **Slide 65.** The three steps are run together with stray "item" markers; the intended list is the one in §5.10.
- **Slide 24.** The λJ penalty on the number of leaves is the same idea as XGBoost's γQ (slide 56), under a different letter; the slides' λ on slide 55 onwards is the L_2 penalty on leaf outputs.
- **Slide 6** appears three times in the PDF (an animation exported as separate pages); nothing is lost.

Chapter takeaways

Summary: Learning by combining models

- **Why:** one flexible fit has high variance; averaging competent models with *different* errors cancels part of it. Var $= \sigma^2(\rho + (1-\rho)/M)$: correlation leaves a floor.
- **Voting:** hard counts classes, soft averages posteriors and needs calibration.
- **Trees:** partitions with one constant per leaf (mean/median/majority); greedy Gini splits; flexible but high-variance.
- **Bagging:** bootstrap samples (63.2% in, 36.8% out); OOB error is free internal validation, not a test set.
- **Random forests:** m_{try} random candidates per split decorrelate trees; tune at forest level; trees until the OOB plateau.
- **Importance:** MDI (training, fast, biased) and permutation (prediction, honest); masking by correlation; never causality.
- **Boosting:** $F_m = F_{m-1} + \eta T_m$; fit the negative gradient; squared error $\Rightarrow$ residuals; M, η, depth tuned jointly with early stopping.
- **Classification:** boost the logit; $g = \widehat{p} - y$, $h = \widehat{p}(1-\widehat{p})$; Newton leaf $\sum(-g)/(\sum h + \lambda)$.
- **XGBoost:** one regularised second-order objective for splits and outputs; $w^* = -G/(H+\lambda)$; gain minus γ per split.
- **Multi-class:** K trees per round, diagonal Hessian separates classes.

- **Stacking** learns a global combination from out-of-fold predictions; **mixture of experts** learns an input-dependent one jointly with the experts.
- **Limits:** shared missing information, identical models, noisy labels, reused validation data, deployment shift.